

FACULTAD DE CIENCIAS
GRADO EN FÍSICA
TRABAJO FIN DE GRADO
CURSO ACADÉMICO [2025-2026]

TÍTULO:

**ALEATORIEDAD CON COMPUTACIÓN CUÁNTICA:
VIABILIDAD Y COMPARACIÓN CON MÉTODOS DE
ALEATORIEDAD EN COMPUTACIÓN CLÁSICA.**

AUTOR:

LUCAS NICOLÁS HERNÁNDEZ BELLÓN

Resumen

En este TFG se evalúa la viabilidad de producir QRNG a partir de Computadores Cuánticos disponibles en el mercado. Para ello se escogen seis modelos de RNG en los que se evalúa la aleatoriedad producida por secuencias binarias. Cuatro son QRNG ofertados por AWS e IBM, con distintos principios físicos (superconductores, iones atrapados y átomos neutros). Los dos restantes son PRNG. Uno es el generador de NumPy (PCG64, de la familia de PRNG más usados). El otro es BcryptGenRandom, un CSPRNG de Microsoft usado en sus sistemas operativos.

Primero se define la aleatoriedad de una secuencia, sustentándose en los principios de *impredictibilidad*, *uniformidad* e *independencia*. Tras ello, se modeliza cada propiedad por separado. La impredictibilidad estará relacionada con la Entropía de Shannon. La uniformidad e independencia se evaluarán a partir de Tests Estadísticos y con una estimación de la Complejidad de Kolmogorov, definida en la Teoría de la Información Algorítmica. Una vez planteado el fundamento matemático se desarrolla el fundamento de los computadores usados, basándose en el modelo de puertas cuánticas y en el modelo AHS para el computador de átomos neutros. Se ha diseñado y programado una librería independiente para extraer secuencias de cada RNG y analizar su aleatoriedad. Se extraen secuencias en binario para evitar los problemas de truncado de números reales, que añadirían ruido al análisis. Se han obtenido y analizado más de dos mil millones de bits de QRNG y mil millones y medio de bits de PRNG. Para aseverar la fiabilidad de los tests de primer orden, pese al ruido generado por la aproximación numérica de las distribuciones teóricas, se aplica un test de Segundo Orden basado en Kolmogorov-Smirnov.

Se ha concluido que la viabilidad utilizar computadores cuánticos para producir QRNG se encuentra al límite de los estándares, si bien dichas circunstancias pueden cambiar a corto plazo. Los computadores de átomos neutros basados en el modelo AHS no están capacitados para generar QRNG debido al efecto de interacción entre qbits. Los computadores de iones atrapados presentan cualidades prometedoras, pero su alto coste ha sido limitante para analizar sus capacidades. Los computadores de superconductores presentan los mejores resultados, siendo más rápidos y baratos que sus competidores. Éstos, además, presentan mejores estándares de calidad a PRNG en independencia, iguales en impredictibilidad, pero inválidos para uniformidad al detectarse asimetrías entre 0s y 1s. Es posible que puedan mejorarse los resultados de impredictibilidad minimizando los errores de lectura en los qbits, así como los errores de las puertas cuánticas. Ésto puede conseguirse mejorando la tecnología, o sacrificando coste en tiempo y dinero por mayor fiabilidad al usar menos qbits.

La librería está disponible para su uso en el Anexo, de tal forma que se pueden revisar los resultados o continuar investigando con las propuestas planteadas en las conclusiones.

Abstract

This Undergraduate Thesis studies the viability of generating QRNG using the Quantum Computers currently available. We chose six different RNG to which we evaluate the randomness in their sequences. Four of them are QRNG offered by AWS and IBM, based on different physical principles (superconductors, trapped ions and neutral atoms). The remaining two are PRNG. One of them is the NumPy Random (PCG64, one of the most used PRNG family). The other one is BcryptGenRandom, a CSPRNG of Microsoft used in their OS.

Firstly we define the randomness of a sequence by the concepts of unpredictability, uniformity and independency. The unpredictability of a sequence will be related to Shannon's Entropy. Uniformity and independency will be evaluated by Statistical Testing and Kolmogorov Complexity estimations, which will be defined by the Algorithmic Information Theory. Secondly the fundamentals of the Quantum Computers will be introduced based on Quantum Gates and the AHS model for the neutral atoms QC. We developed a public repository of code able to extract sequences of each RNG and analice their randomness. We decided to extract the sequences in binary in order to avoid inaccuracies from slicing into floats. Lastly, we obtained and analyzed more than two thousand million bits of QRNG, as well as a thousand and a half million bits of PRNG. In order to evaluate the uncertainties of the First Order Tests (due to unavoidable approximations of computing theoretical statistical distributions) we implemented a Second Order Kolmogorov-Smirnov Test.

We concluded that producing reliable QRNG by direct extraction of bits from the QCs is in the margins of being viable. However, it is possible than in some years the situation will improve. Computers based on Neutral Atoms technology are and won't be able to produce reliable QRNG due to the van der Waals interactions between their qbits. Trapped computers offer promising results, even though their high cost disabled us to make a thorough testing. The superconducting computers, lastly, show the best results, with both fast bits generation as well as cheapest costs and the best st0atistical results. QRNG based on superconducting computers present better independency than PRNG, around the same unpredictability but worse uniformity, to the point that does not comply with the Tests' minimuns. It is, on the other hand, possible that they will reach the standart's by diminishing the decoherence noise, principally by reducing readout error, as well as one-qbit gates error. Those goals could be achieved by bettering the technology or by reducing the qbits used (thus trading fidelity with higher costs and slower bit generation).

The code's repository is public and is linked in the Appendix so that the results can be consulted and recreated. It is also self-contained, so it can be used to further the research with the proposals given by the conclusion.

Índice

1. Introducción	6
1.1. Objetivos	6
1.2. Motivación	6
1.3. Metodología	8
1.4. Estructura del Trabajo	9
2. Fundamento Teórico	10
2.1. Impredictibilidad como Entropía del sistema	11
2.2. Tests Estadísticos: Evaluando la independencia y uniformidad	15
2.2.1. AIT: Complejidad de Kolmogorov y la incompresibilidad de secuencias	17
2.2.2. Familia de Tests Estadísticos	22
2.2.3. Tests de Segundo Orden	25
2.2.4. NIST Test Suite	28
2.3. Computación Cuántica	29
2.3.1. Computadores Cuánticos	29
2.3.2. Puertas cuánticas	30
2.3.3. Fuentes de error	32
2.3.4. Modelo AHS	34
2.3.5. Circuito usado para AHS	36
3. Evaluación de la Aleatoriedad	38
3.1. Procedimiento	38
3.1.1. Extracción de secuencias	39
3.1.2. Coste Económico de los Computadores	40
3.1.3. Estimación de entropía	41
3.1.4. Estimación de Complejidad de Kolmogorov	42
3.1.5. Tests Estadísticos	43
3.1.6. Tests de Segundo Orden	43
3.1.7. Estudio de la tasa de generado	44
3.2. Resultados	44
3.2.1. Entropía	44
3.2.2. Complejidad	45
3.2.3. Aplicación de NIST test suite	45
3.2.4. Tests de Segundo Orden	48
3.2.5. Tasa de Generado y Costos	48
4. Conclusión	49
5. Anexos	51
5.1. Algunos tiempos de Decoherencia	60

Referencias

86

1. Introducción

1.1. Objetivos

El propósito de este TFG es estudiar la viabilidad de los computadores cuánticos a la hora de generar números aleatorios impredecibles, uniformes y con una tasa de generación útil en pos de responder a la siguiente pregunta:

¿Hasta qué punto es viable la creación de números aleatorios robustos a partir de los computadores cuánticos disponibles actualmente en el mercado?

1.2. Motivación

La generación de números aleatorios (RNG) es una de las herramientas fundamentales en la tecnología e investigación científica. Un claro ejemplo es el campo de la criptografía, completamente dependiente del uso de buenos generadores en la creación de *keys*. Sin ir más lejos, los sistemas operativos como *Windows*, *Linux* o *MacOS* llevan incorporados generadores de números cuya calidad puede evaluarse [9]. En el campo de la modelización científica, por otro lado, tenemos la amplia gama de métodos Monte Carlo, cuya orden fundamental (aceptar o rechazar un cambio infinitesimal en el sistema simulado) se sustenta en la generación de números aleatorios.

A la hora de diseñar generadores de números aleatorios es importante considerar tres propiedades: La *aleatoriedad* del mismo, la *dispersión estadística* de los números generados y, finalmente, la *tasa de generación* de números. La aleatoriedad de una secuencia de RNG impone que el output no pueda ser predecible, mientras que la dispersión estadística implica que los números obtenidos se adecúen a la distribución estadística deseada. Finalmente, la tasa de generación nos indica el ritmo de salida de números aleatorios utilizables a un tiempo dado [53]. Estas propiedades se estudian con modelización matemático-computacional como Teoría de la Información (estimando la entropía del sistema) y Teoría de la Información Algorítmica (estudiando la complejidad asociada a los datos obtenidos), y mediante pruebas de estrés como las NIST [12] basadas en inferencia estadística. Pese a ello, se debe tener en cuenta que estas propiedades son muy difíciles de generalizar, y su caracterización es aún un problema abierto en muchos sistemas que usan algoritmos muy diversos [37].

Existen distintas técnicas de generación de números aleatorios. Actualmente se pueden clasificar en dos ramas fundamentales. Los métodos que se basan en una pequeña toma de aleatoriedad en forma de *semilla*, para luego ampliarla como secuencias de números aparentemente aleatorios mediante técnicas algebraicas se denominan *Generadores de números pseudoaleatorios* (PRNG). En cambio, los métodos que se basan en la adquisición constante de aleatoriedad mediante sistemas físicos se denominan *True Random Number Generators* (TRNG) [53].

Los generadores pseudoaleatorios presentan una buena tasa de generación de números. Además, son muy versátiles por la alta gama de distintos algoritmos de generación cons-

truidos. Actualmente son los más usados en simulaciones Monte Carlo, además de estar implementados en librerías como NumPy de Python¹, entre otras. La figura 1 presenta algunos de los PRNG disponibles. Estas ventajas han hecho que tengan un papel protagonista durante el auge de la modelización científica, y ciertamente se han presentado adecuados para simulaciones a pequeña y mediana escala [37]. Sin embargo, suponen una amenaza creciente en sistemas donde se requieran cantidades abismales de números aleatorios. La razón es que son deterministas: Dada una misma semilla, ningún algoritmo ejecutado con una memoria finita es capaz de evitar periodicidad en la secuencia [46]. Además, se corre el riesgo de que los algoritmos usados para el generado sean defectuosos y presenten un patrón explotable, por sutil que sea. Un caso reciente se ha dado con el algoritmo xorshift128+ en 2019, donde se ha visto que el uso de la suma aritmética causa correlaciones entre los números [32]. Marsaglia, a su vez, demostró ya en 1968 que todos los Generadores de Lehmer, o *Linear Congruential Generators* LCG (basados en la multiplicación con módulo de enteros) presentan una falla fatal de correlación entre outputs [54]².

	Statistical Quality	Prediction Difficulty	Reproducible Results	Multiple Streams	Period	Useful Features	Time Performance	Space Usage	Code Size & Complexity	k-dimensional Equidistribution
PCG Family	Excellent	Challenging	Yes	Yes (e.g. 2^{63})	Arbitrary	Jump ahead, Distance	Very fast	Very compact	Very small	Arbitrary*
Mersenne Twister	Some Failures	Easy	Yes	No	Huge 2^{19937}	Jump ahead	Acceptable	Huge (2 KB)	Complex	623
Arc4Random	Some Issues	Secure	Not Always	No	Huge 2^{1699}	No	Slow	Large (0.5 KB)	Complex	No
ChaCha20†	Good	Secure	Yes	Yes (2^{128})	2^{128}	Jump ahead, Distance	Fairly Slow	Plump (0.1 KB)	Complex	No
Minstd (LCG)	Many Issues	Trivial	Yes	No	Tiny $< 2^{32}$	Jump ahead, Distance	Acceptable	Very compact	Very small	No
LCG 64/32	Many Issues	Published Algorithms	Yes	Yes 2^{63}	Okay 2^{64}	Jump ahead, Distance	Very fast	Very compact	Very small	No
XorShift 32	Many Issues	Trivial	Yes	No	Small 2^{32}	Jump ahead	Fast	Very compact	Very small	No
XorShift 64	Many Issues	Trivial	Yes	No	Okay 2^{64}	Jump ahead	Fast	Very compact	Very small	No
RanQ	Some Issues	Trivial	Yes	No	Okay 2^{64}	Jump ahead	Fast	Very compact	Very small	No
XorShift* 64/32	Excellent	Unknown?	Yes	No	Okay 2^{64}	Jump ahead	Fast	Very compact	Very small	No

* For the PCG family, arbitrary k -dimensional equidistribution (and the huge periods it implies) requires PCG's extended generation scheme.

† ChaCha entry based on [an optimized C++ implementation](#) of ChaCha, kindly provided by Orson Peters.

Figura 1: Algoritmos de PRNG disponibles al público más populares [62].

Por otro lado, los TRNG no son deterministas y parten de aportar aleatoriedad fundada en sistemas físicos. Esta propiedad es muy potente, pero se ha de ir con cuidado: En algunos de los sistemas que se usan se supone aleatoriedad debido a nuestro modelo parcial del

¹Numpy en concreto usa por defecto la familia de algoritmos PCG. [69]

²Marsaglia diseñó el algoritmo xorshift en 2003, cuya vulnerabilidad descubrieron Haramoto y Matsumoto en 2019. En el caso de que ambos decidan diseñar otro PRNG, esperaremos con sumo interés comprobar si las vulnerabilidades se siguen descubriendo en tiempos cada vez menores.

mismo. Esto ocurre en especial en los sistemas basados en ruido de calor o atmosférico. En ellos es muy sencillo caer en una falacia *ad ignorantiam*, cuando avances fisico-matemáticos podrían volver esos sistemas deterministas. Sin ir más lejos, en 2025 los matemáticos Filipis y Giuseppe han realizado un importante avance en la resolución de EDOs capaces de describir el ruido atmosférico con más precisión [26].

Existe un tipo de TRNG donde la aleatoriedad es un pilar de la teoría física, además de ser cuantificable teóricamente. Estos son los basados en Mecánica Cuántica, denominados *Quantum Random Number Generators* (QRNG). Idealmente estos sistemas generan cadenas de números aleatorios fundamentados en los Postulados de la Mecánica Cuántica, y serían lo más próximo a una aleatoriedad no simulada a lo que podríamos aspirar. Es por ello que hay un interés creciente en el estudio y aplicabilidad de los QRNG.

Encontramos dispositivos especializados en esta generación de números aleatorios, habiendo un especial interés en los basados en sistemas ópticos [53]. Sin embargo, existen alternativas con mayor aplicabilidad en la mayoría de disciplinas, como pueden ser las finanzas, la modelización numérica, o la criptografía. Los computadores cuánticos presentan una oportunidad estupenda para generar RNG mediante la evaluación de Qbits, además de poder cumplir otras tareas. De igual forma hay mayor disponibilidad de los mismos gracias a las herramientas ofrecidas por IBM, AWS y otras empresas.

1.3. Metodología

Primero realizaremos un estudio bibliográfico de las herramientas usadas para evaluar aleatoriedad. Esto incluye un barrido sobre Teoría de la Información y Teoría de la Información Algorítmica para entender cómo se evalúa la aleatoriedad de un sistema; la teoría de inferencia estadística aplicada a tests de aleatoriedad, y los tests disponibles hasta el momento. Dado que la aleatoriedad es un concepto complejo de definir y aún abierto a debate, trataremos de aportar claridad en la definición a partir de los conceptos de *impredecibilidad*, *independencia* y *uniformidad*. Conectaremos estos conceptos a los modelos de aleatoriedad presentes en la bibliografía. Esta sección incluirá, a su vez, una introducción somera a la Mecánica Cuántica y a los fundamentos de los computadores cuánticos usados. De esta forma, podremos contrastar la aleatoriedad teórica que podrían alcanzar con lo que seamos capaces de extraer en este trabajo, y entender dónde puede presentar limitaciones la metodología empleada. Será en este momento donde presentaremos, el circuito cuántico usado para extraer secuencias de números aleatorios.

Usaremos cuatro computadores cuánticos de las empresas IBM, Rigetti, QuEra y AQT³ para aplicar este circuito, y compararemos los resultados entre sí así como con un simulador de un ordenador cuántico y un generador criptográfico estándar (PRNG). Analizaremos la entropía y complejidad estimada en cada sistema, así como su respuesta a la suite NIST y otros tests de orden superior. También evaluaremos la tasa de generado de cada uno y

³Inicialmente se pensó en usar IonQ por la mayor cantidad de qbits de su computador *Forte-1*. Sin embargo, una avería en su computador lo dejó inhabilitado hasta el 22 de junio de 2026.

los costes monetarios en pos de matizar su viabilidad. Finalmente, construiremos y aplicaremos una librería de código para evaluar la calidad de RNG de cada modelo, así como exploraremos posibles mejoras del análisis.

1.4. Estructura del Trabajo

En el Fundamento Teórico desglosaremos las herramientas en las que se sustentará la investigación. Estas son nuestra definición de aleatoriedad, la *entropía de un sistema*, el fundamento de los tests estadísticos y una descripción de los que vamos a usar. Además, en ella recopilaremos información útil para las secciones posteriores que se citará a menudo. A lo largo de la sección 2.3 comentaremos cómo se ha diseñado el circuito cuántico para la obtención de las secuencias, así como las fuentes de error que se han considerado.

En la sección 3 analizaremos las secuencias a partir de las herramientas de la sección 2. También estimaremos la tasa de generado de cada computador, descartando el tiempo de conexión a los mismos. Finalmente, razonaremos qué pesos puede tener cada característica (cuántica y no cuántica) del sistema en la aleatoriedad.

En la conclusión indicaremos qué secuencias cumplen con nuestra definición de aleatoriedad, e indicaremos qué dispositivo presenta la mejor calidad de RNG y por qué. También plantearemos otros caminos para verificar estos resultados, o cómo se podría ampliar el programa.

2. Fundamento Teórico

La definición de una *secuencia de números aleatoria* es difícil de determinar. Esto se debe, en parte, a que esta propiedad incluye comportamientos antiintuitivos para nosotros, además de haber sido un tema evitado en la teoría de probabilidad y la estadística [44]⁴. Pongamos por ejemplo la tirada de una moneda no trucada en la que se admiten los resultados *cara* (H) y *cruz* (T), con probabilidades $p = \frac{1}{2}$ para cada uno. Este sistema es uno de los casos más sencillos de presentar aleatoriedad. Sin embargo, para muchas personas la secuencia de n tiradas $S(n)$:

$$S(n) = \{H_1, H_2, H_3, \dots, H_n\} \quad (1)$$

No sería aleatoria. Es muy probable que nuestra poca intuición general en probabilidad sea parcialmente culpable [85], pues las secuencias aleatorias pueden comportarse como no aleatorias. Así, una inspección superficial de una secuencia nunca será suficiente para determinar su naturaleza. Debemos, pues, hacer un estudio previo de qué vamos a considerar aleatorio, y cómo lo vamos a estimar. La primera cuestión, más bien filosófica, la resolveremos en los próximos párrafos. La segunda quedará resuelta cuando modelicemos debidamente las propiedades de *impredictibilidad*, *uniformidad* e *independencia* en una secuencia de números.

En la literatura encontramos múltiples definiciones de aleatoriedad para secuencias de números. Por ejemplo, podemos enfocarla desde el punto de vista de la incompresibilidad de una cadena aleatoria (definición de Martin-Löf [57] y de Kolgomorov [48]), pero también podemos hacerlo como una muestra de números que aproximan la distribución uniforme $U(0, 1)$ (Definición de L'Ecuyer) [48]. Una recopilación de las distintas teorías de probabilidad y de lógica algorítmica que tratan de definir el concepto se encuentra en la conferencia de Downey y Hirschfeldt [23]. En el caso de este trabajo nos basaremos en una definición parcial propuesta por Lehmer, pero recogida por Kurth [44]⁵.

Lehmer determina que una secuencia de números es aleatoria cuando cada elemento es impredecible, además de ser capaz de superar los tests propuestos para cumplir una tarea específica. Queremos destacar tres puntos respecto a la definición:

- No se menciona que los números deban ser independientes de los anteriores, pese a que sea una cualidad a menudo impuesta en la bibliografía [48][10][12][46]. Se puede inferir de la impredictibilidad de la secuencia, pero algunos PRNG como el de Lehmer o xorshift crean los números a partir de relaciones recursivas [54], [64]⁶. Nosotros vamos a imponer explícitamente que los números sean independientes entre ellos.

⁴Como nota de interés, los axiomas de la probabilidad están enfocados de tal forma que la teoría de la probabilidad sea utilitaria. Así es capaz de describir las probabilidades en un sistema aleatorio, pero no es capaz por sí misma de determinar qué es la aleatoriedad [44].

⁵Una definición más formal se encuentra en el apéndice de la misma referencia. Dicha definición es robusta y se ampara en teoría de la medida y en parte de las ideas ya presentadas por Kolgomorov, así como perfecciona la definición intuitiva presentada por Lehmer. Sin embargo, hemos considerado que en el contexto de esta sección introductoria una definición más informal es suficiente.

⁶También es cierto que esta imposición de independencia se suele imponer sobre todo en los TRNG, y son un punto más a su favor: presentan condiciones de aleatoriedad más estrictas que los PRNG.

- Desde este punto de vista, la aleatoriedad puede interpretarse como una propiedad circunstancial. Este enfoque es igual al de pruebas estadísticas que discutiremos más adelante, como las NIST [12]. A su vez, esta visión también justifica por qué asegurar la aleatoriedad de los RNG tiene distintas prioridades según la disciplina: La definición de aleatoriedad puede ser más estricta en criptografía que en modelización científica, y por ello en la segunda se ha tardado más en plantear la necesidad de mejorar los generadores usados.

Otro ejemplo en el que se evidencia la subjetividad inherente a la definición de aleatoriedad es el caso de los LCG ya mencionados. Aunque se encontró que los números generaban planos al aplicarse a un mapa en dos o más dimensiones, se siguieron utilizando en algunas disciplinas siempre y cuando superasen el Test Espectral, diseñado específicamente para ellos [49].

- Finalmente, para tener una conexión más sólida con las distribuciones de probabilidades que podemos determinar, vamos a modificar ligeramente la definición de la aleatoriedad de Lehmer introduciendo el matiz de la semejanza con la distribución $U(0, 1)$.

Queda, pues, clarificado que el concepto de aleatoriedad es subjetivo. Nuestra definición de aleatoriedad será:

Una secuencia de números será aleatoria cuando cada elemento sea **impredecible, uniforme e independiente** del resto. Esto significa que la secuencia tendrá que superar una serie de tests estadísticos que prueban dichas propiedades, y la distribución de probabilidades asociada a la secuencia deberá asemejarse a la distribución de probabilidad uniforme $U(0, 1)$.

De esta forma, las conclusiones que se alcancen en este proyecto estarán contenidas a la definición de aleatoriedad usada, y a los tests a los que se hayan sometido las secuencias. Nótese que esta definición de aleatoriedad corresponde a la que se suele atribuir para secuencias de bits dadas por una distribución de Bernoulli equiprobable (el caso discreto de la uniforme) dentro de un espacio métrico [75] [2].

Veamos ahora cómo podemos cuantificar la impredecibilidad a partir de Teoría de la Información.

2.1. Impredictibilidad como Entropía del sistema

La Teoría de la Información es la rama de las matemáticas que enuncia y modeliza el concepto de *información de un sistema*. Un sistema bajo este formalismo puede ser un generador de secuencias de números. La información de este sistema, por otro lado, está relacionada con las probabilidades asociadas a cada secuencia. Dado que la aleatoriedad está relacionada con nuestra incapacidad de predecir qué secuencia va a devolver el generador,

es claro que nos interesa modelizar primero el concepto de información. A continuación haremos un tratamiento matemático de la cuestión, planteando las definiciones y propiedades básicas de la teoría de la probabilidad que nos permitirán definir la información⁷. Esta información, finalmente, la relacionaremos con la propiedad de impredecibilidad mencionada en la definición de aleatoriedad.

Definición 2.1 (Nomenclatura de probabilidad).

Usaremos las siguientes expresiones para definir los aspectos más comunes de probabilidad y teoría de la información:

- Denominaremos *evento* o *sistema*⁸ a cualquier enunciado que pueda tener distintos resultados. Se denotará por letras latinas mayúsculas (A).
- Denominaremos *suceso* o *estado* al resultado posible de un evento. Se denominarán con la letra minúscula que defina al evento y un índice latino. Así, el evento A tiene un conjunto de sucesos $\{a_i\}_{i \in \mathbb{N}}$.
- Denotaremos *Probabilidad de que ocurra el suceso a_i* como $P(a_i)$. Se debe imponer la normalización de las probabilidades:

$$P(a_i) \in [0, 1] \mid \sum_{i \in \mathbb{N}} P(a_i) = 1 \quad (2)$$

- Dados dos sucesos a_i y b_j , denotaremos la *probabilidad de que ocurra a_i y b_j* como $P(a_i, b_j)$. Lo denominaremos *probabilidad dual de a_i y b_j* . Nótese que es simétrica: $P(a_i, b_j) = P(b_j, a_i)$. Nótese también que, por definición, se cumple que:

$$P(a_i) = \sum_{j \in \mathbb{N}} P(a_i, b_j) \quad P(b_j) = \sum_{i \in \mathbb{N}} P(a_i, b_j) \quad (3)$$

Además, si los eventos A y B son independientes la probabilidad dual pasa a ser el producto de las probabilidades individuales.

- Denotaremos la *probabilidad de que ocurra b_j habiendo ocurrido a_i* como $P(b_j|a_i)$. Nótese que, en este caso, antes de realizar el evento B conocemos de antemano el resultado de A , mientras que en la definición anterior no se conoce ninguno previamente. Además, esta probabilidad no tiene por qué ser simétrica. Este tipo de probabilidades se denominan *Probabilidades Condicionadas*.

La probabilidad condicionada es fundamental en teoría de la información por ser la forma de modelizar cómo el conocimiento de un suceso afecta a la probabilidad de otros.

Teorema 2.2 (Relación entre la probabilidad condicionada y la probabilidad dual).

Sean A y B dos eventos no excluyentes entre sí. La probabilidad dual y condicionada entre dos sucesos a_i y b_j cualesquiera están relacionadas como:

$$P(b_j|a_i) = \frac{P(b_j, a_i)}{P(a_i)} \quad P(a_i|b_j) = \frac{P(b_j, a_i)}{P(b_j)} \quad (4)$$

⁷Las demostraciones de los teoremas se encuentran en el Anexo.

⁸Usaremos la nomenclatura de *sistema* más a menudo cuando estemos con sistemas físicos. Para el formalismo matemático tenderemos a usar *evento*.

Se puede interpretar como que la probabilidad de que ocurran ambos sucesos no es más que la probabilidad de que haya ocurrido uno dado el otro, teniendo en cuenta que se ha debido de dar el primero para empezar.

Observación 2.3. Podemos generalizar estas definiciones y resultados a probabilidades con N eventos $\{A_i\}_{i=1,2,\dots,N}$, basta con definir eventos a pares $B_{ij} = A_i \otimes A_j$ para acabar recuperando expresiones con eventos A y B .

Con estos conceptos podemos plantearnos qué es la *información* asociada a un evento, y cómo se relaciona con las probabilidades de cada suceso.

Definición 2.4 (Información de un suceso).

Dado un evento con sucesos $\{a_i\}_{i \in \mathbb{N}}$. Denotamos la *información que nos aporta que A se determine en a_i* como:

$$h[P(a_i)] \equiv -\log_{(2)}(P(a_i)) \quad (\text{bits}) \quad (5)$$

Donde, a partir de ahora, $\log$ corresponderá a logaritmo en base dos.

En la sección de ampliación del Anexo incluimos una justificación a la expresión de la información.

Definición 2.5 (Información de un evento).

Dado un evento A con sucesos $\{a_i\}_{i \in \mathbb{N}}$. Denotamos la *información total que nos aporta el evento A* como:

$$H(A) = \sum_{i \in \mathbb{N}} P(a_i) h[P(a_i)] = - \sum_{i \in \mathbb{N}} P(a_i) \log(P(a_i)) \quad (6)$$

Nótese que esta expresión de la información es muy parecida a la entropía de Gibbs de Mecánica Estadística, salvo la constante de Boltzmann k_B [41]. Esto no es ninguna casualidad, y es por ello que muchas veces a la información de un sistema también se le denomina *entropía del sistema* y, formalmente, *Entropía de Shannon*. Nosotros usaremos los términos indistintamente.

A continuación vamos a presentar un ejemplo muy sencillo para evidenciar la información que son capaces de aportar las distribuciones de probabilidad. Además, el sistema que vamos a plantear será muy útil cuando tratemos de generar secuencias de números aleatorios a partir de los bits obtenidos por computación cuántica.

Ejemplo 2.6 (Entropía de un bit).

Sea un bit preparado en $a_0 = 0$ con probabilidad p . Si el evento A es comprobar en qué estado está el bit, tenemos los siguientes sucesos posibles:

$$\{a_i\} = \{a_0 \equiv \text{El bit está en } 0, a_1 \equiv \text{El bit está en } 1\} \quad (7)$$

Con probabilidades $P(a_0) = p$ y $P(a_1) = 1 - p$ por normalización. La entropía del sistema será de la forma:

$$H(A) = -p \log(p) - (1 - p) \log(1 - p) \quad (8)$$

La figura 2 presenta la entropía del sistema para distintas probabilidades p . Podemos ver que el máximo de la entropía se produce cuando más incertidumbre tenemos respecto al

estado del bit. Es decir, vamos a obtener la mayor cantidad de información del sistema cuanto menos conozcamos del mismo previamente. Es decir, cuando las probabilidades estén uniformizadas. Esta idea es clave para entender cómo la impredecibilidad se puede estimar con una cota inferior de la entropía.

Otro detalle a destacar es que este sistema tiene justo como entropía máxima $H(A) = 1$. Por ello este sistema actúa como medida de información, y la misma tiene unidades de *bits* en teoría de la información.

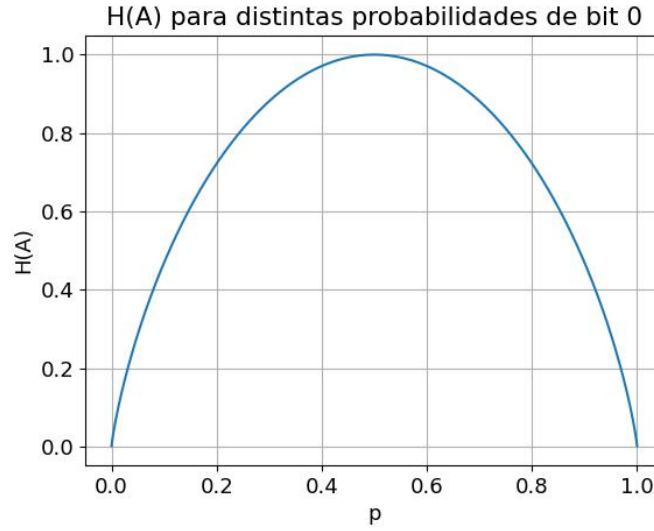


Figura 2: Valor de la entropía para distintas probabilidades de tener un bit en 0

Teorema 2.7 (Algunas propiedades de la información).

Sea $H(A)$ la entropía de un evento A con sucesos $\{a_i\}_{i \in \mathbb{N}}$. Se cumple que:

1. El valor máximo de $H(A)$ se alcanza cuando los sucesos son equiprobables. Dicho valor es $H_{\max}(A) = \log(n)$, con n el número posible de sucesos.
2. Si nivelamos las probabilidades de los sucesos con una transformación $P(a_i) \rightarrow P'(a_i)$, la entropía resultante $H'(A)$ será mayor o igual a la original. Dicho formalmente:

$$\text{Sea } P'(a_i) = \sum_{j \in \mathbb{N}} \lambda_{ij} P(a_j) \mid \sum_j \lambda_{ij} = \sum_i \lambda_{ij} = 1 \text{ y } \lambda_{ij} \geq 0 \quad \forall i, j \quad (9)$$

$$\text{Entonces } H'(A) \geq H(A)$$

Observación 2.8. La propiedad 1 será extremadamente útil en estimar la aleatoriedad de un sistema, ya que la aleatoriedad impone que los sucesos sean equiprobables y, por ende, dichos sistemas deberán tener la entropía máxima. La propiedad 4 está relacionada con el principio de Máxima Entropía, ya que la naturaleza tiende a uniformizar las probabilidades en sistemas aislados.

Con todo el aparatage matemático engrasado⁹, ya podemos plantear una definición más utilitaria de la información y cómo ésta se relaciona con la aleatoriedad.

⁹Algunas cuestiones pendientes, pero poco relevantes para el desarrollo del TFG, se encuentran en el Anexo.

La información de un sistema es una medida de la incertidumbre que presenta dicho sistema antes de ser medido. Además, conociendo un observable del sistema, podemos usarlo como ligadura para buscar las probabilidades de los estados que maximizan la entropía del sistema. Esto es así gracias a que la información se almacena en la configuración de los sistemas físicos. Esta forma de determinar las probabilidades de un sistema se denomina *Método de Máxima Entropía* [10], y puede resultar muy útil para, dadas algunas probabilidades y un observable, estimar la entropía máxima que puede presentar el sistema.

Por ejemplo, en una secuencia aleatoria de números la distribución de probabilidades es la equiprobable $U(1, 0)$. Si esta secuencia aleatoria es de n números, una estimación teórica de la entropía máxima que puede tener una secuencia aleatoria es la dada por la expresión del teorema 2.7:

$$H_{max}(n) = \log(n) \quad (10)$$

Para poder realizar esta estimación hemos tenido que imponer que la distribución sea la equiprobable lo que, en los casos reales, puede no cumplirse. Por ello es necesario diseñar cotas superiores más precisas a partir de tests estadísticos. Nosotros podremos estimar una cota superior más sencilla gracias al ejemplo 2.6.

También es necesario estimar una cota inferior de la entropía H_{min} . Este valor se podrá relacionar con el número de la secuencia con mayor frecuencia. Es decir, el que presenta mayor desviación de la distribución uniforme y por ende de nuestra definición de aleatoriedad. Esta cota también es un problema abierto, y hay esfuerzos constantes en intentar determinarla para QRNG [19] [51].

A modo de conclusión: La entropía nos será útil porque nos permite estimar probabilidades medias de todos los sucesos posibles, así como las probabilidades críticas. Ambas, a su vez, nos sirven para comprobar si se cumplen las propiedades de impredecibilidad y, parcialmente, de uniformidad que mencionamos en nuestra definición de aleatoriedad. Una mayor entropía indicará mayor uniformidad entre las probabilidades y, con ello, una mayor impredecibilidad. Dado que el circuito cuántico generará un sistema muy semejante al ejemplo 2.6, la entropía ideal que podemos obtener es del orden de $H = 1$ bit. La independencia y uniformidad vendrán dadas por los tests estadísticos.

2.2. Tests Estadísticos: Evaluando la independencia y uniformidad

¿Cómo podemos evaluar si un sistema físico cumple las propiedades de uniformidad e independencia entre valores que esperamos? La respuesta son los tests estadísticos.

Cuando aplicamos teoría de probabilidad a un sistema real, estamos trabajando con una hipótesis modelizada en el espacio de probabilidades¹⁰ como una distribución estadística. Dicha distribución debe ser puesta a prueba con los sucesos que podamos obtener del sistema, es decir, con datos reales. Los tests estadísticos son las herramientas matemáticas que usamos

¹⁰Nótese que en esta sección usaremos una interpretación de la probabilidad basada en teoría de la medida: Cada evento tiene asociado un espacio métrico, donde la medida corresponde a la probabilidad de que se obtenga un suceso del subconjunto estudiado.

para poder aceptar o rechazar la hipótesis de acuerdo a los datos reales. Este enfoque de enfrentarse a la hipótesis nula es una técnica de inferencia estadística denominada *contraste de hipótesis* [28].

Nuestro sistema es el conjunto de todas las secuencias de bits que podemos obtener con n medidas independientes en la Esfera de Bloch (véase la sección 2.3.1 para más detalles del sistema físico). Teóricamente corresponde a n *Ensayos de Bernoulli*, con distribución de probabilidades asociada la Distribución de Bernoulli Equiprobable¹¹. Nuestra hipótesis nula que resume estas propiedades, H_0 , es:

H_0 : El sistema es aleatorio. Es decir, los sucesos siguen una probabilidad basada en la Distribución de Bernoulli Equiprobable.

Para poder demostrar (o desmentir) H_0 usaremos inferencia estadística. Bajo este formalismo se plantean las siguientes definiciones recogidas por Shen [2].

Definición 2.9 (Test estadístico).

Sea $T = \{t_i\} \subset X$ un conjunto de sucesos del evento X . Sea P una distribución de probabilidades teórica que modeliza las probabilidades de X de acuerdo a la hipótesis H_0 . Diremos que T *conforma un test estadístico para H_0* cuando las probabilidades $\{\epsilon_i\}$ asociadas a T sean significativas de acuerdo a P para desafiar la hipótesis nula. En el caso de que, en un experimento, se obtenga un $x_i \in T$, diremos que *el test ha fallado*, o que se ha desmentido la hipótesis nula.

En nuestro caso los sucesos $\{t_i\}$ son significativos en el sentido de que, de ocurrir, tendríamos a pensar que el generador no produce secuencias aleatorias. Volviendo a la tirada de monedas, la secuencia de obtener n caras seguidas ($S(n)$) podría formar parte de un test estadístico que detecte secuencias anómalas, de tal forma que si la obtuviéramos en una moneda real podríamos plantearnos seriamente que la moneda esté trucada. Este último ejemplo plantea una problemática: si P corresponde a la Distribución de Bernoulli Equiprobable, todas las secuencias son equiprobables, de tal forma que obtener n caras no debería sorprendernos. Es un problema de objetividad, pues para cualquier conjunto de eventos A_i que obtengamos podríamos definir un test estadístico que contenga a A_i y rechace la hipótesis. Por ende, debemos ser capaces de definir previamente qué será un *buen test*, construyéndolo antes de conocer los datos a analizar.

Este enfoque no siempre es posible, ya que muchas veces se requiere un estudio previo parcial para poder escoger qué batería de tests puede ser más útil. Además, al no depender de los datos muchos tests pueden ser contradictorios entre sí, o generar ruido entre ellos. Un claro ejemplo viene dado por la Corrección de Bonferroni.

Supongamos que tenemos N tests definidos previamente y con probabilidad ϵ equiprobable. Si sólo existiera uno de estos tests, y los datos fallaran el test, entonces podríamos rechazar

¹¹Es decir, tenemos n "tiradas de moneda" que se modelizan con la distribución discretizada de $U(0, 1)$. Con los p-values recuperaremos una noción continua de la distribución de probabilidades asociada al modelo.

H_0 con la confianza de equivocarnos con probabilidad máxima ϵ . Sin embargo, al existir N tests definidos, si encontramos que se falla uno de los tests la probabilidad de que esto ocurriera es ahora $N\epsilon$. Es decir, la existencia de diversos tests no ha hecho más que disminuir nuestra confianza sobre nuestro veredicto. Esta incertidumbre en los tests se cuantifica utilizando como confianza el valor $N\epsilon$ en vez de ϵ , siendo N el número de tests que se usan en la disciplina con misma probabilidad. Se conoce como *Corrección de Bonferroni*. Sin embargo, esta disminución de confianza no se puede controlar a nivel práctico. Un investigador que aplique contraste de hipótesis no podría saber con absoluta certeza que no existen más tests que los que ha tenido en cuenta con la corrección [2].

Queda, pues, claro que no es sencillo determinar cuándo un test es bueno. Nosotros nos basaremos en un enfoque basado en la Teoría de la Información Algorítmica (AIT), rama de la teoría de computadores iniciada por Andréi Kolmogorov. En esta disciplina los buenos tests de aleatoriedad son aquellos que son característicos o que evalúan la incompresibilidad. Planteemos primero un fundamento matemático de la aleatoriedad como incompresibilidad de secuencias, tal y como hicimos con la entropía.

2.2.1. AIT: Complejidad de Kolmogorov y la incompresibilidad de secuencias

En AIT el concepto fundamental es el de Complejidad de Kolmogorov. Éste modeliza la naturaleza de una secuencia de bits a partir de la mínima secuencia que, con ayuda de una función U como algoritmo de descompresión, es capaz de recuperarla¹². Al conjunto de todas las secuencias de bit de longitud n se denotará $\{0, 1\}^n$ y, cuando la longitud sea arbitraria, $\{0, 1\}^*$ ¹³.

Definición 2.10 (Complejidad de una secuencia).

Sea $U : A \subseteq \{0, 1\}^* \rightarrow \{0, 1\}^*$ un algoritmo de descompresión. Definimos la *complejidad de una secuencia binaria x asociada a U* como:

$$C_U(x) = \min_{y \in \{0, 1\}^*} [|y| : U(y) = x] \quad (11)$$

Donde $|y|$ corresponde a la longitud de y , o el número de bits que la conforma.

Observación 2.11. Nótese que la complejidad descrita de esta forma depende del algoritmo de descompresión utilizado. No se conoce actualmente un "lenguaje universal" que nos permita catalogar la mínima longitud posible de bits de una secuencia [2]. Sin embargo, AIT asevera que existe un conjunto de lenguajes óptimos, cuyos algoritmos de descompresión $\{U'_n\}_{n \in \mathbb{N}}$ generan complejidades mínimas y que sólo difieren entre ellas por una constante. En estos casos, diremos que *difieren a orden 0*, o que su diferencia es de $O(1)$. Este resultado se conoce como *optimización asintótica* de los algoritmos, y fue descubierto por Kolmogorov

¹²Denominaremos algoritmo de descompresión a cualquier función que tome bits de longitud n y genere bits de longitud $n + k$.

¹³Por cuestiones de extensión, las demostraciones de los teoremas y proposiciones planteados en esta sección pueden encontrarse en el paper [76]. Un recorrido más profundo de todos los conceptos se encuentra en el libro de Tadaki [45]. Una referencia clásica de la teoría de Kolmogorov se encuentra descrita por Chaitin, uno de los fundadores de la teoría [27].

[76]. Los algoritmos que forman parte de este conjunto se denominan *algoritmos asintóticamente óptimos*.

Definición 2.12 (Complejidad de Kolmogorov). Sea U un algoritmo asintóticamente óptimo. Definimos la *Complejidad de Kolmogorov asociada a la secuencia x* como la complejidad asociada al algoritmo U , $C : \{0, 1\}^* \rightarrow \mathbb{R}^+$ tal que:

$$C(x) \equiv C_U(x) \quad (12)$$

Es decir, la Complejidad de Kolmogorov modeliza la síntesis máxima que puede tener una secuencia de bits. A partir de ahora usaremos indistintamente *complejidad* y *Complejidad de Kolmogorov*.

Observación 2.13. Nótese que para la Complejidad de Kolmogorov sí que requeriremos que recoja todo el dominio de $\{0, 1\}^*$ ¹⁴. Para ello utilizaremos distintos algoritmos asintóticamente óptimos, realizando una envoltura de $\{0, 1\}^*$ con sus dominios. Las secuencias que no tengan algoritmos de descompresión posibles tienen asociados el algoritmo identidad ($U'(x) = x$)¹⁵. En el caso opuesto de encontrarnos superposiciones de dominios (varios algoritmos distintos pueden dar complejidades distintas de un mismo x), se trabajan con clases de equivalencia para absorber la diferencia de $O(1)$ entre complejidades. De todos modos, este problema de la no unicidad lleva a que la Complejidad de Kolmogorov sea una función no computable¹⁶. Por simplicidad a este conjunto de algoritmos $\{U_n\}$ lo tomaremos como un algoritmo U único definido como la clase de equivalencia del conjunto.

Teorema 2.14 (Propiedades de C).

Sea $C : \{0, 1\}^* \rightarrow \mathbb{R}^+$ la complejidad de Kolmogorov. Presenta las siguientes propiedades:

1. La complejidad siempre es menor a orden 0 de la longitud del bit:

$$C(x) \leq |x| + O(1) \quad \forall x \in \{0, 1\}^* \quad (13)$$

2. Dada una longitud n , hay un número finito de secuencias con complejidades menores a esa longitud. En concreto:

$$\text{card}\{x \mid C(x) \leq n\} < 2^n \quad k \in \mathbb{R}^+ \quad (14)$$

3. Más general, hay un número finito de secuencias con complejidades menores a una longitud $c < n$:

$$\text{card}\{x \mid C(x) \leq n\} < 2^{n-c} \quad k \in \mathbb{R}^+ \quad (15)$$

¹⁴Dicho con nomenclatura más precisa: la complejidad de una cadena es una función parcial en $\{0, 1\}^*$, mientras que la Complejidad de Kolmogorov es una función total de $\{0, 1\}^*$.

¹⁵Más adelante veremos que este caso es más común de lo que aparenta, y es la base para definir la aleatoriedad desde el punto de vista de la incompresibilidad y tipicabilidad de una cadena.

¹⁶De forma superficial: una función computable es aquella que puede ejecutarse con una máquina de Turing, y con dominio $\{0, 1\}^*$. Una descripción más precisa se encuentra en la sección *Computability* del libro de Tadaki [45].

A continuación enunciamos un teorema fundamental para el análisis de las siguientes secciones.

Teorema 2.15 (Conservación a orden 0 de la complejidad).

Sea x una secuencia de bits con complejidad $C(x)$. Sea f una función computable. Entonces $f(x)$ presenta la misma complejidad a orden 0 que x . Es decir:

$$\exists k \in \mathbb{R} \mid C(f(x)) \leq C(x) + k \quad \forall x \in \{0, 1\}^* \quad (16)$$

Observación 2.16. Este teorema es muy potente, ya que podemos descartar el incrementar la complejidad con el uso de implementación computada de ruido. No se puede insuflar aleatoriedad a una secuencia mediante la superposición de programas aparentemente aleatorios. Un caso extremo es el divertido experimento realizado por Marsaglia en una de las secuencias de números que acompañaba a la suite de tests de aleatoriedad *Diehard* [29], concretamente *cdmake.ps*¹⁷. Partiendo de un sustrato "idealmente" aleatorio (de acuerdo a los fabricantes del RNG de hardware que usó), decidió tratar de mejorar la secuencia combinándola con la de otros RNG. Combinó distintos PRNG muy populares en el momento con ruido blanco y ruido negro obtenido de grabaciones de música de rap. Otra fuente de ruido fue el scrappeo de píxeles de la red, siendo algunos incluso de fotogramas de, como él enuncia, "señoritas sin ropa". Combinando las distintas secuencias con una operación *xor*, se obtuvo la secuencia *cdmake.ps*. Pese a todos los esfuerzos, el resultado no generó aleatoriedad bajo este formalismo. La razón es sencilla: muchos de estos procesos (salvo quizás la música de rap y las señoritas sin ropa) se pueden simular con funciones computables, y por ende la complejidad de Kolmogorov no varió de sobremanera.

Nótese que no existe un teorema que genere una cota inferior de esta naturaleza: el uso de programas puede disminuir la complejidad previa. Un ejemplo a este respecto se encuentra también en las secuencias de prueba de Diehard. Los archivos *canada.bit* y *germany.bit* los obtuvo Marsaglia a partir de dos RNG de hardware. Según los fabricantes, los bits generados por los dispositivos cumplirían los estándares de aleatoriedad. Sin embargo, al probarlos se encuentran claras frecuencias de subcadenas de 13 10 y una ausencia clara de subcadenas de 10 10, fallando gran parte de los tests de Diehard. La razón es que, al reescribir el fichero de un sistema Unix a un MSDOS, el salto de línea se codifica con bytes distintos, sobrepoblando un valor de byte concreto [2]. La función computable (en este caso, la traducción a sistema MDOS) disminuyó la complejidad de la secuencia.

Ahora que tenemos los fundamentos de AIT, centrémonos en el concepto de secuencia irreducible y cómo se relaciona con la aleatoriedad.

Definición 2.17 (Complejidad de un par de cadenas).

Sea

$$\begin{aligned} U : \{0, 1\}^* \otimes \{0, 1\}^* &\longrightarrow \{0, 1\}^* \\ (x, y) &\longrightarrow D(x, y) \in \{0, 1\}^* \end{aligned} \quad (17)$$

¹⁷Una recreación del CD se puede encontrar en el siguiente repositorio de github <https://github.com/jeffThompson/DiehardCDROM> [39]

una función computable e inyectiva. Definimos la complejidad asociada a la tupla de secuencias (x, y) como la asociada a la preimagen de U :

$$C_U(x, y) = \min_{a, b \in \{0,1\}^*} [|a| + |b| \equiv |[a, b]| : U(a, b) = [x, y]] \quad (18)$$

Nótese que se cumple que:

$$C(x, y) \geq C(x) \quad C(x, y) \geq C(y) \quad (19)$$

Definición 2.18 (Complejidad condicional).

Sea

$$\begin{aligned} D : \{0, 1\}^* \otimes y \in \{0, 1\}^* &\longrightarrow \{0, 1\}^* \\ (p, y) &\longrightarrow D(p, y) \end{aligned} \quad (20)$$

Una función computable de dos argumentos. Definimos la *complejidad de x cuando se conoce y* como:

$$C_D(x|y) = \min_{p \in \{0,1\}^*} [|p| : D(p, y) = x] \quad (21)$$

Es decir, corresponde a la longitud de la secuencia más pequeña que, con ayuda de y , permite reconstruir x . p se denomina el *descriptor* de x , y D el *algoritmo de descompresión condicional*. Si D es exponencialmente óptimo para todo x e y , entonces hablaremos de la *Complejidad de Kolmogorov Condicional*, $C(x|y)$.

Definición 2.19 (Secuencia incompresible).

Diremos que $i \in \{0, 1\}^n$ es incompresible si:

$$C(i|n) \geq n \quad (22)$$

Es decir, si, aunque conozcamos su tamaño, no exista algoritmo exponencialmente óptimo que sea capaz de recuperar la cadena original a partir de una de menor tamaño ¹⁸.

Se dirá además que una cadena $i_k \in \{0, 1\}^n$ es k – incompresible cuando:

$$C(i_k|n) \geq n - k \quad (23)$$

Que corresponde a atribuir un intervalo de complejidades en el que consideraremos incompresibilidad.

Bajo AIT, una secuencia será aleatoria cuando sea incompresible. Esta idea es muy potente, ya que nos permite plantear una definición de aleatoriedad independiente del contexto de la secuencia. Hay un problema práctico, y es que no podemos determinar con exactitud la complejidad asociada a una cadena porque la función Complejidad de Kolmogorov no es computable. Existen teoremas que enuncian que la complejidad es computable superiormente [76], [22] [66]. Sin embargo, no existe aparataje matemático suficiente para reconocer cuándo una sucesión de funciones computables presenta una aproximación de la función de

¹⁸Nótese que utilizamos una desigualdad en vez de una igualdad por las fluctuaciones de orden 0 que introduce la familia de algoritmos exponencialmente óptimos.

complejidad lo suficientemente buena. Sin embargo no es trabajo en balde, pues se pueden utilizar los algoritmos de compresión más populares para estimar una cota superior de complejidad. Con esta idea se puede construir —y construiremos— un test estadístico.

Utilicemos lo avanzado con la incompresibilidad para abarcar el concepto de aleatoriedad como tipicidad de una secuencia. Es aquí donde entra el segundo teorema más importante del capítulo:

Teorema 2.20 (Población de secuencias irreducibles).

Sea $n \in \mathbb{N}$. Se cumple que:

1. Existen cadenas $i \in \{0, 1\}^n$ incompresibles
2. La fracción de cadenas i_c c -incompresibles con respecto a las compresibles de longitud n es mayor a $1 - 2^{-c}$. Dicho de otra forma:

$$\frac{\text{card}\{i_c\}}{\text{card}\{0, 1\}^n} > 1 - 2^{-c} \quad (24)$$

Demostración. Podemos escribir el cardinal de las cadenas c -incompresibles como la diferencia de los cardinales del conjunto total (el de todas las cadenas de longitud n) y el de cadenas con complejidades menores a $n - c$:

$$\frac{\text{card}\{i_c\}}{\text{card}\{0, 1\}^n} = \frac{\text{card}\{0, 1\}^n}{\text{card}\{0, 1\}^n} - \frac{\text{card}\{x \in X \mid C(x) < n - c\}}{\text{card}\{0, 1\}^n} \quad (25)$$

Sin embargo, por las propiedades de la complejidad (teorema 2.14) el cardinal del conjunto de cadenas con complejidades menor a una longitud $n - c$ está acotado superiormente por 2^{n-c} . Invirtiendo el signo de la desigualdad por la resta, y dado que el cardinal del conjunto de todas las secuencias de longitud n es 2^n , tenemos:

$$1 - \frac{\text{card}\{x \in X \mid C(x) < n - c\}}{\text{card}\{0, 1\}^n} > 1 - \frac{2^{n-c}}{2^n} = 1 - 2^{-c} \quad (26)$$

Por ende:

$$\frac{\text{card}\{i_c\}}{\text{card}\{0, 1\}^n} > 1 - 2^{-c} \quad (27)$$

□

Observación 2.21. Vemos que el porcentaje de cadenas que consideramos incompresibles crece con una exponencial inversa de parámetro c . Por ejemplo, a partir de $c = 10$ en torno al 99,9% de las secuencias se considerarían incompresibles. De tal forma, para secuencias de millones de bits, una tolerancia de 10 bits para considerar que la cadena es incompresible es más que aceptable, y es por ello que podemos decir que las secuencias más típicas en $\{0, 1\}^n$ son las incompresibles, en especial si $n \rightarrow \infty$.

Este resultado supone un cambio de paradigma en este TFG. Hasta ahora se entendían las secuencias aleatorias como las más extravagantes de los conjuntos, pero matemáticamente ¡es todo lo contrario! La propiedad de uniformidad se puede interpretar como la

cantidad abrumadora de secuencias irreducibles respecto a las no irreducibles, hablando de una secuencia "aleatoria" cuando es "común"¹⁹.

A continuación definimos la medida más usada a la hora de plantear tests estadísticos basados en la complejidad de una secuencia: la función de deficiencia.

Definición 2.22 (Función de deficiencia).

Sea $d : \{0, 1\}^* \rightarrow \mathbb{R}^+$ una función semicomputable por límite inferior. Diremos que d es una *función de deficiencia* si, para cada $k \in \mathbb{R}^+$ la fracción de $x \in \{0, 1\}^n$ que cumplen $d(x) > k$ es del orden de $O(2^{-k})$. Además, existe una familia de funciones de deficiencia maximales a orden $O(1)$ de tal forma que:

$$d(x) = n - C(x|n) + O(1) \quad \forall x \in \{0, 1\}^n \quad (28)$$

Nótese que la condición de fracción de secuencias con valor mayor a k está estrechamente relacionada con la Complejidad de Kolmogorov y la tipicabilidad de acuerdo al teorema 2.20. Así, la función de deficiencia expresa cuánto se separa la secuencia de ser idealmente aleatoria. El threshold que definimos como intervalo en el que considerábamos que una secuencia era incompresible se ve aquí como una cota de la función de deficiencia. Considerar que la secuencia es c-incompresible es equivalente a pedir que d esté acotada superiormente por c . La función de deficiencia se puede interpretar como un déficit de aleatoriedad: a mayor valor de d , menos aleatoria es la secuencia de acuerdo al marco de AIT.

Hemos refinado nuestra definición de aleatoriedad, pudiendo cuantificar la propiedad de impredecibilidad a partir de la entropía del sistema. Ahora somos también capaces de cuantificar la uniformidad a partir de la incompresibilidad//tipicabilidad de la secuencia²⁰.

2.2.2. Familia de Tests Estadísticos

Es el momento de profundizar en los tipos de tests que vamos a usar. En la bibliografía encontramos un uso predominante de tests basados en *p-values* [2][12][73] [44]. Así, nosotros también nos enfocaremos en los tests de esta naturaleza.

Sea X el espacio de todos los sucesos posibles del experimento. En nuestro caso, si tomamos n medidas de bit, corresponde a todas las secuencias de bits que podemos generar

¹⁹Ocurre algo parecido con los estados en Mecánica Estadística, pues en el equilibrio encontramos muchos más estados indistinguibles que distinguibles. Véase el caso de un gas ideal. Si discretizamos el volumen en el que se encuentra el gas, y a cada celda le asociamos un cero si no hay una partícula y un uno si hay una, encontraríamos muchas más secuencias "típicas", o incompresibles, que anómalas (por ejemplo, encontrarnos la mayoría de las partículas en una esquina de la sala).

²⁰Como última matización, los conceptos de Entropía de Shannon y de Complejidad de Kolmogorov se pueden relacionar entre sí, pese a partir de principios distintos. Shannon buscaba estudiar la información obtenida por un sistema generador de secuencias, mientras que Kolmogorov trata de estudiar exclusivamente la información obtenida por las secuencias generadas. Sin embargo, si la distribución estadística asociada al sistema es computable, la Entropía de Shannon corresponde a la Complejidad de Kolmogorov esperada. Es un resultado muy interesante y prometedor para QRNG de computadores cuánticos. Sin embargo, por la extensión del TFG nos vemos forzados a excluirlo del análisis y redirigir al lector más curioso a Grünwald y Vitányi [66].

con n ensayos de Bernoulli. Los p-value tests son pruebas donde a cada secuencia posible $x \in X$ le asociamos un valor real a partir de una función ξ :

$$\begin{aligned} \xi : \{0, 1\}^n &\longrightarrow \mathbb{R} \\ x &\longrightarrow \xi(x) \end{aligned} \quad (29)$$

De tal forma que a cada secuencia posible le asignamos probabilidades según sus imágenes de ξ , y éstas se usen como criterio en los tests estadísticos. Dicha función a menudo tendrá un sentido físico. Por ejemplo, la frecuencia del bit 1 en la cadena puede cumplir perfectamente el rol de esta función. Otro ejemplo de función ξ es precisamente la función de deficiencia. De esta forma, los p-value tests están modelados por la elección de ξ como propiedad de la secuencia a analizar, y están descritos para un continuo en $\mathbb{R}$.

Definición 2.23 (p-value).

Sea $c \in X$ una secuencia obtenida del experimento. Sea ξ una función que evalúa una propiedad de las secuencias de X . Diremos que el *p-value de la secuencia c* es la probabilidad²¹ asociada al conjunto de secuencias con imágenes en ξ mayores a la de c :

$$P(c) = Pr(\{x \in X | \xi(x) \geq \xi(c)\}) \quad (30)$$

Esta probabilidad representa la "particularidad" de la secuencia c . I.e.: ¿Cuál era la probabilidad de obtener este resultado bajo la hipótesis establecida?

Definición 2.24 (p-value test).

Sea X el conjunto de secuencias posibles del experimento. Sea ξ una función evaluadora de una propiedad de X . Sea $p : X \longrightarrow [0, 1]$ la función de p-values posibles de X por ξ . Definimos *la familia de tests asociadas a p* como el conjunto de todas las secuencias con un p-value menor a una cota:

$$T_\epsilon = \{x \mid P(x) \leq \epsilon\} \subset X \quad (31)$$

Al definirse por una cota de probabilidades, también se denominan *probability bounded tests*. *Observación 2.25* (Grado de confianza del test // tolerancia).

La definición formal de un p-value test indica que el test está conformado por todas las secuencias que, bajo el prisma de una propiedad concreta del experimento, presentan una "particularidad" lo suficientemente característica como para dudar de nuestra hipótesis. La libertad del test se encuentra en la elección de la cota, o *tolerancia del test* (ϵ).

Un test estadístico nos puede dar dos resultados posibles: o bien ninguna secuencia obtenida pertenece al conjunto de alarma del test, y se aprueba H_0 ; o bien el test falla y se desmiente H_0 . Sin embargo, estos casos se pueden desglosar en otros dos: puede ser que la hipótesis de aleatoriedad sea cierta, pero que el test haya fracasado erróneamente, o la inversa. El caso de tener un falso positivo se asocia a un *error tipo I*, y el de un falso negativo, un *error tipo II*. La tolerancia de los tests de p-values corresponde al error tipo I. La elección de esta cota es relativamente libre, y depende mucho de la disciplina en la que

²¹Entendemos aquí la probabilidad como una medida, de acuerdo a Teoría de la Medida.

se vaya a aplicar el test. Por ejemplo, en criptografía los valores aceptables de threshold se encontrarían con un máximo de $\epsilon = 0,01$ [12].

Observación 2.26 (Algunos tests). A continuación enumeramos algunos tests basados en p-values posibles [2]

1. Podemos construir tests a partir de resultados de teoría de probabilidad. Éstos consisten en evaluar la similitud de la distribución de probabilidades teórica con la obtenida en el experimento mediante un parámetro, e imponerle una cota de mínima verosimilitud. Las herramientas que se pueden usar como funciones ξ para este caso son funciones de distancia entre distribuciones. Más adelante trabajaremos con la prueba χ^2 de Pearson, y el test de Kolmogorov-Smirnov con sus respectivas funciones distancia.
2. Al igual que podemos construir tests para teoría de la probabilidad, podemos hacer lo mismo para modelos estadísticos. De esta forma, podemos calcular frecuencias de bits bajo distintas condiciones, obtener los vectores del Espacio de Fourier, comprobar si hay dependencias lineales entre secciones de la secuencia, y demás técnicas que estudien de forma secundaria las propiedades de impredecibilidad. Este es el enfoque de gran parte de las suites disponibles. Hablaremos más en detalle de algunos de estos tests a partir de la suite de NIST.
3. Podemos tratar de estimar la entropía del sistema y comprobar la diferencia con la esperada. Este es un método bastante directo de los conceptos presentados, pero hay un problema: en muchos casos no somos capaces de estimar la entropía teórica que debería tener el sistema, mucho menos estimar con confianza la del dispositivo experimental [46]. De esta forma, lo que se suele hacer es usar este acercamiento como una primera batida de la aleatoriedad inherente al sistema. Se utiliza la versión más suave de la entropía: la *Min-Entropy*. Esta es la entropía más pequeña que puede tener un suceso del experimento. Existen metodologías para calcularla a tiempo real para QRNG [19].
4. Relacionado con la entropía, podemos tratar de buscar otra variable física que dependa de procesos estocásticos²² —algo común en Mecánica Estadística, como el Modelo de Ising— y tratar de simularla con un Metrópolis Monte-Carlo para imponer un threshold a su diferencia con la teórica. El problema de este enfoque es que requiere código adicional, trabajar con los errores numéricos inherentes a la simulación, y tener un trasfondo teórico profundo del sistema. Pese a ello, existen artículos que han abordado este enfoque [50].
5. Podemos tratar de acotar superiormente la complejidad de Kolmogorov a partir de los algoritmos de compresión más populares y utilizados. De esta forma se puede estimar una función de deficiencia y, a partir de ella, obtener p-values.
6. Finalmente, existe un enfoque secundario relacionado con demostrar existencia de objetos representativos de aleatoriedad, o de la ausencia de ella. Los ejemplos de rechazo de los algoritmos *xorshift* y *LCG* son ejemplos de esta técnica. Al estudiar las formas

²²Recordemos: Un proceso estocástico es aquel donde una o varias variables siguen una distribución de probabilidades que varía en el tiempo.

geométricas que se pueden presentar en sistemas aleatorios podemos comprobar qué formas *no* pueden surgir (como el caso de los planos en dimensiones superiores). Este es el estudio de las *superficies aleatorias*, con objetos matemáticos muy interesantes y con múltiples aplicaciones en física como la *Esfera Browniana* o *La esfera pura de Liouville para gravedad cuántica*. [74]

Observación 2.27. Test estadístico a partir de la función de deficiencia

Basándonos en el artículo de Shen [2], podemos usar la función de deficiencia para construir un test estadístico. La función de deficiencia, por definición, cumple que la proporción de secuencias con deficiencia mayor a una constante k es del orden de $O(2^{-k})$. Bajo hipótesis de aleatoriedad, por nuestra definición de la misma la probabilidad de obtener cada secuencia sigue la distribución uniforme. Además, el conjunto $\{0, 1\}^n$ es finito (en concreto, tiene cardinal 2^n). De esta forma, la proporción de secuencias corresponde a la probabilidad del conjunto:

$$\frac{\text{card}\{x \in \{0, 1\}^n \mid d(x) \geq k\}}{\text{card}\{0, 1\}^n} \leq 2^{-k} \Leftrightarrow P(d(x) \geq k) \leq 2^{-k} \quad (32)$$

Al tener una cota superior de la probabilidad, podemos crear un test estadístico T basado en $\xi \equiv d$, donde los p-values se definen con la cota de deficiencia. Si encontramos secuencias con deficiencia mayor a un k por determinar (dependerá de la tolerancia del test, que elijeremos en la sección 3.1), diremos que la secuencia no cumple nuestros estándares de independiencia y uniformidad para aceptar aleatoriedad.

2.2.3. Tests de Segundo Orden

Los Tests de Segundo Orden son tests estadísticos que se aplican a los p-values obtenidos por tests de la secuencia. De esta forma, se aplican al conjunto de secuencias usadas en los tests de primer orden y evalúan la fiabilidad de los resultados. Nuestros tests de segundo orden se enfocarán en comprobar si los p-values de cada prueba generan la distribución $U(0, 1)$. La razón de que esperemos que los p-values esta distribución es que, si las secuencias son aleatorias, por tipicabilidad en promedio deben presentar los mismos resultados en los tests, obteniendo así p-values uniformes. Un fracaso en estos tests podría deberse a dos circunstancias: O bien la secuencia no es aleatoria, o bien los tests generan error en forma de ruido. Para minimizar el segundo problema y poder centrarnos en la hipótesis nula, Shen propone un nuevo acercamiento a cualquier test que se base en comparar una distrubución experimental con una teórica que se deba simular [77] [2].

Según Shen, los tests estadísticos presentan tres problemas fundamentales:

1. Los outputs individuales no generan una medida significativa en el espacio de probabilidades (no son lo suficientemente significativos). Se resuelve con una aplicación astronómicamente grande de tests, y por ende no se puede hacer mucho para resolverlo.
2. Errores de programación de los tests. Es importante revisar el código de las suites cuidadosamente, y comprobar que todo esté en orden antes de tratar de concluir nada a partir de ellas. Nosotros realizamos el trabajo previo estudiando los tests NIST a partir de su manual [12], y comprobamos que corresponda al código planteado por

Kho [43].

3. Errores de modelización numérica de las distribuciones teóricas. Toda distribución continua, en especial las estadísticas, presentarán errores de aproximación cuando se trabaje con ellas en un computador. La razón es el truncado de los valores en la distribución, así como la determinación de las constantes estadísticas en las mismas. Marsaglia admitió en numerosas ocasiones que en algunos de sus tests los parámetros estadísticos se debían obtener a partir de simulaciones [2], y ha habido casos de error en tests por aplicar erróneamente estas distribuciones. Por ejemplo, la primera versión de la suite de NIST presentaba un test que tuvo que ser retirado [2].

Se puede evitar el último problema con una perspectiva muy interesante: en vez de evaluar la aleatoriedad por comparación con las distribuciones teóricas, podemos hacerlo comparándola con las distribuciones ya numéricas obtenidas por RNG de confianza. Veamos cómo hacerlo a partir del Test de Kolmogorov-Smirnov.

El Test de Kolmogorov-Smirnov evalúa la similitud entre dos distribuciones. Existen dos variantes. La variante *One-Sample* compara una distribución problema con una teórica. La variante *Two-Sample*, por otro lado, evalúa la similitud entre dos distribuciones problema. Denominamos *distribución problema* a aquellas que están obtenidas por datos experimentales y, por ende, están eminentemente discretizadas. Dentro de cada variante, podemos realizar el análisis *two-sided* o *one-sided* si consideramos la diferencia entre distribuciones con o sin valor absoluto, respectivamente. Nosotros usaremos la estadística *Two-Sample y two-sided*, descrita de acuerdo al paper de Han, Li y Liu [24].

Definición 2.28 (Estadística de Kolmogorov-Smirnov (Two-Sample, Two-Sided)).

Sea $\{x_1, x_2, \dots, x_n\}$ un conjunto de secuencias con distribución teórica asociada F . Sea $\{y_1, y_2, \dots, y_k\}$ otro conjunto de secuencias con distribución teórica asociada G . Sean F_n y G_k las distribuciones teóricas empíricas:

$$F_n(t) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(x_i \leq t) \quad G_k(t) = \frac{1}{k} \sum_{i=1}^k \mathbb{I}(y_i \leq t) \quad (33)$$

Donde $\mathbb{I}(a \leq t)$ corresponde al conteo de elementos a de cada secuencia que cumplen la condición.

Definimos la *Estadística de Kolmogorov-Smirnov (TS, TS)* como:

$$D_{n,k} = \sup_{t \in \Omega} |F_n(t) - G_k(t)| \quad (34)$$

Donde Ω es la intersección de los dominios de cada distribución (sustentados en $\mathbb{R}^+$).

Al final la Distribución de Kolmogorov Smirnov (a partir de ahora KS) nos indica la máxima diferencia que hay entre las dos distribuciones en todo su dominio para n y k datos de cada una, respectivamente.

A la hora de aplicar KS para probar aleatoriedad, lo recomendable es verificar H_0 de acuerdo a la comparación con la distribución $U(0, 1)$. Es decir, a partir de los p-values asociados a cada test de primer orden, definimos una familia de distribuciones de probabilidades

empírica $G_{\xi(X^m), j \in \{1, 2, \dots, m\}} : \mathbb{R}^+ \rightarrow [0, 1]$ que cuente la frecuencia de las secuencias con p-values menores a un parámetro t tomando j secuencias experimentales [1]. Por ejemplo, si tomamos las m secuencias que tuviéramos podríamos obtener la distribución:

$$G_{p,m}(t) = \frac{1}{m} \text{card}\{1 \leq i \leq m \mid p_i \leq t\} \quad (35)$$

Con el acercamiento clásico, compararíamos $G_{p,m}$ con $U(0, 1)$ mediante KS de One-Sample. Si la distancia de Kolmogorov cumple que:

$$D_{p,n,k} < K(\epsilon) \quad (36)$$

Donde $K(\epsilon)$ es una función cota que, para una cantidad de datos n, m grandes y un threshold ϵ pequeño se puede aproximar como [1]:

$$K(\alpha) \simeq \sqrt{-\frac{1}{2} \log \left(\frac{\epsilon}{2} \right)} \quad (37)$$

Nosotros consideraremos esta cota, pero usaremos la vía de Shen en la creación de las distribuciones. En vez de usar la distribución teórica, tomaremos otra familia empírica $E_{\xi(\Omega^m), j \in \{1, 2, \dots, m\}}$ construida con un RNG de confianza y aplicaremos KS Two-Sample entre ellas bajo la nueva hipótesis H'_0 :

Los datos entre las muestras son independientes entre ellas y aleatorias, pero con misma distribución teórica asociada.

De esta forma no dependemos de ninguna forma de la distribución teórica —podría incluso no ser la distribución uniforme— evitando computar. Si fracasa el test podrá ser por dos razones:

1. Los datos entre RNGs no son independientes entre sí. Podemos evitarlo tomando los datos en circunstancias distintas o con mecanismos distintos.
2. Una de las secuencias no es aleatoria. Como confiamos en el generador de referencia, la secuencia a juicio no es aleatoria.

El problema se ha trasladado a encontrar un RNG lo suficientemente confiable como para considerarlo el ideal de aleatoriedad. Podemos usar una propiedad de KS que nos permite relajar la condición de que el RNG de referencia sea aleatorio a que sea independiente del de prueba [2] procediendo del siguiente modo:

1. Obtenemos dos conjuntos de secuencias $\{x_1, x_2, \dots, x_m\} \subset X^m$ e $\{z_1, z_2, \dots, z_m\} \subset X^m$ del RNG de prueba. Tomamos los tests y obtenemos la distribución $G_{p,n}$ del RNG de prueba.
2. Obtenemos un conjunto de secuencias $\{y_1, y_2, \dots, y_m\} \subset Y^m$ del RNG de referencia. Creamos secuencias entremezcladas $\{z_i\}$ e $\{y_i\}$ con una operación *xor*²³ entre los bits de las secuencias. Así, obtenemos secuencias $\{y'_1, y'_2, \dots, y'_m\}$.
3. Aplicamos los tests a las secuencias mixtas, obteniendo la distribución $E'_{p,m}$.

²³Una operación *xor* corresponde a, dados dos bits, devolver 1 si y solo si son diferentes entre sí [44].

4. Aplicamos KS los resultados de $\{x_i\}$ y $\{y'_i\}$ bajo la nueva hipótesis H_0'' :

Las secuencias de prueba son aleatorias, y los generadores son independientes entre sí.

Así, se ha relajado la condición a que ambos generadores sean independientes y, si no se cumple H_0'' , será porque el RNG de prueba no genera secuencias aleatorias. La limitación de este método es que necesitamos el doble de datos del generador de prueba.

2.2.4. NIST Test Suite

NIST800-22-1a corresponde a la publicación de una batería de tests de p-values diseñada a principios de los 2000, y actualizada en 2010 [12]. Está diseñada por el Instituto Nacional de Estándares y Tecnología de Estados Unidos, y sus tests se usan como estándares de calidad en RNG para criptografía y modelización²⁴. Una desventaja de aplicación, sin embargo, es que sus tests están diseñados para secuencias binarias. Si bien esta metodología está amparada por multitud de herramientas, como hemos visto en el fundamento teórico, esto nos fuerza a descartar otras técnicas basadas en el continuo de $\mathbb{R}$. Algunos ejemplos son el estudio de aleatoriedad a partir de tests basados en simulaciones físicas por métodos de Metrópolis Monte Carlo [30], o la determinación de constantes matemáticas como el número π con también el método Metrópolis.

La hipótesis de los tests es la misma: Tomamos como hipótesis nula H_0 que la secuencia es aleatoria. Son tests de p-values, de tal forma que con cada prueba obtendremos un p-value y, dada una cota, un veredicto de aceptación o rechazo de la hipótesis. Al obtener *True*, aceptaremos la hipótesis de aleatoriedad para ese test. Hay que destacar un matiz importante: el rechazo de aleatoriedad en uno de los tests no supone un rechazo de aleatoriedad en la secuencia. Como citan los propios autores, es esperable que secuencias aleatorias fallen algunos tests [12]. Tampoco esperan que la suite sea capaz de verificar con total completitud la aleatoriedad de la secuencia. El foco de estos tests es comprobar la existencia de estructuras regulares debido a ruido de ondas u otros fenómenos periódicos [77]. Es por ello que tendremos cierta libertad a la hora de descartar familias de tests, añadir otros o interpretar los resultados obtenidos. En el Anexo enumeramos los tipos de tests de la suite, con una breve indicación de qué trata de evaluar cada uno y sus tests específicos.

Realizaremos un análisis de las secuencias obtenidas aplicando los tests a partir de la adaptación programada por Steven Kho para Python [43]. En pos de aseverar la fidelidad de los resultados pese al problema de la computación de las distribuciones²⁵, aplicaremos Tests

²⁴La definición de aleatoriedad de esta suite también tiene como base los Ensayos Equiprobables de Bernoulli, con independencia, impredecibilidad y uniformidad como propiedades fundamentales. No tendremos que realizar concesiones semánticas para usarla.

²⁵Se ha de enfrentar el problema descrito por Shen: muchos de estos tests calculan p-values a partir de comparaciones de los datos con distribuciones teóricas aproximadas por computadores. Supone un desafío poder escoger adecuadamente los parámetros de las distribuciones estadísticas, y generan problemas de confianza en los p-values obtenidos [2]. En especial NIST tiene preferencia por utilizar distribuciones χ^2 en multitud de sus tests. También usan la Función Gamma Incompleta, y la función complementaria de error

de Segundo Orden propios siguiendo la recomendación de Shen [77], usando como inputs los p-values de los test de la librería NIST.

2.3. Computación Cuántica

2.3.1. Computadores Cuánticos

Los computadores cuánticos son dispositivos capaces de controlar sistemas cuánticos²⁶, pudiendo realizar transformaciones ordenadas a unos estados iniciales (inputs), de tal forma que una distribución estadística (output) nos permita realizar operaciones. Este control tan fino no pudo obtenerse hasta los años 70, pero en la actualidad existen distintas técnicas con sus ventajas e inconvenientes [56]. Independientemente de la arquitectura usada, el formalismo teórico es el mismo: tomamos sistemas aislados donde trabajemos con dos estados posibles para nuestra medida. Éstos pueden ser un valor de espín en un eje dado (computadores de quantum dots), estado base de un electrón en un átomo o estado excitado (computadores de átomos aislados, o *trapped ions*), exceso de Pares de Cooper en un superconductor (computadores de superconductividad), y demás. Denominamos al autoestado del sistema sin perturbar $|0\rangle$ y, al excitado, $|1\rangle$. Al subconjunto de estados posibles del sistema contruidos por la base $V = \{|0\rangle, |1\rangle\}$ y una fase global se denomina *Esfera de Bloch*²⁷, y el sistema físico se denomina *qbit*. La figura 3 recoge una representación de la esfera de Bloch.

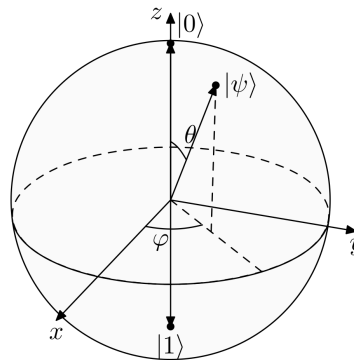


Figura 3: Esfera de Bloch [60]. Son coordenadas esféricas con $R = 1$ (Véase ecuación 39).

Un qbit se inializa con estado $|0\rangle$ y, mediante la aplicación de puertas cuánticas, se cambia su estado con probabilidades controladas por sus coordenadas:

$$|\Psi\rangle = c_1 |0\rangle + c_2 |1\rangle \mid P(E = E_0) = |c_1|^2 \quad P(E = E_1) = |c_2|^2 \quad (38)$$

erfc.

²⁶Un resumen de lo fundamental del Formalismo de Dirac para sistemas cuánticos se encuentra en el Anexo.

²⁷Es interesante mencionar que el álgebra usada en los sistemas cuánticos forma parte del grupo algebraico $SU(2)$, de tal forma que introducir una fase global al sistema lo deja indiferente, entre otras propiedades.

En coordenadas esféricas el estado suele representarse como:

$$|\Psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)e^{i\phi}|1\rangle \quad \theta \in [0, \pi]; \phi \in [0, 2\pi] \quad (39)$$

Donde se ha usado la condición de normalización y de invarianza $SU(2)$ para llegar a la parametrización de las coordenadas con solo θ y ϕ ²⁸.

Actualmente somos capaces de utilizar computadores cuánticos con acceso a más de cien qbits, lo que significa que se trabajan con sistemas multipartículas de bases el producto tensorial entre todas las Esferas de Bloch de los qbits (da lugar a una *Hiperesfera de Bloch*, que se trabaja igual que una Esfera de Bloch). Sin embargo, aún sigue siendo un problema técnico aumentar el control extendido en el tiempo de los estados, el número de qbits, disminuir el error de ruido al aplicar puertas cuánticas, y demás desafíos.

2.3.2. Puertas cuánticas

Un computador cuántico trabaja con los estados del sistema mediante puertas cuánticas²⁹. Estos son operadores lineales y unitarios en el Espacio de Hilbert que abstraen las acciones físicas que realizamos al estado cuántico³⁰. A partir de un conjunto pequeño de puertas cuánticas somos capaces de diseñar un conjunto infinito de puertas. Estas puertas básicas se denominan *puertas universales* y para acciones en las que interviene un solo qbit vienen dadas por las Matrices de Pauli [56]³¹. Son de la forma:

$$X \equiv \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Y \equiv \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \quad Z \equiv \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (40)$$

A partir de las matrices de Pauli podemos diseñar los operadores de rotación del vector de Bloch. Éstos nos permiten movernos a lo largo de la esfera de Bloch a partir de rotaciones respecto a los diferentes ejes de la esfera (como si de ángulos de Euler se tratasen)³²:

$$\begin{aligned} R_x(\theta) \equiv e^{-i\theta\frac{X}{2}} &= \begin{pmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix} & R_y(\theta) \equiv e^{-i\theta\frac{Y}{2}} &= \begin{pmatrix} \cos\frac{\theta}{2} & \sin\frac{\theta}{2} \\ -\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix} \\ R_z(\theta) \equiv e^{-i\theta\frac{Z}{2}} &= \begin{pmatrix} e^{-i\frac{\theta}{2}} & 0 \\ 0 & e^{i\frac{\theta}{2}} \end{pmatrix} \end{aligned} \quad (41)$$

²⁸Nótese que se usa $\frac{\theta}{2}$ en vez de θ porque los estados con $\theta > \frac{\pi}{2}$ son equivalentes a los menores por una fase global $\xi = \pi$.

²⁹Este enfoque es una forma de interpretar la computación cuántica: definimos unas órdenes básicas y, con ellas, empezamos a construir algoritmos y a definir propiedades como la la profundidad del circuito (la cantidad de puertas usadas). Más adelante hablaremos del enfoque AHS, basado en modificar el Hamiltoniano del sistema con una evolución temporal.

³⁰A fin de aligerar la nomenclatura, prescindiremos del acento en los operadores y consideraremos suficiente denotarlos con letras mayúsculas.

³¹Como no podría ser de otra forma al ser los representantes del grupo $SU(2)$. Una introducción a la importancia de la teoría algebraica de grupos en física se puede encontrar en el excelente libro de Matthew [55], o en concreto para Mecánica Cuántica la base matemática se puede encontrar en el libro de Hall [16].

³²La razón de usar rotaciones con $\frac{\theta}{2}$ vuelve a ser la fase global $\xi = \pi$.

Y, dado que estas matrices son independientes entre sí (pues están conformadas cada una por una matriz de Pauli distinta), las matrices de rotación se pueden usar para construir cualquier operador lineal y unitario (i.e.: Son también representantes del grupo $SU(2)$). Este resultado se formaliza en el siguiente teorema.

Teorema 2.29. *Descomposición de operadores de un solo qbit*

Sea U un operador unitario para un solo qbit. Entonces existen $\alpha, \beta, \gamma, \delta \in \mathbb{R}$ tales que U se puede escribir como una composición de puertas R_x , R_y , R_z con una fase global.

Por ejemplo, usando solo puertas R_z y R_x :

$$U = e^{i\alpha} R_z(\beta) R_x(\gamma) R_z(\delta) \quad (42)$$

Observación 2.30. La fase global α actúa para desplazarnos de la semiesfera inferior a la superior, donde tenemos los estados definidos sin repeticiones por la ecuación 39.

Por otro lado, la demostración del teorema puede consultarse en el libro de Nielsen [56].

A partir de estas puertas podemos definir la puerta *Hadamard*. Esta operación de rotar el espacio respecto al eje $\hat{n} = \hat{x} + \hat{z}$ un ángulo $\xi = \pi$ genera la máxima entropía posible en un qbit en el que inicialmente conocíamos el estado:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \mid H \mid 0 \rangle = \frac{1}{\sqrt{2}} (\mid 0 \rangle + \mid 1 \rangle) \quad (43)$$

Recordemos que esta configuración de probabilidades es la de máxima entropía por lo visto en el ejemplo 2.6. Es decir, cuando las probabilidades son uniformes. De esta forma, la entropía máxima que podemos obtener de un qbit es de 1 bit.

La puerta Hadamard se puede escribir a partir de puertas R_z como:

$$H = R_z\left(\frac{\pi}{2}\right) \sqrt{X} R_z\left(\frac{\pi}{2}\right) \quad (44)$$

Donde $\sqrt{X}$ es la puerta SX:

$$\sqrt{X} \equiv \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix} = e^{i\frac{\pi}{4}} R_x\left(\frac{\pi}{2}\right) \quad (45)$$

Esta expresión obedece el teorema 2.29, y es lo que implementarán los computadores cuánticos al transpilar el circuito escrito al lenguaje de cada computador.

Cuando aplicamos puertas Hadamard a n qbits, dicha operación se denomina *Walsh-Hadamard Transform*. Esta acción corresponde a colocar el sistema completo en un estado donde todas las combinaciones de bits de salida son equiprobables. Si tenemos n qbits, esta puerta se describe como:

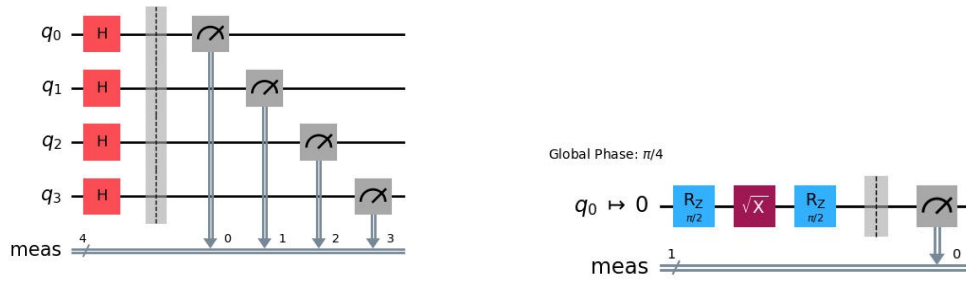
$$H^{\otimes n} \mid \vec{0} \rangle = \frac{1}{\sqrt{2^n}} \sum_{\vec{x} \in V^{\otimes n}} \mid \vec{x} \rangle \quad (46)$$

Donde $V^{\otimes n}$ corresponde al conjunto de todas las combinaciones posibles entre los estados de los qbits. Este es el sistema que usaremos para tomar cadenas de muchos bits.

Finalmente, el operador medida corresponde a realizar la proyección del estado en la base computacional ($\{\mid 0 \rangle, \mid 1 \rangle\}$). Esta acción es irreversible: una vez midamos, el estado pasa a

ser uno de los de la base y no podemos recuperarlo. La medida física acarrea ciertos errores debidos a los detectores, lo cual tendremos en cuenta en el análisis de los computadores escogidos.

El circuito cuántico que implementaremos se encuentra en la figura 4. Corresponde a un Walsh-Hadamard Transform con unas puertas de medida. Si bien inicialmente se hicieron pruebas usando un solo qbit, los tiempos de ejecución superaban los cinco minutos para secuencias de más de 10^6 bits. De esta forma, se decidió utilizar todos los qbits que tuviera el computador, aunque esto incrementa el ruido y disminuye la complejidad de Kolmogorov a la del qbit que más error tenga (recuérdese la observación 2.16).



Circuito Cuántico usado con $n = 4$ Transpilación usada por *ibm_kingston*

Figura 4: Circuito cuántico usado en la obtención de secuencias

2.3.3. Fuentes de error

Los computadores reales presentan limitaciones según el mecanismo usado, así como sus componentes. Al conjunto de todos los efectos que pueden contaminar el estado en el que se encuentren los qbits se denomina *quantum noise*. Una de las principales fuentes de ruido es la decoherencia. Esto es, que el sistema interactúe siquiera levemente con su entorno y su configuración cambie con el tiempo. Un ejemplo de decoherencia puede verse en los sistemas de espín $s = 1/2$, donde el ruido térmico genera fluctuaciones entre los estados $s_z = \pm 1/2$. Así, es necesario entender cuáles pueden ser nuestras fuentes de error en las secuencias aleatorias obtenidas.

Un buen computador cuántico debe cumplir las siguientes cuatro propiedades [56]:

1. Conserve fidedignamente la información cuántica (i.e.: Mantenga los estados)
2. Pueda realizar todas las transformaciones unitarias
3. Pueda preparar con seguridad un estado inicial
4. Capacidad de leer con fidelidad el estado final

En el caso de nuestro circuito, compuesto exclusivamente de puertas Hadamard y de medida, debemos estudiar con más detalle las posibles fuentes de error en los puntos 1,3 y 4.

La conservación fidedigna de la información cuántica está estrechamente relacionado con la decoherencia. Cada mecanismo físico presenta distintos tiempos medios en los que

el estado se puede mantener inmutable. A su vez, las operaciones en ellos también tienen diferentes órdenes temporales. En conjunto esto nos da un número limitado de operaciones que podemos aplicar en cada iteración antes de que la decoherencia deje el sistema insertible³³. Nuestro circuito cuántico sólo presenta cuatro puertas para cada qbit, así que no nos tendremos que preocupar por alcanzar estos límites. Sin embargo, existen medidas más específicas de cada computador relacionadas con la decoherencia que sí que pueden tener efecto. Estos son los tiempos T_1 y T_2 . El tiempo T_1 hace referencia al *tiempo de relajación del sistema*. Es decir, cuánto tiempo en promedio transcurre antes de que un estado excitado, por disipación con su entorno, transicione al estado $|0\rangle$. El tiempo T_2 hace referencia al *tiempo de desfase*, lo que corresponde al tiempo medio antes de que las coordenadas del sistema varíen ligeramente [80]. En nuestro caso debemos prestar mucha atención al tiempo de desfase, ya que es el que puede generar probabilidades no uniformes entre los bits 0 y 1 y, por ende, afectar negativamente a la entropía y a los tests relacionados con la frecuencia de los mismos. La tabla 2 recoge esta propiedad, entre otras, de los computadores escogidos.

Respecto a la preparación del estado inicial, hay sistemas a los que les resulta complicado asegurarse de inicializarse en un estado $|0\rangle$ puro. Por ejemplo, los sistemas basados en excitación de electrones en átomos neutros atrapados se requiere enfriar el estado en pos de evitar excitaciones al estado $|1\rangle$ en la inicialización del sistema [56]³⁴. De esta forma, debemos preocuparnos de tener inicializado el qbit en $|1\rangle$ por error. Consideremos el efecto de una puerta Hadamard al estado $|0\rangle$ y al estado $|1\rangle$:

$$\begin{aligned} H|0\rangle &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \\ H|1\rangle &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \end{aligned} \quad (47)$$

Vemos que no generará diferencia en las probabilidades de los bits, de tal forma que podemos descartar errores de inicialización como posibles fuentes de error.

Finalmente, los errores de medida son complejos y, de nuevo, dependen del mecanismo físico del computador. Debemos tener en cuenta los errores de que acarreen las puertas R_z y las R_x , pues son las usadas en la transpilación de la puerta Hadamard. IBM ofrece información muy precisa de estos errores, hasta el punto de poder consultar el error de cada puerta para cada qbit. Dado que otras empresas no ofrecen tanta información, tomaremos como error asociado a las puertas el error máximo que tenga para un qbit dado. Los errores de las puertas para cada computador usado se encuentran en la tabla 3. Los errores se expresan como grados de fidelidad del resultado experimental con respecto al teórico, indicando la probabilidad de que, dado un estado obtenido, este no corresponda con el teórico.

³³Puede consultarse en la tabla 10 del Anexo una estimación algo burda de estas propiedades para algunos sistemas.

³⁴Las superposiciones accidentales entre estados $|0\rangle$ y $|1\rangle$ son más complicadas, estando más presentes en sistemas de moléculas que no usaremos en el análisis.

2.3.4. Modelo AHS

Recientemente se están investigando computadores basados en la evolución dinámica de su Hamiltoniano. Este paradigma se conoce como *Analog Hamiltonian Simulation*. El computador Aquila, de QuEra, utiliza los estados de Rydberg de átomos neutros, de tal forma que su programación y propiedades difieren de sobremano con la computación cuántica más típica. AHS presenta muy buenas propiedades para el uso de simulaciones, machine learning y procesos que requieran interacción entre qbits [40]. Sin embargo, no hemos encontrado ningún artículo sobre su viabilidad para QRNG; probablemente porque esa facilidad de interacción de qbits actúa en detrimento para este tipo de aplicaciones. A continuación haremos una introducción ínfima al formalismo, de tal forma que podamos explicar cómo se ha implementado el QRNG y qué limitaciones presenta respecto a sus competidores.

El Modelo AHS se basa en la evolución adiabática de un sistema cuántico para programar estados como en un computador cuántico. El hamiltoniano usado es de la forma [38]³⁵:

$$\hat{H}(t) = \frac{1}{2}\Omega(t) \left(\sum_{j=1}^N e^{i\phi(t)} |g_j\rangle \langle r_j| \right) - \Delta(t) \sum_{j=1}^N \hat{n}_j + C_6 \sum_{j < k} \frac{\hat{n}_j \hat{n}_k}{x_{ij}^6} \quad (48)$$

Que corresponde a un sistema de dos niveles y N partículas. Aquila utiliza electrones encerrados en átomos neutros como qbits, oscilando entre el estado base $|g\rangle$ y el de Rydberg³⁶ $|r\rangle$ a partir de una oscilación de Rabi con frecuencia Ω ³⁷. El primer sumando del Hamiltoniano corresponde a la oscilación de cada qbit entre el estado base y el de Rydberg. Dado que buscamos equiprobabilidad entre estados el valor de la fase ϕ lo podemos forzar a cero por simplicidad. Respecto a la frecuencia de Rabi Ω , existe una expresión general que la define a partir de la amplitud del campo externo junto a la diferencia de energía entre los estados. Sin embargo, para el caso concreto de Aquila se puede simplificar a [40]:

$$\Omega = -dE_0 \quad (49)$$

Donde E_0 es la amplitud del campo eléctrico suministrado por el láser, y d es el momento dipolar generado por la oscilación del electrón entre los estados de energía.

El segundo sumando, por otro lado, modeliza el efecto del bombeo del láser. La variable Δ se denomina *detuning*, y define la diferencia entre la frecuencia del láser ω_0 y las frecuencias asociadas a las energías de los estados (w_r y w_g). De esta forma:

$$\Delta = w_0 - (w_r - w_g) \quad (50)$$

$\hat{n}$ corresponde al operador número, definido en el Formalismo de la Segunda Cuantización³⁸

³⁵Usamos unidades de $\hbar = 1$

³⁶Un estado de Rydberg es un estado muy energético de un electrón ligado a un átomo, solo accesible mediante bombeo láser [38].

³⁷Una oscilación de Rabi corresponde a oscilaciones en las coordenadas de los dos autoestados de energía debido a la interacción con un campo externo dependiente del tiempo. En el caso de Aquila, corresponde a la interacción con un campo eléctrico $E(t)$ que controlamos [40].

³⁸Una introducción muy somera a la segunda cuantización se encuentra en la referencia clásica de Sakurai

como:

$$\hat{n}_j \equiv |r_j\rangle \langle r_j| \quad (51)$$

Finalmente, el último sumando corresponde a una interacción de Van der Waals: una interacción a pares entre los qbits. Nótese que decae con la potencia sexta de la distancia, de tal forma que, si bien no se pueden considerar los qbits independientes entre sí, esto sólo será cierto si se encuentran muy cercanos entre ellos.

Aquila fija las variables Ω , Δ y C_6 del sistema para átomos de ^{87}Rb moviendo los átomos a partir de pinzas ópticas, y suministrando energía a los electrones con láseres de diversa frecuencia [40]. Los átomos de ^{87}Rb tienen como constante asociada a Van der Waals $C_6 = 862,690 \cdot 2\pi \text{ MHz } \mu\text{m}^6$. Existe una relación entre estas tres variables y el efecto denominado *Rydberg Blockade*. Cuando los átomos están a una distancia menor al *Radius Blockade*, la interacción de Van der Waals se impone al bombeo por láser y causa que tener ambos átomos cercanos entre sí en el estado de Rydberg sea altamente penalizado. Este radio se calcula como:

$$R_b = \left(\frac{C_6}{\sqrt{\Omega^2 + \Delta^2}} \right)^{\frac{1}{6}} \quad (52)$$

Este efecto puede ser muy útil, pero para nuestro modelo es altamente perjudicial ya que descarta la hipótesis de independencia entre bits. De tal forma, debemos minimizarlo tomando valores de Ω y Δ muy grandes en comparación a C_6 , pero, a su vez, que nos permitan mantener la equiprobabilidad entre estados.

A modo de resumen, tenemos cinco parámetros del computador para modificar el Hamiltoniano a nuestro gusto. La frecuencia de Rabi se modifica con la amplitud del láser de bombeo y modifica las probabilidades entre los estados. La frecuencia ϕ la tomaremos 0 por simplicidad. El detuning expresa cuán cerca nos encontramos de tener resonancia en la frecuencia de bombeo del láser respecto a la frecuencia de transición entre estados. La constante C_6 expresa la intensidad de la interacción de Van der Waals entre átomos, y está fija para Aquila en $C_6 = 862,690 \cdot 2\pi \text{ MHz } \mu\text{m}^6$. Finalmente, podemos modificar la distancia entre átomos; trataremos de buscar la configuración que, usando todos los qbits, maximice la distancia entre ellos en pos de evitar el efecto de *Rydberg Blockade* donde los qbits dejarían de ser independientes entre sí.

Debido a la interacción de Van der Waals y el efecto de *Rydberg Blockade* es muy optimista considerar que los estados de cada qbit adyacente sean independientes entre sí. Así, esperamos que este computador presente malos resultados en los tests estadísticos (en especial en los relacionados con frecuencias en la cadena). Sin embargo, hemos decidido incluirlo en el análisis porque puede servir como verificación de los resultados. Si viéramos que Aquila muchos mejores resultados que el resto de computadores, nos indicaría un posible error en la implementación de los tests o de los circuitos cuánticos. Por otro lado, si vemos resultados

[36], sección 7.5). Sin embargo, al ser un concepto fundamental en Teoría Cuántica de Campos el lector puede consultar una descripción más asequible en el libro de Lancaster y Blundell [83], concretamente los capítulos dos y tres.

coherentes con esta interacción entre estados, estaremos reforzando la verosimilitud de las pruebas.

2.3.5. Circuito usado para AHS

Veamos el Hamiltoniano que usaremos para generar estados equiprobables. Si consideramos que tenemos los qbits lo suficientemente lejos de tal forma que podemos despreciar el efecto de Van der Waals, y forzamos Ω y Δ constantes, el Hamiltoniano para un qbit se reduce a [40]:

$$H(t) = \frac{1}{2} (\Omega |g\rangle \langle r| + \Omega^* |r\rangle \langle g|) - \Delta |r\rangle \langle r| \quad (53)$$

Con este Hamiltoniano el estado de un qbit se puede describir como el estado en la esfera de Bloch (véase ecuación 39), donde $\Omega t = \theta(t)$ y $\Delta \cdot t = \phi(t)$. Podemos tomar $\Delta = 0$ y centrarnos exclusivamente en el valor de Ω . Queremos un valor de Ω lo más grande posible por dos razones. La primera es disminuir el Radius Blockade y, la segunda, es que podremos alcanzar antes el valor de $\theta = \pi/2$ ³⁹ antes, evitando los errores de decoherencia.

Para terminar de especificar los valores (distancia entre átomos y valor de Ω) recurrimos a las especificaciones de la tabla 1. La tabla 9 del Anexo presenta más especificaciones del dispositivo.

Especificaciones	Altura Grid Atómico (H , μm)	Anchura Grid Atómico (W , μm)	Ω_{max} (rad/s)
Valor	76.000	75.000	$1,58 \cdot 10^7$

Tabla 1: Especificaciones usadas de Aquila [40].

Primero estudiemos el espaciado máximo entre qbits. Despreciemos el micrómetro extra de altura del grid, y supongamos que tenemos una superficie en la que queremos colocar N qbits, maximizando la distancia entre ellos. Por simetría consideramos distancias simétricas en el eje x e y , denominadas d . Esto corresponde a querer cubrir el área a partir de subáreas cuadradas. Si el área grande es A , y el área pequeña es a , se debe cumplir que:

$$N = \frac{A}{a} \quad (54)$$

Pero al ser cuadrados $A = A \cdot W \simeq W^2$ y $a = d^2$. De esta forma:

$$d(N) = W \frac{1}{N} \quad (55)$$

Si usamos los 256 qbits, la distancia máxima a la que podemos separarlos es $d_{max} = 4,6875 \mu m$. Si usamos diez veces menos el valor máximo de Ω (para no estar en los límites del computador y no tener que modificar la frecuencia de forma muy brusca al inicio y al final). Por la ecuación 52:

$$R_b = \left(\frac{2\pi \cdot 8,62690 \cdot 10^5 \text{ (rad/s)}}{1,58 \cdot 10^6 \text{ (rad/s)}} \right)^{\frac{1}{6}} \mu m \simeq 1,2281 \mu m \quad (56)$$

³⁹Recordemos que el valor es $\theta = \frac{\pi}{2}$ porque en la expresión de un qbit se trabaja con $\frac{\theta}{2}$

Por lo que superamos el Radius Blockade. Consideraremos a los qbits independientes entre sí, pero es posible que aún así los efectos de interacción entre qbits se hagan presentes. Usaremos $\Omega = 1,58 \cdot 10^6 \text{ rad/s}$. Respecto al tiempo que debe transcurrir antes de medir, podemos obtenerlo sabiendo que debemos alcanzar $\frac{\theta}{2}$. Pero debemos tener en cuenta el hardware tarda un tiempo en alcanzar el Ω deseado, y luego debe volver a bajar para poder medir. De esta forma, lo que realmente debemos hacer es imponer la condición de la integral:

$$\int_0^T \Omega(t) dt = \frac{\pi}{2} \quad (57)$$

Donde T es el tiempo final (en el que mediremos). Si usamos un crecimiento sinusoidal de la forma:

$$\Omega(t) = \Omega_{max} \sin^2 \left(\frac{\pi t}{T} \right) \quad t \in [0, T] \quad (58)$$

Podemos obtener T como:

$$\int_0^T \sin^2 \left(\frac{\pi t}{T} \right) dt = \frac{\pi}{2} \implies T = \frac{\pi}{\Omega} \quad \text{Uds Tiempo} \quad (59)$$

Con esto tenemos todos los parámetros para obtener los bits. El código al completo puede consultarse en el Anexo. Sin embargo, queremos destacar un último matiz: en el ordenador de QuEra, obtener un resultado de bit 0 corresponde a que el electrón estaba en el estado excitado o se había escapado de la trampa, mientras que un bit 1 corresponde a que el electrón estaba en el estado fundamental [40]. Esto corresponde a una inversión semántica respecto al formalismo de puertas cuánticas. Por ejemplo, ahora que todos los qbits de Aquila estén inicializados en el estado base corresponde a que se inician todos en el estado $|g\rangle = |1\rangle$. Nosotros invertiremos todos los resultados obtenidos por Aquila para tener el mismo convenio en todos los ordenadores.

3. Evaluación de la Aleatoriedad

La tabla 2 presenta los RNG usados para el TFG, indicando sus características más destacables⁴⁰. Una información más completa sobre el estado del arte de cada mecanismo físico se encuentra en el paper de Awschalom et. al. [20].

Computador	Mecanismo	Empresa	Qbits	T_1 (μs)	T_2 (μs)
ibm_kingston	Superconductores	IBM	156	495.70	536.96
Aquila	Átomos Neutros	QuEra	256	$7,5 \cdot 10^4$	$5 \cdot 10^3$
IBEX Q1	Iones Atrapados	AQT	12	$(1 \sim 100) \cdot 10^6$	
Cepheus TM -1-108Q	Superconductores	Rigetti	107	24	11
StatevectorSampler	PRNG (PCG64)	NumPy	~ 20 (según RAM)	∞	∞
os.urandom	CSPRNG (BCryptGenRandom)	Microsoft	libre	∞	∞

Tabla 2: Sistemas escogidos. Propiedades en la documentación [35] [40] [72] [7] [61] [63] [5]

La tabla 3 recoge los errores específicos de las puertas. Estos errores se tendrán en cuenta en el análisis de los resultados, pero no se cuantificarán en los valores obtenidos de aleatoriedad, complejidad ni p-values.

Computadora	R_z Err (%)	R_x Err (%)	Readout Error (%)	Notas
ibm_kingston	0	0.1286	1.03	
Aquila	–	–	$p(\text{Falso } 1\rangle) = 0.08$ $p(\text{Falso } 0\rangle) = 0.01$	Al ser otro paradigma, sus errores se encuentran en la tabla 10 del Anexo.
IBEX Q1	0.01		–	
Cepheus TM -1-108Q	0.53 ± 0.11		–	No hay información pública sobre el error de medida del dispositivo [72].

Tabla 3: Errores específicos de las puertas [35] [40] [7] [72]

3.1. Procedimiento

La determinación de aleatoriedad se realizará con la comprobación y cuantificación de las propiedades de *impredecibilidad*, *independencia* y *uniformidad*. Dichas propiedades se estiman mediante tests de p-values de la suite NIST, así como con medidas extra de Entropía y Complejidad de Kolmogorov mediante los procesos estudiados en el fundamento teórico. Finalmente, para reforzar la fiabilidad de los resultados de la suite, realizaremos Tests de Segundo Orden con el procedimiento propuesto por Shen de la sección 2.2.3.

La tolerancia usada en los tests y en los valores estadísticos será de $\epsilon = 0,01$ por ser el estándar en criptografía [12]. Los códigos usados, así como los datos brutos extraídos, pueden consultarse en el Anexo.

⁴⁰IBM ofrece también los valores de error de medida de cada qbit del computador. Se podría tratar de adaptar el código para no usar los qbits problemáticos.

3.1.1. Extracción de secuencias

Primero realizamos una estimación preliminar de la longitud óptima de las secuencias, así como de la cantidad de secuencias necesarias de cada computador para obtener resultados significativos. Idealmente queremos una longitud de secuencias lo más grande posible. Sin embargo, debemos tener en cuenta las limitaciones de ejecución. La implementación eficiente del circuito cuántico será crítica para maximizar la longitud de las secuencias.

IBM ofrece diez minutos mensuales para ejecutar códigos. AWS, por otro lado, ofrece un total de 100 créditos para ejecutar código en los diferentes computadores cuánticos, pero cobra por número de shots pedidos. Además, sus computadores tienen un número mínimo y máximo de shots que se pueden enviar, distinta según el computador. Estas limitaciones hacen que el proceso de adquisición de secuencias en AWS sea tortuosa y costosa. Como cada computador tiene sus limitaciones, el código de extracción de secuencias deberá adaptarse a este caso para calcular el número de tasks necesarias para rellenar cada fichero con el número de bits deseado. Esto se realizará con la división entera n_{shots}/l_s , siendo l_s el número máximo de shots que se puede mandar por tarea en el computador objetivo. Los bits correspondientes al resto de la división se desecharán.

Inicialmente probamos a ejecutar con *ibm_marrakesh* un circuito de un qbit con puertas Hadamard y medida como los de la figura 4 y 10^6 shots, llevando un tiempo de ejecución de 4 minutos y 20 segundos. Dado que necesitaríamos como mínimo decenas de secuencias de esta forma, no era viable realizar los circuitos con un solo qbit. Es por ello que nos decantamos por el uso de Walsh-Hadamard Transforms, usando el número de qbits de los que dispusiera el ordenador y alcanzando velocidades de 18 segundos para secuencias de 10^7 bits, además de requerir menos shots ($n_{shots} = \lfloor \frac{n_{secuencia}}{n_{qbits}} \rfloor$). Este enfoque acarrea menos Complejidad de Kolmogorov y empeora ligeramente los resultados de aleatoriedad, como discutimos en la observación 2.16. El efecto también se ve reflejado en los valores de entropía obtenidos, pasando en cadenas de 10^6 bits de entropías de $H_1 = 0,99996$ bits a $H_{100} = 0,99967$ bits, un orden de magnitud mayor. Sin embargo, este error se puede disminuir usando cadenas más largas: para cadenas de 10^7 bits volvemos a recuperar el orden de magnitud con entropías de $H_{156} = 0,99983$ bits. De esta forma, tomaremos ficheros de 10^7 bits siempre que podamos.

La simulación de Qiskit solo es capaz de soportar 20 qbits antes de hacer overflow⁴¹, de tal forma que la creación de secuencias es más lenta. Finalmente, a partir de pruebas límite el generador del sistema operativo puede generar sin problemas secuencias arbitrariamente grandes en un tiempo más que competitivo.

Respecto a la cantidad de ficheros, dado que queremos aplicar tests de segundo orden debemos generar una cantidad de ficheros lo suficientemente grande como para poder producir distribuciones estadísticas con los resultados. Dado que la mitad de los ficheros se utilizarán para el entremezclado *xor* de la propuesta de Shen, consideramos adecuado generar 100 ficheros para cada computador.

En conclusión, en pos de minimizar los errores de entropía y de los tests, obtendremos

⁴¹Esto es porque en la simulación se almacenan 2^n amplitudes complejas (las coordenadas de los estados).

ficheros de 10^7 bits para cada computador. Para poder tener distribuciones estadísticas mínimamente significativas en el test de segundo orden, impondremos un mínimo de 100 ficheros de 10^7 bits.

3.1.2. Coste Económico de los Computadores

Dado que AWS tiene criterios muy estrictos con la ejecución de los computadores, es importante conocer cómo incrementa el coste de económico con el número de bits deseado. Para ello diseñamos las *funciones costo*. Aprovecharemos estas funciones para comparar la viabilidad económica del algoritmo con el escalado de bits.

IBM cobra por minutos ejecutados en el computador. De esta forma, la función costo será una recta de la forma:

$$F_{king}(n) = c_t \cdot t(n) \quad (60)$$

Donde c_t es el coste de IBM por unidad de tiempo, y t es el tiempo de ejecución. El coste por minuto de IBM depende del plan. En el plan de pago más simple (Pay as you Go), es de 96 \$/min [35]; será el que usaremos en la comparación. Podemos escribir el tiempo de ejecución en función a los bits pedidos si suponemos un incremento lineal ⁴². Si usamos unidades de segundo:

$$F_{king}(n) = \frac{96}{60} \frac{n}{v_s} \quad \$ \quad (61)$$

Donde ahora v_s es la tasa de generado de bits por segundo (unidades de *bit/s*). Para hacer una estimación preliminar del coste calculamos la tasa de generado para un fichero de 10^7 bits. Al tardar 18 s, tenemos una tasa de $v_s \simeq 5,555 \cdot 10^5$ bits/s, pudiendo obtener los 100 ficheros necesarios a lo largo de esta investigación. La figura 7 muestra la función costo con la tasa de generado final obtenida.

Respecto a AWS, debemos implementar una función que, dados límites superior e inferior de shots de cada computador, así como sus precios, compute el mínimo precio de la tarea. Dado que el coste por task siempre es mayor al coste por shots, habremos minimizado el coste cuando hayamos minimizado las tasks. De esta forma, usaremos el máximo número de shots posibles por tarea. Los bits restantes se generarán en una última tarea. Se pueden dar dos casos: o bien los bits restantes requieren más shots que el mínimo, o bien menos y por tanto no puede ejecutarse tal cual. En ese último caso, las dos últimas tareas se repartirán los bits. Así, tendremos una función a trozos de la forma:

$$F_{AWS}(s \equiv n // \#qbits) = \begin{cases} c_T(s // l_s + 1) + c_s [l_s(s // l_s) + s \% l_s] & s \% l_s \geq l_i \\ c_T(n // l_s + 2) + c_s [l_s(s // l_s) + s \% l_s] & s \% l_s < l_i \\ c_T + c_s \cdot s & s // l_s = 0 \end{cases} \quad (62)$$

Donde c_T es el coste por task, c_s el coste por shot, l_s la cota superior de shots por task, y l_i

⁴²Sin más información del software y hardware de los computadores es difícil asegurar que el crecimiento temporal vaya a seguir esta tendencia.

la cota inferior. $n//l_s + 1$ corresponde al número de tasks completas que debemos realizar⁴³. Finalmente, $n \% l_s$ es el resto de bits que se deben conseguir con la última task. La ventaja de esta familia de funciones es que no dependen de resultados empíricos, de tal forma que podemos ya obtenerlas para cada computador de AWS. La figura 5 recoge las funciones para cada computador. En el Anexo, la tabla 11 indica los parámetros de cada computador.

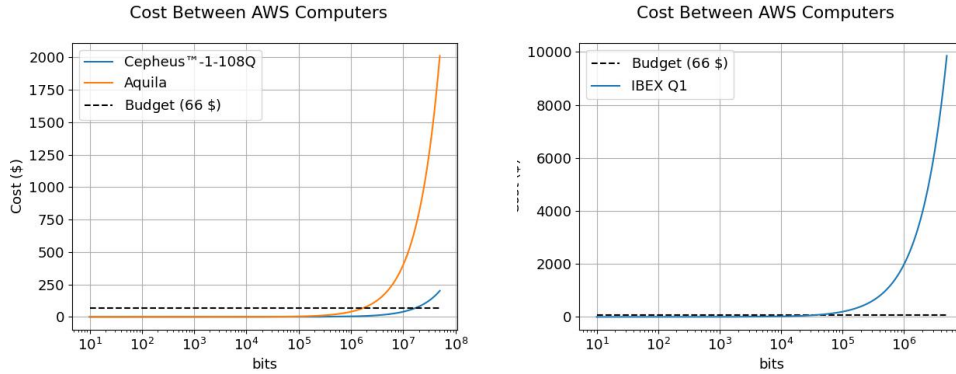


Figura 5: Comparación de las Funciones Costo de los computadores de AWS.

Separamos IBEX Q1 porque su menor cantidad de qbits y mayor coste por shot hace que trabaje con varios órdenes de magnitud de bits por debajo. Usamos eje x logarítmico.

Al tener solo 200 créditos⁴⁴ que debemos repartir en 3 computadores, cada uno tiene un presupuesto máximo de unos 66 créditos (línea discontinua de las figuras). Este presupuesto nos limita de varias formas. Para empezar, será inviable realizar tests de segundo orden a los computadores de AWS, pues no podremos más que conseguir un fichero lo suficientemente grande como para aplicar tests de primer orden. Además, sólo obtendremos un fichero 10^7 bits en el caso de Cepheus. Generaremos un fichero de 10^6 bits para *Aquila*, y para *Ibex* uno de $6 \cdot 10^4$ (superando el presupuesto con 118 \$ para Ibex).

En conclusión, el coste económico de los computadores sólo nos permite alcanzar la situación ideal de 100 ficheros de 10^7 bits (obtener 10^9 bits para el análisis) con PRNG y con QRNG de IBM. Para AWS sólo podremos aplicar tests de primer orden con 10^7 bits para Rigetti, 10^6 bits para QuEra, y $6 \cdot 10^4$ bits para AQT.

3.1.3. Estimación de entropía

Estudiemos la entropía que esperaríamos del sistema. Si consideramos la distribución de probabilidades asociada a las secuencias de n , bits, deberíamos considerar la entropía asociada a la expresión 46. Con la definición de Entropía de Shannon obtendríamos la función $H(n)$ de la figura 6.

Podemos ver que la entropía del sistema tiende a cero rápidamente por efecto del factor 2^{-n} . Así, una estimación directa de la entropía precisaría de una cantidad abrumadora de secuencias para llegar a un resultado siquiera confiable. Nosotros estimaremos la entropía

⁴³El 1 añadido es porque cuando $n < l_s$ estamos realizando aún así un task.

⁴⁴Realizamos las tareas opcionales de AWS que aportan otros 100 créditos.

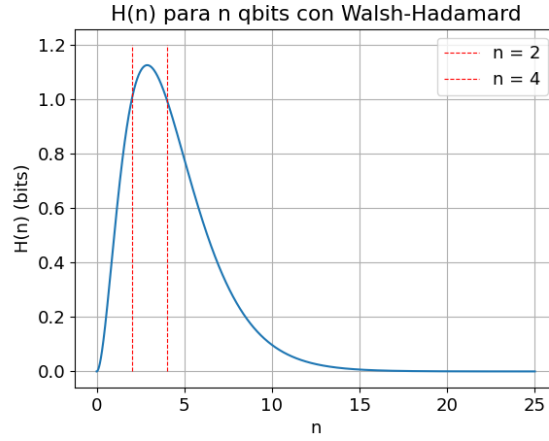


Figura 6: Entropía de Walsh-Hadamard para n qubits. Es de la forma:

$$H(n) = \frac{n^2}{2^n}$$

asociada a la aparición de bits 0 y 1 en cada secuencia, en vez de comprobar la frecuencia de las secuencias per se, recuperando la estimación teórica de entropía del ejemplo 2.6:

La secuencia presentará impredecibilidad desde el formalismo de Teoría de la Información cuando la entropía asociada a la aparición de bit 0 y 1 sea de $H \in [1 - \epsilon, 1]$, con $\epsilon = 0,01$ el parámetro de confianza ya fijado.

El procedimiento será como sigue:

1. Leemos a bloques el fichero de bits. Denominemos al bloque b .
2. Contamos en b la cantidad de 1s. Obtenemos la cantidad de 0s de ese bloque como $|b| - 1s$. Almacenamos 1s, 0s y $|b|$ en contadores por separado, denominados $|1|$, $|0|$, $|B|$.
3. Cuando esté el fichero leído al completo, computamos las probabilidades de obtener un bit 1 en la secuencia y un bit 0 en la secuencia como:

$$P(1) = \frac{|1|}{|B|} \quad P(0) = \frac{|0|}{|B|} \quad (63)$$

4. Estimamos H_{min} como $\max\{P(1), P(0)\}$
5. Estimamos H como $H = -P(0) \log(P(0)) - P(1) \log(P(1))$

3.1.4. Estimación de Complejidad de Kolmogorov

Buscamos plantear una cota superior de Complejidad de Kolmogorov de las secuencias. Para ello utilizamos los algoritmos de compresión sin pérdida de información más utilizados. Usaremos el algoritmo de Huffman, el compresor de Cadenas de Markov, y el algoritmo PPM [67][52] [34]. Procedemos de la siguiente forma:

1. Cargamos en memoria la secuencia de bits con tipo de dato 'bit'. De esta forma minimizamos el riesgo de overflow de memoria. Lo denominamos y .
2. Leemos la longitud de la secuencia, $|y|$.

3. Comprimimos con las librerías citadas la secuencia bajo los diferentes algoritmos. Medimos la longitud de la secuencia comprimida⁴⁵.
4. Tomamos como Complejidad de Kolmogorov la longitud mínima de las tres secuencias comprimidas. Con ella calculamos el valor de la función de deficiencia d de acuerdo a la definición 2.22.
5. A la función de deficiencia le aplicamos el test planteado en la *Observación* 2.27, cuya cota K determinaremos ahora. Devolveremos el valor de la función de deficiencia, así como la probabilidad máxima de habernos encontrado esa deficiencia bajo hipótesis de aleatoriedad:

$$p(x) = 2^{-d(x)} \quad (64)$$

Veamos qué valor de k debemos usar para que la probabilidad de encontrarse con una secuencia de complejidad mayor a k sea de $\epsilon = 0,01$. Basta con resolver la ecuación:

$$2^{-k} = 0,01 \implies k = -\log(0,01) = \log(10^2) \simeq 6,64 \quad (65)$$

Si $d < 6,64$, el test aseverará que las secuencias cumplen los estándares de independencia y de uniformidad para aceptar aleatoriedad bajo AIT. Es equivalente, bajo formalismo de tests estadísticos, a decir que x falla el test si $p(x) < \epsilon$.

3.1.5. Tests Estadísticos

Los tests estadísticos que aplicaremos son todos los de la suite NIST implementados por Kho [43], salvo los tests Overlapping. La razón es que, en un principio, no tenemos ningún patrón en concreto que sospechemos que se pueda estar repitiendo. El Test Binary_Matrix se realiza con una función propia, pues Kho no terminó dicho script. Usamos como tolerancia de los tests $\epsilon = 0,01$. Así, obtendremos un conjunto de resultados de los tests para cada fichero obtenido. Para los computadores con múltiples ficheros programamos un algoritmo que resuma los resultados con tratamiento estadístico.

3.1.6. Tests de Segundo Orden

Para reforzar la fiabilidad de las conclusiones de los tests NIST, aplicaremos un test de segundo orden basado en Kolmogorov-Smirnov y la propuesta de Shen [77]. El test consistirá en comprobar que la distribución de p-valores de los tests obtenida para Kingston y Statevector corresponde a la distribución que se puede obtener con el generador BCrypt usado por Microsoft⁴⁶. Todos los detalles de la implementación se encuentran explicados al final de la sección 2.2.3. La cota $K(\epsilon)$ mencionada en la ecuación 37 para $\epsilon = 0,01$ es de $K(0,01) \simeq 1,955$. De esta forma, consideraremos que los resultados de un test de la suite

⁴⁵Nótese que usaremos el máximo nivel de compresión posible, sin importar el tiempo de procesado, pues la Complejidad no tiene en cuenta el coste de tiempo del algoritmo de descompresión.

⁴⁶Como ya mencionamos, la discusión de si las secuencias obtenidas por BCrypt se pueden considerar aleatorias no influye en el test, pues con aseverar independencia entre los generadores es suficiente para aplicar hipótesis de aleatoriedad exclusivamente a las secuencias problema.

NIST son robustos cuando la distancia entre las distribuciones de p-values obtenidas D cumplan $D < 1,955$.

3.1.7. Estudio de la tasa de generado

Para el estudio de la tasa de generado trataremos de extraer los tiempos usados de hardware cuántico. De esta forma no añadiremos ruido como el tiempo de conexión a los computadores, el de transpilación o el de la escritura de ficheros. Este valor sólo está disponible en IBM, de tal forma que en el resto de computadores lo aproximaremos restando los tiempos de cola con el tiempo de hardware guardado en la metadata del resultado. Cuando ya se hayan obtenido todas las secuencias, calcularemos una media de todos los tiempos usados para cada computador por separado, incluyendo un error estadístico. Este valor se dividirá por el número de bits promedio de las secuencias para tener el tiempo de generado de bit aleatorio promedio⁴⁷. Este valor se calculará también para PRNG.

3.2. Resultados

Los bits totales obtenidos para cada sistema se encuentran en la tabla 4.

Computador	Bits totales	Ficheros	Bits por Fichero
Aquila	999 936	1	999 936
Ibex	66 145	1	66 145
Cepheus	9 937 711	1	9 937 711
Kingston	1 099 990 320	110	9 999 912
StatevectorSampler	1 000 000 000	100	10 000 000
os.urandom	1 500 000 000	150	10 000 000

Tabla 4: Bits totales y repartición de ficheros

3.2.1. Entropía

La tabla 5 recoge los valores de entropía medios obtenidos para todos los ficheros, donde el error corresponde al error estadístico de la media. Se puede comprobar que Aquila presenta unos resultados pésimos, lo que es indicativo de que se ha producido Rydberg Blockade⁴⁸. En el simulador de Aquila se obtenían relaciones entre 0s y 1s de 65-45 %. Reducir el número de bits uniformiza las probabilidades, pero entonces el coste del computador se dispara. Es por ello que nos vemos forzados a continuar el análisis con esta limitación.

Respecto a Ibex, vemos que los resultados son aceptables pese a haber usado pocos bits.

⁴⁷Idealmente usaremos el mismo número de bits para todas las cadenas, pero por completitud incluimos el cálculo del número de bits promedio.

⁴⁸Sabemos que es Rydberg Blockade porque las frecuencias de 0s son abrumadoramente mayores a las de 1s. Esto es, los estados de Rydberg están perjudicados por sus vecinos. Los datos sobre las frecuencias se pueden obtener con la función `Compute_Prob` en los bits de Aquila, ambos en el Anexo.

Es muy probable que si se hubieran alcanzado los 10^7 bits hubieran obtenido entropías equivalentes a sus rivales. Aún así supera la condición de aleatoriedad a partir de entropía para H . Finalmente, tanto Kingston como los PRNG devuelven resultados buenos, viendo mayor uniformidad en los PRNG por tener un error estadístico de un orden inferior al de IBM. En los ficheros resumen, además (incluidos en el Anexo) se adjunta el valor de las entropías en el peor caso encontrado en los ficheros. En los tres computadores se sigue superando la condición de aleatoriedad por entropía incluso en el caso más pesimista.

Comp (ficheros)	Aquila (1)	Ibex (1)	Cepheus (1)	Kingston (60)	Statevector (50)	urandom (50)
H_{min} (bits)	$0,145 \pm 0$	$0,886 \pm 0$	$0,92223 \pm 0$	$0,9909 \pm (5 \cdot 10^{-4})$	$0,99905 \pm (9 \cdot 10^{-5})$	$0,99876 \pm (1,3 \cdot 10^{-4})$
H (bits)	$0,45436 \pm 0$	$0,99513 \pm 0$	$0,99779 \pm 0$	$1,0000 \pm (5 \cdot 10^{-4})$	$1,00000 \pm (9 \cdot 10^{-5})$	$1,00000 \pm (1,3 \cdot 10^{-4})$

Tabla 5: Entropías obtenidas. Las cifras significativas se escogen de acuerdo al error.

3.2.2. Complejidad

Las complejidades se recoge en la tabla 6. Todos los RNG fallan el test, pues la deficiencia es muchos órdenes mayor a los 6,67 de la sección 3.1.4. Es interesante comprobar que incluso los PRNG criptográficos son mayormente compresibles. No creemos que se haya sido excesivamente estricto con la condición de incompresibilidad, si no que el ideal de aleatoriedad como incompresibilidad no es tan aplicable a secuencias finitas como pensábamos. De todas formas, el hecho de que los QC (salvo, por supuesto, Aquila) presenten incompresibilidades parecidas a los PRNG indica que a nivel de complejidad de Kolmogorov trabajan con niveles de aleatoriedad similares.

Comp (ficheros)	Aquila (1)	Ibex (1)	Cepheus (1)	Kingston (60)	Statevector (50)	urandom (50)
C (bits)	$61\,741 \pm 0$	$9\,529 \pm 0$	$1\,305\,454 \pm 0$	$1\,315\,560 \pm 9$	$1\,315\,633 \pm 7$	$1\,315\,629 \pm 8$
Deficiencia	$938\,195 \pm 0$	$56\,616 \pm 0$	$8\,632\,257 \pm 0$	$8\,684\,352 \pm 9$	$8\,684\,367 \pm 7$	$8\,684\,371 \pm 8$
Porcentaje Incompresible (%)	$6 \cdot 10^{-4} \pm 0$	$14,40 \pm 0$	$13,13 \pm 0$	$13,15572 \pm (2 \cdot 10^{-5})$	$13,15633 \pm (1 \cdot 10^{-5})$	$13,15629 \pm (1 \cdot 10^{-5})$
p-Values	0.0					

Tabla 6: Complejidades obtenidas. Las cifras significativas se escogen de acuerdo al error.

3.2.3. Aplicación de NIST test suite

Dada la cantidad de tests de la suite, los resultados se pueden consultar en el Anexo. Aquila presenta un fracaso absoluto en todas las pruebas, salvo en los tests de Random Excursion y Linear Complexity. El fracaso en Random Excursions nos servirá para comentar

un detalle: superar ciertos tests de Random Excursions mientras se fracasan en otros es un indicativo de asimetría entre los 0s y 1s de la secuencia. La tabla 7 recoge los valores aproximados para Aquila de este test.

Paso del Random Walk	-4	-3	-2	-1	+1	+2	+3	+4
p-value	10^{-14}	10^{-10}	0.001	0.70	0.96	0.70	0.70	1

Tabla 7: *P-values de Random Excursions Test para Aquila*

Si a la asimetría entre valores negativos y positivos le sumamos que el p-value del test Cusum es $p = 0$, es claro que hay algo muy mal en la secuencia. Que el resultado para valores negativos sea de probabilidades ínfimas mientras que en el positivo sea alta indica que hay más 0s que 1s en la secuencia, pues se está pasando múltiples veces por los valores más extremos negativos y nada por los positivos. En una secuencia aleatoria se presentaría un Random Walk, donde en el límite tenemos una distribución binomial con media en $x = 0$ [70]. Los valores -4 y +4 apenas serían visitados, y es por ello que tenemos un falso positivo en el +4: es cierto que apenas se está visitando $x = +4$, pero por los motivos equivocados. Este razonamiento se verificó con recuento de bits en esta secuencia y en la de los otros computadores, lo que refuerza la fiabilidad de los resultados. Respecto al buen resultado de Aquila en el Linear Complexity test, creemos que se debe a que el Rydberg Blockade no impide que, para secuencias de 0s grandes, la probabilidad de obtener un 1 sea de nuevo 50/50. De esta forma se impide encontrar un LFSR lo suficientemente pequeño que tenga en cuenta esta aleatoriedad a grandes distancias en la secuencia.

Los resultados de Ibex resultan prometedores: los p-values de el test Binary Matrix y Linear Complexity superar con creces la cota, con $p \simeq 0,320$ y $p \simeq 0,752$, respectivamente. Sin embargo, se ve que hay una inclinación a tener más 1s que 0s, pues Cusum se falla con $p \simeq 0$ y se ve que para Random Excursions tenemos un mínimo de pvalues en +2, con $p = 0,0244$. Aún así, todos los Random Excursions se superan con éxito, por lo que la desviación debe ser leve. En efecto, si se calcula la desviación esta resulta en torno a un 8 % más 1s que 0s. Esta desviación también es la causa de que fracasen todos los tests relacionados con las frecuencias, como son el test Monobit, Within a Block o los Approximate Entropy. Sin embargo, el Spectral se supera con $p = 0,1522$. Debemos tener en cuenta que los resultados de Ibex no son concluyentes por la baja cantidad de bits. El test de Maurer, por ejemplo, requiere un mínimo de 387 840 bits [12], y por ende no es aplicable a este caso.

Respecto a Cepheus, este computador fracasa en todos los tests de Frecuencia con valores de $p \simeq 0$. Es probable que se deba al uso de los 106 qbits del computador, como ya mencionamos en el caso de la entropía en IBM durante la sección anterior. Al usar tantos qbits acabamos acumulando el error por decoherencia de todos los qbits. Los tests de Random Walk y Entropía devuelven resultados contradictorios. Por un lado, el test de Approximate Entropy devuelve $p = 0$, pese a tener una entropía bit a bit muy buena. A su vez, el test Cusum devuelve $p = 0$, aunque los tests de Random Excursions se superan con valores bastante uniformes. El espectral tampoco revela ningún patrón predominante, con $p \simeq 0,22$.

De esta forma, de haber alguna secuencia repetida, ésta debe de ser por una abundancia de más 1s que de 0s. El test Maurer refuerza la idea, con un valor $p \simeq 10^{-59}$. Finalmente, la complejidad lineal parece ser suficiente por superar los tests de Binary Matrix y Linear Complexity. En conclusión, Cepheus está muy cerca de cumplir los estándares de aleatoriedad impuestos en el fundamento teórico, pero la decoherencia parece estar generando leves patrones no lineales.

Kingston, de IBM, parece presentar un problema parecido. IBM también falla los tests de frecuencias, pero no de forma tan marcada. Supera el test de Frequency Within a Block y Longest One Block, aunque falle Monobit y Run Test. A diferencia de Cepheus, que utiliza su mismo mecanismo físico, Kingston falla el test espectral con $p \simeq 10^{-80}$. Fallan también los tests de Approximate Entropy y Cusum, aunque supera los tests de Random Excursions. Es interesante, además, comentar que los p-values de los tests de Random Excursions se encuentran todos en torno a $p = 0,5$ (salvo $x = -4$ y $x = -9$). Esto indica una alta uniformidad. Finalmente, los tests Maurer, Binary y Linear también los supera con, en promedio, $p \simeq 0,5$.

En conclusión, Kingston parece presentar un problema de decoherencia semejante a Cepheus pero, al tener más ficheros para hacer un estudio promediado, vemos que en conjunto la decoherencia parece afectar sólo a los tests de frecuencias. Es posible que pueda superarlos si se usaran sólo qbits con un error de decoherencia menor a una cota. En el momento del análisis se encontró que los qbits Qbits 121,146 y 73 presentaban errores de readout de 10^{-1} , frente a los errores de sus compañeros de hasta dos ordenes de magnitud menores. Además, los Qbits 120 y 131 presentan errores en la puerta $\sqrt{X}$ de $6 \cdot 10^{-2}$ y $3 \cdot 10^{-2}$. Esto nos indica que, a la hora de preparar los estados con la puerta Hadamard, dichos qbits no estarán en estados equiprobables[35]. Es muy probable que estos errores sean los culpables de fallar los tests de frecuencia, aunque en ausencia de ellos podría rendir como los PRNG.

Comenzando con los casos de PRNG, los ficheros del modelo PCG de Statevector Sampler presentan resultados muy positivos. State Vector supera todos los tests de la suite con p-values en torno a 0,5. Sin embargo, es destacable mencionar que en los peores ficheros los p-values pueden reducirse considerablemente. Esto se debe a que, en pos de simular aleatoriedad, recordemos que secuencias aparentemente no aleatorias forman parte del conjunto. De esta forma, no nos debe extrañar que existan secuencias límite generadas que fallen tests, en especial tests como el Longest One Block. Que el simulador haya funcionado tan bien aporta esperanza a QRNG, pues idealmente podrían rendir como el Statevector Sampler pero sin los problemas deterministas propios de PRNG.

Finalmente, discutamos los resultados del PRNG Criptográfico. BCrypt también supera todos los tests, y con valores más cercanos a $p = 0,5$ que el modelo PCG. Sin embargo, en los peores ficheros los fracasos son más abundantes y mayores que los de PCG. En el peor caso de Statevector para el test Maurer tenemos $p \simeq 0,015$, mientras que para BCrypt encontramos $p \simeq 0,0064$. Esta diferencia causa que uno supere el test en el caso más pesimista, mientras que el otro no. De esta forma, podemos concluir que BCrypt presenta en promedio el mejor resultado de todos los sistemas, aunque presente menos uniformidad entre ficheros.

3.2.4. Tests de Segundo Orden

Para el caso de Kingston, todos los tests NIST⁴⁹ superan el test de segundo orden. Es sorprendente el hecho de que el test Monobit ahora se supera con $p = 1$, lo que implica que, si bien Kingston por su cuenta falla el test, la diferencia entre el mismo y los ficheros de BCrypt son apenas diferentes. Es posible entonces que el test Monobit no sea del todo fiable, y que las propiedades de uniformidad de QRNG no sea tan mala como inicialmente pensamos.

Respecto al PCG64, encontramos que todos los tests NIST superan el test de segundo orden, si bien ningún p-value supera los 0,2. De todas formas supera el test sin problemas, por lo que podemos considerar que los resultados de la suite para Statevector son fiables.

3.2.5. Tasa de Generado y Costos

La tabla 8 recoge la tasa de generado obtenida para cada computador. La estimación preliminar de IBM era errónea: la tasa de generado de bits de Kingston está a la par con la de los algoritmos PCG. Esto abarata aún más los costes de IBM. La figura 7 compara todas las funciones costo de los QC. Se ve que IBM es la clara vencedora por órdenes de magnitud. Si bien habría que estimar el coste para PCG y BCryptGen, es muy probable que aún así que IBM no sea tan rentable como sus rivales PRNG.

Comp (ficheros)	Aquila(1)	Ibex (1)	Cepheus (1)	Kingston (96)	Statevector (118)	BCrypt (157)
Tasa (bits/s)	<i>Repetir</i> ± 0	<i>Repetir</i> ± 0	$(2,746947 \pm 0) \cdot 10^5$	$(5,309 \pm 0,012) \cdot 10^7$	$(8,803 \pm 0,021) \cdot 10^7$	$(3,482 \pm 0,009) \cdot 10^{16}$

Tabla 8: Tasas de Gerado de los diferentes modelos. El error corresponde al error estadístico de la media, y se usa para las cifras significativas.

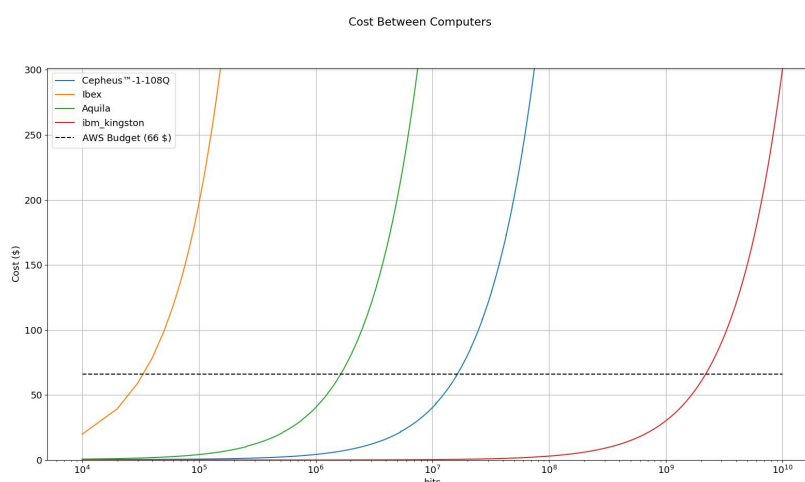


Figura 7: Comparación de las funciones costo

⁴⁹Los resultados de los tests de segundo orden a Kingston y Statevector se encuentran en el Anexo.

4. Conclusión

Las secuencias de QRNG de superconductores se encuentran en el límite de cumplir los estándares de aleatoriedad de sus rivales PRNG. Si bien presentan valores adecuados de entropía, indicando una buena impredecibilidad, los tests de frecuencias y de sumas parciales indican un rechazo sistemático de la propiedad de uniformidad. La razón de este rechazo es muy probable que se deba a la aparición de ruido periódico en las secuencias, fruto de los errores por decoherencia y lectura de los qbits. Sin embargo, estas variaciones deben de ser muy pequeñas, pues superan otros tests que también evalúan dicha propiedad (como los tests de Segundo Orden). Los QRNG evaluados presentan excelentes propiedades de independencia lineal, superando los tests de Linear Complexity y Binary Matrix. El test propio de Complejidad de Kolmogorov es inválido, pues tanto las secuencias PRNG como QRNG han fracasado con p-values de 0. Hemos descartado error numérico proveniente de la aproximación numérica de las distribuciones en la suite gracias a la cantidad elevada de bits usados para IBM, PCG64 y Crypt y los tests de segundo orden

La tabla 9 evalúa por orden decreciente de calidad los distintos RNG para cada propiedad de aleatoriedad considerada.

Pilar de aleatoriedad	1	2	3	4	5	6
Impredecibilidad	PCG64	BCrypt	Kingston	Cepheus	Ibex	Aquila
Uniformidad	PCG64	BCrypt	Kingston	Cepheus	Ibex	Aquila
Independencia	Kingston	BCrypt	PCG64	Cepheus	Ibex	Aquila

Tabla 9: Ordenamiento en calidad decreciente de los RNG utilizados

De esta forma, PRNG se posiciona en mejores condiciones para dos de las tres propiedades fundamentales de la aleatoriedad. Sin embargo, el margen de diferencia entre ambos es leve, principalmente los tests de Frecuencias de la suite. Es probable que con la disminución de decoherencia (ya mejorando la tecnología, o usando menos qbits) también cumplieran esta propiedad, como ha demostrado la variación de H_{min} obtenida para Kingston al usarse diferentes qbits. En impredecibilidad se posicionan prácticamente al mismo nivel de calidad.

El computador de IBM se ha mostrado superior a los de AWS, tanto a nivel de resultados como de tasa de generado y costos. AWS impone limitaciones al número de shots por task, presenta precios más elevados y la extracción de metadata de los tasks es más obtusa. Esto dificulta la obtención de secuencias, así como su analizado. Dentro de AWS *Cepheus* presenta la mejor calidad de QRNG, lo que nos indica que posiblemente los computadores cuánticos de superconductores presenten la tecnología mejor adaptada para RNG. Creemos que los computadores de iones atrapados como *Ibex* presentarían calidad de RNG similar a los superconductores. Sin embargo, el alto coste de su tecnología lo coloca en un escalón inferior. Por último, *Aquila* se debe descartar para generar QRNG, pues el efecto de Rydberg Blockade genera correlaciones entre bits inevitables.

Respondiendo a la pregunta inicial, actualmente es viable la generación de QRNG a

partir de computadores cuánticos superconductores. Para ello, sin embargo, sería necesario tener un control constante de los errores de decoherencia en los qbits usados, así como vigilar el costo para computadores basados en coste por shots, como los de AWS. Se debería buscar un equilibrio entre un número reducido de qbits, para controlar mejor los errores, y el aumento de costo que implicaría el mayor número de shots implicado.

Encontramos dos claras vías de mejora de la investigación. Por un lado podemos mejorar la librería de código creada. Por otro, podemos ampliar el análisis de los computadores.

Respecto a la mejora de la librería, sería interesante optimizar el código de la Suite NIST. Para el análisis de los 200 ficheros de la sección 3 se tardó en torno a 48 horas de ejecución continuada. Se podría reescribir la librería de Kho para adaptarla a Numpy, como hicimos con el test de *Binary Matrix*, o incluso aplicar Numba en diversas secciones de la suite para agilizar los procesos mediante programación paralela. Además de la librería NIST, convendría refactorizar el código de *Analysis_Data* para mayor redibilidad y eficiencia⁵⁰. También sería interesante revisar el algoritmo de cálculo de costos y obtención de bits por ficheros, pues hay leves variaciones en los bits pedidos y los finalmente obtenidos para AWS. Por último, convendría revisar el test de Complejidad de Kolmogorov y tratar de discernir si su fracaso se debe al, programación, o si efectivamente sus resultados son significativos.

Respecto a la vía de ampliar el análisis, primero de todo se debería comparar los resultados de IBM a un PCG64 con ruido similar al del computador cuántico. Podría usarse la librería *Aer* de qiskit, especializada en simular computadores cuánticos con ruido realista [68]. Si el PRNG rinde de forma similar al QRNG en presencia de ruido, los QRNG presentarían un prospecto muy optimista pues de minimizar el ruido presentarían las ventajas de los PRNG actuales sin su determinismo. También sería conveniente repetir el análisis para Ibex y Cepheus con los 100 ficheros de 10^7 bits, en pos de verificar si la hipótesis de equidad en calidad entre los QRNG es cierta. De serlo, recomendaríamos buscar un computador de atrapamiento de iones más económico que Ibex para futuros estudios. Sería útil probar a analizar 100 ficheros de 10^7 bits de Kingston usando una cantidad muy reducida de qbits. De esta forma se podría verificar hasta qué punto los errores individuales de los qbits están afectando a los resultados de uniformidad en las secuencias. Finalmente, recomendaríamos aplicar este análisis a los dispositivos especializados en QRNG más estudiados en la actualidad [53][46], y comprobar hasta qué punto difiere su calidad de los PRNG y los computadores cuánticos utilizados.

Existen propuestas mixtas. Sería interesante aplicar *Randomness Extraction* a las secuencias de IBM para tratar de maximizar sus resultados. Como comenta Matarazzo [75] se podrían suprimir los errores de decoherencia con técnicas de extracción de aleatoriedad. Otra propuesta sería incluir otros tests de la observación 2.26. Podrían implementarse tests para números de coma flotante. Para ello habría que escribir un script que traduzca los ficheros de binario a floats. Nos resultan atractivos los tests basados en procesos estocásticos de Física, o de superficies aleatorias.

⁵⁰Especialmente el test de Segundo Orden, pues se recalculan los p-values de las secuencias problema cuando ya se conocen en ficheros.

5. Anexos

Ampliación de Teoría de la Información

Teorema 5.1 (Teorema de Bayes).

Sean A y B dos eventos no excluyentes entre sí. Las probabilidades condicionadas entre un evento a_i y uno b_j dados están relacionadas como:

$$P(b_j|a_i) = \frac{P(b_j)}{P(a_i)} P(a_i|b_j) \quad \forall i, j \in \mathbb{N} \quad (66)$$

Observación 5.2 (¿Por qué la información se describe con la expresión 5?).

Existe una definición más formal planteada por Shannon [10]. Sin embargo, nosotros sólo vamos a dar algunos detalles intuitivos sobre la forma de la información:

1. Si $P(a_i) = 1$, conocer que A se determina en a_i no nos da información extra, ya que la probabilidad condicionada pasa a ser:

$$P(b_j|a_i) = P(b_j, a_i) = P(b_j) \quad (67)$$

Los logaritmos presentan la propiedad $\log(1) = 0$, por lo que en un primer acercamiento parecen adecuados.

2. A menor $P(a_i)$ más información obtenemos del sistema porque más influye en las probabilidades condicionadas, hasta el caso límite $P(a_i) = 0$. $h(x) = -\log(x)$ es una función monótona decreciente en el intervalo $(0, 1]$ con una asíntota a infinito para $x \rightarrow 0$.
3. En el caso de que A y B sean eventos independientes, conocer ambos nos debería dar una adición de información de tal forma que:

$$h[P(b_j, a_i)] = h[P(b_j)P(a_i)] \doteq h[P(b_j)] + h[P(a_i)] \quad (68)$$

Lo que resulta ser una propiedad de los logaritmos.

A continuación vamos a describir cómo se interrelaciona la información entre distintos sistemas, algo fundamental para describir cómo en los QRNG tenemos distintas fuentes de aleatoriedad.

Definición 5.3 (Información entre varios sistemas // Información múltiple).

Sean A y B dos eventos. Denotamos $H(A, B)$ como la información asociada a conocer A y B :

$$H(A, B) = - \sum_{i,j \in \mathbb{N}} P(a_i, b_j) \log(P(a_i, b_j)) \quad (69)$$

Si tuviéramos N eventos $\{A^n\}_{n \in \mathbb{N}}$:

$$H(A^1, A^2, \dots, A^N) = \sum_{i,j,k,\dots,s \in \mathbb{N}} P(a_i^{(1)}, a_j^{(2)}, a_k^{(3)}, \dots, a_s^{(N)}) \log(P(a_i^{(1)}, a_j^{(2)}, a_k^{(3)}, \dots, a_s^{(N)})) \quad (70)$$

Teorema 5.4 (Otras propiedades de la entropía). Sea $H(A)$ la entropía de un evento A con sucesos $\{a_i\}_{i \in \mathbb{N}}$. Se cumple que:

$H(A)[p]$ es una función continua y cóncava para las distintas probabilidades. A nivel utilitario implica que su segunda derivada siempre es negativa o nula.

La entropía será nula sí y solo sí existe un suceso con probabilidad unidad. (Es decir, no nos aporta información aseverar certezas)

(Expresión alternativa con probabilidades duales) Dados dos eventos A y B , podemos escribir la entropía asociada a cada evento a partir de las probabilidades duales entre sucesos gracias a la propiedad 3:

$$\begin{aligned} H(A) &= - \sum_{i,j \in \mathbb{N}} P(a_i, b_j) \log \left(\sum_{k \in \mathbb{N}} P(a_i, b_k) \right) \\ H(B) &= - \sum_{i,j \in \mathbb{N}} P(a_i, b_j) \log \left(\sum_{k \in \mathbb{N}} P(a_k, b_j) \right) \end{aligned} \quad (71)$$

Teorema 5.5 (Propiedades de la información múltiple).

Supóngase sin pérdida de generalidad que tenemos dos eventos A y B con entropía múltiple $H(A, B)$. Se cumple que:

1. $H(A, B) \geq \max [H(A), H(B)]$
2. $H(A) + H(B) \geq H(A, B)$
3. $H(A, B) = H(A) + H(B) \iff P(b_j, a_i) = P(b_j)P(a_i) \quad \forall i, j \in \mathbb{N}$

Definición 5.6 (Información asociada a B conociendo A).

Sean dos eventos A y B . Definimos la *información asociada a B conociendo que en A ocurre a_0* como:

$$H(B|a_0) = - \sum_{j \in \mathbb{N}} P(b_j|a_0) \log(P(b_j|a_0)) \quad (72)$$

De tal forma, podemos definir también la *información asociada a B conociendo A* como:

$$H(B|A) = - \sum_{i \in \mathbb{N}} P(a_i) H(B|a_i) = H(A, B) - H(A) \quad (73)$$

O, visto de otra forma:

$$H(A, B) = H(A) + H(B|A) \quad (74)$$

Es decir, la información obtenida al conocer A y B es la información obtenida de A más la que podemos obtener de B habiendo conocido ya A . Es equivalente a:

$$H(A, B) = H(B) + H(A|B) \quad (75)$$

Observación 5.7. Nótese que la expresión 74 se asemeja mucho a la expresión del Teorema de Bayes. En parte es un análogo de la misma para la entropía, donde se han aplicado las propiedades de los logaritmos.

Demostraciones de Teoría de la Información

Demostración. Relación entre la probabilidad condicionada y la dual (Teorema 2.2)

Este teorema se puede deducir desde la teoría de la medida a partir de σ -álgebras, como

muestra Kolgomorov[47]. Pero algunos probabilistas como de Finetti lo tratan como un axioma [21]. En nuestro caso vamos a justificar la relación entre las variables como explicó Barnett [10] sin una demostración rigurosa.

Queremos determinar una relación entre las probabilidades duales y las condicionadas entre dos sucesos. Supongamos que queremos, sin pérdida de generalidad, determinar $P(b_j|a_0)$. Esta situación corresponde a una frecuencia de cuántas veces ocurre b_j habiéndose dado a_0 . Así, es coherente esperar que esté relacionada con la probabilidad dual a partir de una constante de proporcionalidad $K(a_0)$:

$$P(b_j|a_0) = K(a_0)P(b_j, a_0) \quad (76)$$

Pero al trabajar con probabilidades, éstas deben estar normalizadas de acuerdo a 2. Si entonces sumamos en todo j , la probabilidad izquierda será la unidad (pues es la probabilidad de que B se determine en un suceso cualquiera). El lado derecho, por la propiedad 3, pasa a ser $P(a_0)$. De esta forma:

$$K(a_0) = P(a_0) \quad (77)$$

Recuperando la expresión del teorema. □

Demostración. Teorema de Bayes 5.1

Tomemos la expresión del teorema 2.2. Si aplicamos la simetría de $P(b_j, a_i)$ podemos igualar ambas expresiones condicionadas:

$$P(b_j|a_i)P(a_i) = P(a_i|b_j)P(b_j) \quad (78)$$

□

Demostración. Propiedades de la información múltiple (teorema 5.5)

• Propiedad 1: Tomemos la diferencia:

$$\begin{aligned} H(A, B) - H(A) &= - \sum_{i,j \in \mathbb{N}} P(a_i, b_j) \log(P(a_i, b_j)) + \sum_{k \in \mathbb{N}} P(a_k) \log(P(a_k)) \\ &= - \left[\sum_{i,j \in \mathbb{N}} P(a_i, b_j) \log(P(a_i, b_j)) - \sum_{k \in \mathbb{N}} P(a_k) \log(P(a_k)) \right] \end{aligned} \quad (79)$$

Usando la ecuación 71:

$$\begin{aligned} H(A, B) - H(A) &= - \left[\sum_{i,j \in \mathbb{N}} P(a_i, b_j) \log(P(a_i, b_j)) - \sum_{k,l \in \mathbb{N}} P(a_k, b_l) \log \left(\sum_{m \in \mathbb{N}} P(a_k, b_m) \right) \right] \\ &= - \sum_{i,j \in \mathbb{N}} P(a_i, b_j) \left[\log(P(a_i, b_j)) - \log \left(\sum_{m \in \mathbb{N}} P(a_i, b_m) \right) \right] \end{aligned} \quad (80)$$

Pero $\sum_{m \in \mathbb{N}} P(a_i, b_m) = 1$ y $P(b_j|a_i) = \frac{P(a_i, b_j)}{P(a_i)}$:

$$H(A, B) - H(A) = - \sum_{i,j \in \mathbb{N}} P(a_i)P(b_j|a_i) \log(P(b_j|a_i)) \quad (81)$$

Y como $P(b_j|a_i) \leq 1 \quad \forall i, j$ (Es una probabilidad a fin de cuentas):

$$H(A, B) - H(A) \geq 0 \implies H(A, B) \geq H(A) \quad (82)$$

Podemos realizar el mismo razonamiento para $H(A, B) - H(B)$, de tal forma que se tiene que cumplir:

$$\begin{aligned} H(A, B) &\geq H(A) \wedge H(A, B) \geq H(B) \\ \implies H(A, B) &\geq \max(H(A), H(B)) \quad \square \end{aligned} \quad (83)$$

• Propiedad 2: Hagamos el mismo desarrollo que en la propiedad 1, pero ahora con:

$$H(A) + H(B) - H(A, B) = \sum_{i,j \in \mathbb{N}} P(a_i, b_j) \log \left(\frac{P(a_i, b_j)}{P(a_i)P(b_j)} \right) \quad (84)$$

Se puede comprobar mediante el Método de los Multiplicadores de Lagrange que esta expresión es mayor o igual a cero siempre. Para un desarrollo completo, véase [10]. $\square$

• Propiedad 3: Supongamos sin pérdida de generalidad que $H(A) \geq H(B)$. Impongamos que los sucesos sean independientes. De cara a las probabilidades condicionadas esto significa que:

$$P(b_j|a_i) = \frac{P(b_j)P(a_i)}{P(a_i)} = P(b_j) \quad (85)$$

Lo cual es esperable, pues implica que conocer el resultado de A no nos influye en el resultado de B . Sustituyendo en 81:

$$H(A, B) - H(A) = \sum_{i,j \in \mathbb{N}} P(a_i)P(b_j) \log(P(b_j)) = \sum_{i \in \mathbb{N}} P(a_i) \cdot \sum_{j \in \mathbb{N}} P(b_j) \log(P(b_j)) \quad (86)$$

Pero $\sum_{i \in \mathbb{N}} a_i = 1$, y $\sum_{j \in \mathbb{N}} P(b_j) \log(P_j) = H(B)$. Así:

$$H(A, B) = H(A) + H(B) \quad (87)$$

Quedan así demostradas todas las propiedades. $\square$

Descripción y análisis de los test NIST

A continuación realizamos un compendio de los tests a partir de la cualidad de aleatoriedad que buscan revisar. Incluimos también las distribuciones numéricas que usan

1. **Tests de frecuencias en la cadena:** Buscan comprobar cómo se reparten los bits en la secuencia. Idealmente se espera una proporción igual de 1s y 0s, con una distribución uniforme en la cadena. Evalúan las propiedades de uniformidad e impredecibilidad.
 - a) Monobit Test: Evalúa la proporción de 1s y 0s en toda la secuencia, y calcula su diferencia con la esperada de $1/2$. p-values bajos implicarán rechazo del test. P-values bajos corresponden a una asimetría considerable en la cantidad de 0s y 1s. Basado en la distribución χ^2 y erfc.
 - b) Frequency Test within a Block: Evalúa la proporción de 1s en bloques de la secuencia. Si cada bloque tiene tamaño M , se esperan frecuencias de 1s de $M/2$. Valores bajos de p-values expresan desviaciones grandes de una proporción uniforme de 0s y 1s en, al menos, uno de los bloques. Basado en distribución χ^2 .
 - c) Runs Test: Evalúa la cantidad y la longitud de subcadenas de un mismo bit dentro de la secuencia. Valores bajos de p-values hubieran indicado o bien oscilaciones muy rápidas de bits, o muy lentas. Basado en χ^2 .
 - d) Test for the Longest Run of Ones in a Block: Evalúa la longitud de subcadenas de bit 1 máxima encontrada para subdivisiones de la secuencia en bloques, y se compara si corresponde a la esperable de una secuencia aleatoria. P-Values bajos indican agrupamientos de bits 1. Basado en χ^2 .
 - e) Test Espectral: Se evalúa la secuencia en el Espacio de Fourier con una Transformada Discreta de Fourier (DFT), comprobando si hay frecuencias características en forma de valores altos de las coordenadas que describen la secuencia con series de Fourier. P-values bajos presentan picos mayores al tamaño dado por el intervalo de confianza escogido. Usa la distribución normal, y erfc
 - f) Serial Test: Comprobamos la frecuencia de todas las subcadenas de longitud m posibles en toda la cadena, comparándolo con su frecuencia esperada en una secuencia aleatoria. De esta forma, estudiamos la uniformidad de la distribución que modeliza la cadena. Es el caso general del test Monobit.
2. **Tests de Mecánica Estadística:** Estos tests tienen en común que las herramientas teóricas usadas se encuentran en Mecánica Estadística. Para el trasfondo físico del TFG hemos considerado adecuado separarlos. Evalúan impredecibilidad e independencia
 - a) Test de Random Walk (Cusum): Se construye un Random Walk unidimensional con los bits, de tal forma que el bit 1 corresponde un paso a la derecha (+1) y el bit 0 a la izquierda (-1). Para secuencias aleatorias se esperan valores de posición final cercanas al origen, de acuerdo a una binomial con media $\bar{x} = 0$. Se usa la distribución normal estándar Φ .
 - b) Approximate Entropy Test: Calculamos la entropía asociada a todos los eventos correspondientes a subsecuencias de longitud m . Se comprueba si la entropía

obtenida es uniforme en todos los bloques (es decir, que la información está distribuida de forma uniforme. Se puede relacionar también con la complejidad). P-values bajos indican irregularidades severas, y por ende habría bloques con mucha más información que otros y, por ende, posibles secciones compresibles. Usa χ^2 .

- c) Random Excursions: Se cuenta el número de veces que se revisita un estado en el Random Walk. Se evalúa para las posiciones discretas en el intervalo $[-4, 4]$. P-values bajos indican una variación importante respecto a la distribución normal para todos los estados centrados en cada posición. Se usa χ^2 .
- d) Random Excursions Variant Test: Corresponde al test de Random Excursions, pero para el intervalo $[-9, 9]$. Usa χ^2 .

3. **Tests de Complejidad de Kolmogorov:** Al igual que con los de Mecánica Estadística, la propiedad en común de estos tests es el fundamento teórico. En este caso, Teoría de Información Algorítmica. Se estudia la capacidad de compresión de la secuencia con las técnicas más usadas, y las herramientas matemáticas más características. Evalúan uniformidad e independencia.

- a) Maurer's "Universal Statistical" Test: Computamos la cantidad de bits entre patrones de bits compresibles. De esta forma, podemos estimar el valor de la función de deficiencia y asociarle un p-value. P-values bajos indican valores de la función de deficiencia altos y, por ende, una complejidad baja.
- b) Binary Matrix: Se estudia el rango de submatrices disjuntas construidas con bits de la secuencia. De esta forma se puede comprobar si hay dependencia lineal entre subcadenas de la secuencia. P-values bajos indican rangos menores a los esperados (i.e., dependencia lineal). De nuevo, se usa χ^2 .
- c) Linear Complexity Test: Se estudia cuál es la longitud mínima que debe tener un LFSR ⁵¹ para poder construir la secuencia dada. Es decir, la complejidad de la secuencia para la familia de algoritmos de descompresión basada en retroalimentación lineal. P-values bajos indican longitudes de LFSR bajas. Usa χ^2 .
- d) Non-overlapping Template Matching Test: Comprueba el número de coincidencias de subcadenas escogidas previamente. P-value bajos indican muchas ocurrencias de las subcadenas en la secuencia. Usa χ^2 y la función Gamma Incompleta.
- e) Overlapping Template Matching Test: Mismo objetivo que el Non-overlapping. La diferencia es técnica: Cuando se da una correspondencia, se busca la siguiente a partir del siguiente bit. En Non-overlapping cuando había una correspondencia se buscaba después de avanzar la longitud de la cadena encontrada. Usa χ^2 y la función Gamma Incompleta.

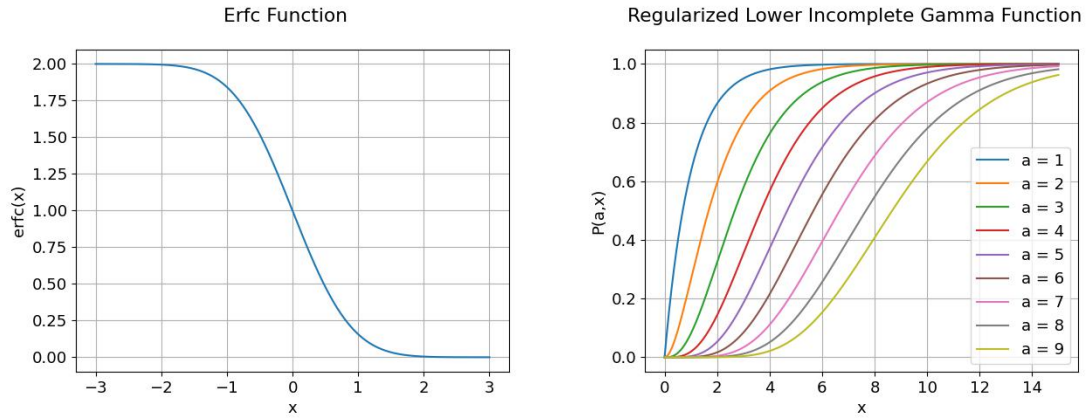
⁵¹Un LFSR es un *Registro de desplazamiento con retroalimentación lineal*, una técnica de PRNG muy común que se basa en relaciones recursivas entre bits para ampliar la longitud de la secuencia. En concreto, realiza una traslación de todos los bits[81].

Prueba χ^2 de Pearson

Sean $\{x_1, x_2, \dots, x_n\}$ un conjunto de secuencias con distribución teórica asociada F. Sean $\{x_1^t, x_2^t, \dots, x_n^t\}$ un conjunto de secuencias obtenidas a partir de la distribución teórica. Definimos el parámetro χ^2 o *bondad del ajuste* como:

$$X^2 = \sum_{i=1}^n \frac{(x_i - x_i^t)^2}{x_i^t} \quad (88)$$

Las distribuciones usadas en los tests NIST se encuentran en la figura 8.



Distribución *erfc*
 $\text{erfc}(z) = 1 - \frac{2}{\sqrt{\pi}} \int_z^\infty e^{-u^2} du$

(Regularized Lower) Función Gamma Incompleta:

$$P(a, x) \equiv \frac{\gamma(a, x)}{\Gamma(a)} = \frac{1}{\Gamma(a)} \int_0^x t^{a-1} e^{-t} dt$$

Con $\Gamma(a) = \int_0^\infty t^{a-1} e^{-t} dt$

Figura 8: Distribuciones usadas en la librería NIST

Mecánica Cuántica bajo el Formalismo de Dirac

A continuación realizamos un breve recordatorio del Formalismo de Dirac. De esta forma podremos describir los computadores cuánticos como un conjunto de transformaciones en el Espacio de Hilbert para un estado inicial dado. Una introducción se encuentra en el curso del profesor Susskind [82], así como en las excelentísimas clases manuscritas de Feynman [71].

En el Formalismo de Dirac los sistemas físicos se modelizan a partir de un Espacio de Hilbert con cuerpo en $\mathbb{C}^n$, denominado H , y distinto según el sistema⁵². Los vectores de estos espacios se denominan *kets*, y corresponden a trayectorias en el espacio de fases del sistema para unas condiciones dadas. Se denomina un posible *estado* del sistema, y los denotaremos con letras griegas de la forma $|\Psi\rangle$. La métrica del espacio, por otro lado, está dada por un producto escalar antihermítico de la forma:

$$||\Psi\rangle|^2 \equiv \langle\Psi|\Psi\rangle = |\Psi\rangle \cdot |\Psi\rangle = \int_{\mathbb{R}^n \subset \mathbb{C}^m} \Psi(\vec{x})^* \Psi(\vec{x}) dx^n \quad (89)$$

Donde $\Psi(x)$ corresponde a la función de onda asociada al ket. El producto escalar definido como $\langle\Psi|\Phi\rangle$ da pie a definir los elementos del dual, denominados *bra*:

$$\begin{aligned} \langle\Psi| : H &\longrightarrow \mathbb{C}^n \\ \langle\Psi| \quad [|\Phi\rangle] &\longrightarrow |\Psi\rangle \cdot |\Phi\rangle \equiv \langle\Psi|\Phi\rangle \end{aligned} \quad (90)$$

Es decir, el bra asociado al ket $|\Psi\rangle$ se denomina $\langle\Psi|$, y es aquella 1-forma que, al evaluar a $|\Psi\rangle$, devuelve su norma asociada⁵³. De esta forma la asignación de *bras* es única, y nos permite trabajar el producto escalar del espacio como la aplicación de *brackets*. Por la antihermiticidad del producto escalar, podemos entender el bra de un ket como el conjugado de sus coordenadas con la base de bra asociada a los kets de la base. Sea $V = \{|\phi_i\rangle\}_{i \in \mathbb{N}}$ una base ortonormal de H . Entonces la base de bra de H^* asociada es $V^* = \{\langle\phi_i|\}_{i \in \mathbb{N}}$ tal que:

$$\langle\phi_i| \quad [|\phi_j\rangle] = \delta_i^j \quad (91)$$

De tal forma que, para cualquier $|\Psi\rangle \in H$ se cumple:

$$|\Psi\rangle = \sum_{i \in \mathbb{N}} c_i |\phi_i\rangle \quad \langle\Psi| = \sum_{i \in \mathbb{N}} c_i^* \langle\phi_i| \quad (92)$$

Con las coordenadas conforman un vector de $\mathbb{C}^n$ con las componentes definidas como en un espacio de Hilbert típico:

$$c_i = \int_{R[n]} \Psi(\vec{x}) \phi_i(\vec{x}) dx^n \quad (93)$$

⁵²Un Espacio de Hilbert es una generalización de un espacio euclídeo de dimensión infinita. En él se define una métrica y se pide que sea *completa*. Es decir, que cualquier sucesión de Cauchy de elementos del espacio debe converger a un elemento del espacio. Una comprensión más profunda de estas ideas recae en el terreno del Análisis Funcional, y se escapa de los propósitos de este trabajo. Para un desarrollo introductorio recomendamos el libro de Cascales et. al. [13]

⁵³Como nota de completitud, la construcción de los bra de esta forma es la misma que la que se usa para construir el concepto de 1-formas para Relatividad General.

Las coordenadas tienen una interpretación probabilística: la norma de un ket representa la probabilidad de que el estado estudiado corresponda al del ket. De esta forma existe la imposición de que los estados con sentido físico sean los de norma $||\Psi\rangle| = 1$, imponiendo una ligadura a las coordenadas:

$$\sum_{i \in \mathbb{N}} c_i \cdot c_i^* = 1 \quad (94)$$

Los estados se modifican a través de operadores lineales y hermíticos definidos como A . Éstos presentan un conjunto de autoestados en los que la medida del sistema viene asociada con un autovalor de la magnitud completamente certero. Normalmente se usan bases de autoestados de un observable dado.

Los operadores se pueden escribir en la base de autoestados como:

$$\hat{A} = \sum_{i,j \in \mathbb{N}} A_{ij} |\phi\rangle_i \langle\phi_j| \quad (95)$$

Donde A_{ij} son los elementos de matriz asociados al operador, definidos como:

$$A_{ij} = \langle\phi_i | \hat{A} | \phi_j \rangle \quad (96)$$

Finalmente, dados n sistemas con bases $V^\alpha = \{|\phi\rangle_i^\alpha\}_{i \in \mathbb{N}}$, podemos describir el sistema compuesto por todos ellos usando como base aquella que incluye todas las combinaciones distintas entre los elementos de las bases. Esto es lo que se conoce como producto tensorial, de tal forma que podemos escribir la base total V como:

$$V^{\otimes n} = \sum_{\alpha, \beta | \alpha < \beta}^n V^\alpha \otimes V^\beta \quad (97)$$

Si $|\phi\rangle \in V_1$ y $|\psi\rangle \in V_2$, el producto $|\phi\rangle \otimes |\psi\rangle$ se suele expresar como $|\phi\psi\rangle$, si el orden del producto tensorial de los espacios ha sido $H^1 \otimes H^2$ (si hubiera sido al revés sería $|\psi\phi\rangle$).

5.1. Algunos tiempos de Decoherencia

Mecanismo	$\tau_{Dec}(s)$	$\tau_{Op}(s)$	$n_{Op} = \frac{\tau_{Dec}}{\tau_{Op}}$
Espín Nuclear	$10^{-2} \sim 10^{-8}$	$10^{-3} \sim 10^{-6}$	$10^5 \sim 10^{14}$
Espín electrónico	10^{-3}	10^{-7}	10^4
Trampa Iones (In^+)	10^{-1}	10^{-14}	10^{13}
Quantum Dot	10^{-6}	10^{-9}	10^3
Superconductores	10^{-4}	10^{-8}	10^4

Tabla 10: Estimación del número de operaciones máximo para cada tipo de ordenador [56] [20].

Como se ve en la tabla 10 el número de operaciones varía en algunos ordendes de magnitud según el mecanismo.

Restriction	Value	Reason
Maximum total number of filled and unfilled user-defined sites	256	Limited laser power for tweezers that generate the user-defined and reservoir sites.
Maximum number of qubits, i.e. maximum number of filled sites	256	Finite filling rates when loading the atom array and limited laser power for laser tweezers.
Maximum site pattern width Maximum site pattern height	75 μm 76 μm	Position-dependent tweezer power efficiency due to the holography method used.
Minimum distance between user-defined sites	4 μm	Optical resolution of the imaging system and of the holography method used.
Minimum vertical spacing between rows	4 μm	Packing geometry of the reservoir sites.
Maximum Rydberg Rabi frequency	15.8 $\frac{rad}{\mu s}$	Limited delivered laser power for driving the ground-Rydberg transition.
Rabi slew rate	$\left \frac{d\Omega}{dt} \right \leq 250 \frac{rad}{\mu s^2}$	Response speed of the acousto-optic device responsible for modulating the Rydberg drive laser.
Detuning range	$ \Delta \leq 125 \frac{rad}{\mu s}$	Electronic bandwidth of the acousto-optic device responsible for modulating the Rydberg drive laser.
Maximum duration of user-defined evolution	4 μs	Approximate timescale of coherent evolution.

Figura 9: Especificaciones de Aquila [40]

Especificaciones del Computador Aquila

Al ser un computador distinto al resto, enunciamos sus fuentes de error y especificaciones en las siguientes figuras.

Name	Value	Explanation
State preparation, measurement, and Hamiltonian errors		
δ_x, δ_y	0.050 μm	Systematic, pattern-dependent error between specified and actual lattice site positions.
σ_x, σ_y	0.200 μm	Random error in the qubit positions during coherent evolution as a result of thermal atom motion.
$\langle \delta \Omega^2 \rangle^{1/2} / \langle \Omega \rangle$	0.02	RMS relative Rabi frequency inhomogeneity over the user region.
$\langle \delta \Delta^2 \rangle^{1/2}$	0.37 $\frac{rad}{\mu s}$	RMS detuning inhomogeneity over the user region.
δ_Δ	0.63 $\frac{rad}{\mu s}$	Systematic error in global detuning from specified value.
$\langle \delta \Delta^2 \rangle^{1/2}$	0.18 $\frac{rad}{\mu s}$	RMS shot-to-shot variance in the detuning.
$\langle \delta \Omega^2 \rangle^{1/2} / \langle \Omega \rangle$	0.008	RMS relative shot-to-shot variance in the Rabi frequency.
ϵ_{fill}	0.007	Typical probability of failing to occupy a site specified by user as 'filled'. These probabilities are dependent on the pattern and site position within the pattern.
ϵ_{det-fn}	0.01	Probability of false-negative atom detection error (mis-detecting a filled site as empty).
ϵ_{det-fp}	0.01	Probability of false-positive atom detection error (mis-detecting an empty site as filled).
$\epsilon_{det-gnd}$	0.01	Probability of mis-detecting a ground-state atom as a Rydberg
$\epsilon_{det-ryd}$	0.08	Probability of mis-detecting a Rydberg atom as a ground-state atom
Ground-Rydberg qubit coherence		
T_2^*	5.8 μs	Qubit dephasing time without drive from individual non-interacting qubits, including coherent and incoherent processes, as measured by a Ramsey protocol (see example 1).
T_2^{echo}	11.4 μs	Qubit dephasing time without drive from individual non-interacting qubits from incoherent processes only, as measured by a spin-echo dynamical decoupling protocol (see example 1).
T_2^{Rabi}	7.5 μs	Driven qubit decoherence under maximum Rabi frequency from individual non-interacting qubits, including coherent and incoherent processes, as measured by Rabi oscillations (see example 1).
$T_2^{blockaded-Rabi}$	8.9 μs	Driven qubit decoherence under maximum Rabi frequency for an isolated pair of mutually blockaded qubits, including coherent and incoherent processes as measured by Rabi oscillations (see example 2).
Application benchmark metrics		
1D correlation length	3.6 sites	The correlation length of an adiabatically prepared Z_2 state (see example 3)
2D correlation length	5.7 sites	The correlation length of an adiabatically prepared checkerboard state (see example 3)

Figura 10: Errores específicos de Aquila [40]

Funciones Costo de los Computadores de AWS

Computador	c_t	c_s	l_s	l_i	n_{qBits}
Cepheus TM -1-108Q	0.3	$4,25 \cdot 10^{-4}$	$5 \cdot 10^4$	10	107
IBEX Q1	0.3	0.0235	2000	1	12
Aquila	0.3	0.01	1000	1	256

Tabla 11: *Propiedades de la función costo de cada computador. Los costos están en dólares.*

Códigos usados en la investigación

Se encuentran subidos en la plataforma *Github.com* para uso público en el siguiente repositorio:

https://github.com/Lucoero/QRNG_With_QC/tree/main

Resultados para cada computador

Los resultados, así como las secuencias en bruto, se pueden consultar en el repositorio mencionado en la sección de códigos:

https://github.com/Lucoero/QRNG_With_QC/tree/main

A continuación adjuntamos los ficheros resumen de cada computador, así como los resultados del test de Segundo Orden (aunque se visualizan mejor en el txt del repositorio):

Listing 1: Aquila

```
Aquila Data Summary: Used 1 files
—> Entropy Estimations:
—> H_min = 0.14470568651884771+/-0 bits
—> Worst H_min = 0.14470568651884771 bits
—> H = 0.45435914196283234+/-0 bits
—> Worst H = 0.45435914196283234 bits
—> Complexity Estimations:
—> Maximum Complexity = 61741.0+/-0 bits
—> Worst Complexity = 61741.0 bits
—> Deficiency Function = 938195.0+/-0
—> Worst Deficiency Function = 938195.0
—> p-value = 0.0+/-0 — False
—> Worst p-value = 0.0 — False
—> NIST Battery Test:
—> FREQUENCY IN SEQUENCES
  (o) Monobit_Test: 0.0+/-0 — False
      Worst p-value: 0.0 — False
  (o) Within a Block: 0.0+/-0 — False
      Worst p-value: 0.0 — False
  (o) Run Test: 0.0+/-0 — False
      Worst p-value: 0.0 — False
  (o) Longest One Block: (7.579099461862028+/-0)e-218 — False
      Worst p-value: 7.579099461862028e-218 — False
SPECTRAL TESTS
  (o) Spectral Test: 0.0+/-0 — False
      Worst p-value: 0.0 — False
STATISTICAL MECHANICS TESTS
```

- (o) Approximate Entropy Test: $0.0+/-0$ — False
Worst p-value: 0.0 — False
- (o) Cusum Test: $0.0+/-0$ — False
Worst p-value: 0.0 — False
- (o) Random Excursions Test:
 - 4: $(3.6679032349766+/-0)e-14$ — False
Worst p-value: $3.6679032349766e-14$ — False
 - 3: $(9.326695044407568+/-0)e-10$ — False
Worst p-value: $9.326695044407569e-10$ — False
 - 2: $0.0010819122096153867+/-0$ — False
Worst p-value: 0.0010819122096153867 — False
 - 1: $0.6999858358786276+/-0$ — True
Worst p-value: 0.6999858358786276 — True
 - +1: $0.9625657732472964+/-0$ — True
Worst p-value: 0.9625657732472964 — True
 - +2: $0.9969687632568645+/-0$ — True
Worst p-value: 0.9969687632568645 — True
 - +3: $0.9991138612111875+/-0$ — True
Worst p-value: 0.9991138612111875 — True
 - +4: $0.9996100613790039+/-0$ — True
Worst p-value: 0.9996100613790039 — True
- (o) Random Excursions Variant Test:
 - 9: $0.8638317428547264+/-0$ — True
Worst p-value: 0.8638317428547264 — True
 - 8: $0.7150006546880893+/-0$ — True
Worst p-value: 0.7150006546880893 — True
 - 7: $0.84451926747294+/-0$ — True
Worst p-value: 0.84451926747294 — True
 - 6: $1.0+/-0$ — True
Worst p-value: 1.0 — True
 - 5: $1.0+/-0$ — True
Worst p-value: 1.0 — True
 - 4: $0.7892680261342813+/-0$ — True
Worst p-value: 0.7892680261342813 — True
 - 3: $0.5270892568655381+/-0$ — True
Worst p-value: 0.5270892568655381 — True
 - 2: $0.6830913983096087+/-0$ — True
Worst p-value: 0.6830913983096087 — True
 - 1: $1.0+/-0$ — True
Worst p-value: 1.0 — True
 - +1: $\text{nan}+/-\text{nan}$ — False

Worst p-value: nan — False
+2: nan+/-nan — False
Worst p-value: nan — False
+3: nan+/-nan — False
Worst p-value: nan — False
+4: nan+/-nan — False
Worst p-value: nan — False
+5: nan+/-nan — False
Worst p-value: nan — False
+6: nan+/-nan — False
Worst p-value: nan — False
+7: nan+/-nan — False
Worst p-value: nan — False
+8: nan+/-nan — False
Worst p-value: nan — False
+9: nan+/-nan — False
Worst p-value: nan — False

KOLMOGOROV COMPLEXITY TEST

- (o) Universal Maurer Test: 0.0+/-0 — False
Worst p-value: 0.0 — False
- (o) Binary Matrix Test: 0.0+/-0 — False
Worst p-value: 0.0 — False
- (o) Linear Complexity Test: 0.8757492405097228+/-0 — True
Worst p-value: 0.8757492405097228 — True

Listing 2: Ibex

AQT Data Summary: Used 1 files

—> Entropy Estimations:

—> H_min = 0.8861561836445685+/-0 bits
 —> Worst H_min = 0.8861561836445685 bits
 —> H = 0.9951314556706178+/-0 bits
 —> Worst H = 0.9951314556706178 bits

—> Complexity Estimations:

—> Maximum Complexity = 9529.0+/-0 bits
 —> Worst Complexity = 9529.0 bits
 —> Deficiency Function = 56616.0+/-0
 —> Worst Deficiency Function = 56616.0
 —> p-value = 0.0+/-0 — False
 —> Worst p-value = 0.0 — False

—> NIST Battery Test:

—> FREQUENCY IN SEQUENCES

(o) Monobit_Test: (5.556310812077539+/-0)e-99 — False
 Worst p-value: 5.55631081207754e-99 — False
 (o) Within a Block: (1.5298904145808185+/-0)e-64 — False
 Worst p-value: 1.5298904145808184e-64 — False
 (o) Run Test: 0.0+/-0 — False
 Worst p-value: 0.0 — False
 (o) Longest One Block: (3.2393147599454863+/-0)e-11 — False
 Worst p-value: 3.239314759945486e-11 — False

SPECTRAL TESTS

(o) Spectral Test: 0.15223186643979453+/-0 — True
 Worst p-value: 0.15223186643979453 — True

STATISTICAL MECHANICS TESTS

(o) Approximate Entropy Test: (3.131480501949582+/-0)e-19 — False
 Worst p-value: 3.131480501949582e-19 — False
 (o) Cusum Test: (2.558673191294982+/-0)e-99 — False
 Worst p-value: 2.558673191294982e-99 — False
 (o) Random Excursions Test:
 -4: 0.9979031595107858+/-0 — True
 Worst p-value: 0.9979031595107858 — True
 -3: 0.9953295932358704+/-0 — True
 Worst p-value: 0.9953295932358704 — True
 -2: 0.984747879018509+/-0 — True
 Worst p-value: 0.984747879018509 — True
 -1: 0.9625657732472964+/-0 — True
 Worst p-value: 0.9625657732472964 — True

```

+1: 0.9625657732472964+/-0 — True
    Worst p-value: 0.9625657732472964 — True
+2: 0.024442548519677403+/-0 — True
    Worst p-value: 0.024442548519677403 — True
+3: 0.4381393147182503+/-0 — True
    Worst p-value: 0.4381393147182503 — True
+4: (7.7503068068224135+/-0)e-09 — False
    Worst p-value: 7.750306806822414e-09 — False
(o) Random Excursions Variant Test:
-9: nan+/-nan — False
    Worst p-value: nan — False
-8: nan+/-nan — False
    Worst p-value: nan — False
-7: nan+/-nan — False
    Worst p-value: nan — False
-6: nan+/-nan — False
    Worst p-value: nan — False
-5: nan+/-nan — False
    Worst p-value: nan — False
-4: nan+/-nan — False
    Worst p-value: nan — False
-3: nan+/-nan — False
    Worst p-value: nan — False
-2: nan+/-nan — False
    Worst p-value: nan — False
-1: 0.6170750774519739+/-0 — True
    Worst p-value: 0.6170750774519739 — True
+1: 0.6170750774519739+/-0 — True
    Worst p-value: 0.6170750774519739 — True
+2: 0.7728299926844475+/-0 — True
    Worst p-value: 0.7728299926844475 — True
+3: 0.5023349543605022+/-0 — True
    Worst p-value: 0.5023349543605022 — True
+4: 0.7054569861112734+/-0 — True
    Worst p-value: 0.7054569861112734 — True
+5: 1.0+/-0 — True
    Worst p-value: 1.0 — True
+6: 0.8801684549067255+/-0 — True
    Worst p-value: 0.8801684549067255 — True
+7: 1.0+/-0 — True
    Worst p-value: 1.0 — True

```

+8: 0.7962534147376392+/-0 — True

Worst p-value: 0.7962534147376392 — True

+9: 0.8083651559145103+/-0 — True

Worst p-value: 0.8083651559145103 — True

KOLMOGOROV COMPLEXITY TEST

(o) Universal Maurer Test: -1.0+/-0 — False

Worst p-value: -1.0 — False

(o) Binary Matrix Test: 0.31960749092921104+/-0 — True

Worst p-value: 0.31960749092921104 — True

(o) Linear Complexity Test: 0.7515788994206876+/-0 — True

Worst p-value: 0.7515788994206876 — True

Listing 3: Cepheus

Rigetti Data Summary: Used 1 files

—> Entropy Estimations:

—> H_min = 0.9222321461088729+/-0 bits
—> Worst H_min = 0.9222321461088729 bits
—> H = 0.9977862246429324+/-0 bits
—> Worst H = 0.9977862246429324 bits

—> Complexity Estimations:

—> Maximum Complexity = 1305454.0+/-0 bits
—> Worst Complexity = 1305454.0 bits
—> Deficiency Function = 8632257.0+/-0
—> Worst Deficiency Function = 8632257.0

—> p-value = 0.0+/-0 — False
—> Worst p-value = 0.0 — False

—> NIST Battery Test:

—> FREQUENCY IN SEQUENCES

(o) Monobit_Test: 0.0+/-0 — False
Worst p-value: 0.0 — False
(o) Within a Block: 0.0+/-0 — False
Worst p-value: 0.0 — False
(o) Run Test: 0.0+/-0 — False
Worst p-value: 0.0 — False
(o) Longest One Block: (1.2657900822677965+/-0)e-57 — False
Worst p-value: 1.2657900822677964e-57 — False

SPECTRAL TESTS

(o) Spectral Test: 0.22337891600865423+/-0 — True
Worst p-value: 0.22337891600865423 — True

STATISTICAL MECHANICS TESTS

(o) Approximate Entropy Test: 0.0+/-0 — False
Worst p-value: 0.0 — False
(o) Cusum Test: 0.0+/-0 — False
Worst p-value: 0.0 — False
(o) Random Excursions Test:
-4: 0.48699978051033865+/-0 — True
Worst p-value: 0.48699978051033865 — True
-3: 0.4730906268082202+/-0 — True
Worst p-value: 0.4730906268082202 — True
-2: 0.6914571220990702+/-0 — True
Worst p-value: 0.6914571220990702 — True
-1: 0.8038469749619624+/-0 — True
Worst p-value: 0.8038469749619624 — True

+1: 0.7311754392663888+/-0 — True
 Worst p-value: 0.7311754392663888 — True
 +2: 0.29141074964631325+/-0 — True
 Worst p-value: 0.29141074964631325 — True
 +3: 0.46182565332913994+/-0 — True
 Worst p-value: 0.46182565332913994 — True
 +4: 0.12897057037021184+/-0 — True
 Worst p-value: 0.12897057037021184 — True
 (o) Random Excursions Variant Test:
 -9: 0.6158978721857425+/-0 — True
 Worst p-value: 0.6158978721857425 — True
 -8: 0.26733319906854414+/-0 — True
 Worst p-value: 0.26733319906854414 — True
 -7: 0.12789124500121973+/-0 — True
 Worst p-value: 0.12789124500121973 — True
 -6: 0.20361888541370987+/-0 — True
 Worst p-value: 0.20361888541370987 — True
 -5: 0.15214293083791652+/-0 — True
 Worst p-value: 0.15214293083791652 — True
 -4: 0.14064517501718557+/-0 — True
 Worst p-value: 0.14064517501718557 — True
 -3: 0.4549751519736983+/-0 — True
 Worst p-value: 0.4549751519736983 — True
 -2: 0.8183578398322772+/-0 — True
 Worst p-value: 0.8183578398322772 — True
 -1: 0.7503158528871919+/-0 — True
 Worst p-value: 0.7503158528871919 — True
 +1: 0.524485879318823+/-0 — True
 Worst p-value: 0.524485879318823 — True
 +2: 0.4623977553582126+/-0 — True
 Worst p-value: 0.4623977553582126 — True
 +3: 0.3931705510376782+/-0 — True
 Worst p-value: 0.3931705510376782 — True
 +4: 0.24091600319065953+/-0 — True
 Worst p-value: 0.24091600319065953 — True
 +5: 0.17623224021150952+/-0 — True
 Worst p-value: 0.17623224021150952 — True
 +6: 0.4286117349551699+/-0 — True
 Worst p-value: 0.4286117349551699 — True
 +7: 0.5964205444568746+/-0 — True
 Worst p-value: 0.5964205444568746 — True

+8: 0.55137911813227+/-0 — True

Worst p-value: 0.55137911813227 — True

+9: 0.5757810610057631+/-0 — True

Worst p-value: 0.5757810610057631 — True

KOLMOGOROV COMPLEXITY TEST

(o) Universal Maurer Test: (2.0267541170187386+/-0)e-59 — False

Worst p-value: 2.026754117018739e-59 — False

(o) Binary Matrix Test: 0.2940431812180739+/-0 — True

Worst p-value: 0.2940431812180739 — True

(o) Linear Complexity Test: 0.14012025632442124+/-0 — True

Worst p-value: 0.14012025632442124 — True

Listing 4: Kingston

```

IBM Data Summary: Used 60 files
—> Entropy Estimations:
—> H_min = 0.9909+/-0.0005 bits
—> Worst H_min = 0.9808547529383512 bits
—> H = 1.0000+/-0.0005 bits
—> Worst H = 0.9998712640029751 bits
—> Complexity Estimations:
—> Maximum Complexity = 1315560+/-9 bits
—> Worst Complexity = 1315373.0 bits
—> Deficiency Function = 8684352+/-9
—> Worst Deficiency Function = 8684539.0
—> p-value = 0.0+/-0 — False
—> Worst p-value = 0.0 — False
—> NIST Battery Test:
—> FREQUENCY IN SEQUENCES
  (o) Monobit_Test: (5+/-5)e-07 — False
      Worst p-value: 1.6802121706699517e-280 — False
  (o) Within a Block: 0.010+/-0.010 — True
      Worst p-value: 7.518073571319198e-226 — False
  (o) Run Test: 0.0+/-0 — False
      Worst p-value: 0.0 — False
  (o) Longest One Block: 0.29+/-0.04 — True
      Worst p-value: 4.338323512796372e-08 — False
SPECTRAL TESTS
  (o) Spectral Test: (8+/-7)e-80 — False
      Worst p-value: 2.6544906774133083e-154 — False
STATISTICAL MECHANICS TESTS
  (o) Approximate Entropy Test: 0.005+/-0.004 — False
      Worst p-value: 6.062732103054007e-105 — False
  (o) Cusum Test: (4+/-4)e-07 — False
      Worst p-value: 3.099597781489997e-281 — False
  (o) Random Excursions Test:
      -4: 0.46+/-0.04 — True
          Worst p-value: 2.64000316051373e-06 — False
      -3: 0.50+/-0.04 — True
          Worst p-value: 0.00026465364966427317 — False
      -2: 0.55+/-0.04 — True
          Worst p-value: 0.003458298780615612 — False
      -1: 0.52+/-0.04 — True
          Worst p-value: 2.3187585803101154e-05 — False

```

+1: 0.598+/-0.034 — True
Worst p-value: 8.188962713339898e-05 — False

+2: 0.54+/-0.04 — True
Worst p-value: 0.004581562317697556 — False

+3: 0.49+/-0.04 — True
Worst p-value: 0.0011461968894336736 — False

+4: 0.50+/-0.04 — True
Worst p-value: 1.259114344723498e-07 — False

(o) Random Excursions Variant Test:

-9: 0.58+/-0.04 — True
Worst p-value: 7.717040577191723e-08 — False

-8: 0.55+/-0.04 — True
Worst p-value: 2.6785223425411122e-05 — False

-7: 0.55+/-0.04 — True
Worst p-value: 2.8666261254657593e-05 — False

-6: 0.55+/-0.04 — True
Worst p-value: 4.517450988213631e-07 — False

-5: 0.55+/-0.04 — True
Worst p-value: 1.1093369049350335e-06 — False

-4: 0.53+/-0.04 — True
Worst p-value: 1.901276281788005e-05 — False

-3: 0.54+/-0.04 — True
Worst p-value: 3.9401582725131885e-05 — False

-2: 0.54+/-0.04 — True
Worst p-value: 2.4586136714663832e-05 — False

-1: 0.52+/-0.04 — True
Worst p-value: 0.002183044737362255 — False

+1: 0.538+/-0.035 — True
Worst p-value: 0.0008561456864208789 — False

+2: 0.532+/-0.032 — True
Worst p-value: 0.0018471852434783177 — False

+3: 0.531+/-0.033 — True
Worst p-value: 0.0006250009784932352 — False

+4: 0.53+/-0.04 — True
Worst p-value: 9.96021973463293e-05 — False

+5: 0.55+/-0.04 — True
Worst p-value: 0.0002211706320908 — False

+6: 0.56+/-0.04 — True
Worst p-value: 0.0009111188771537128 — False

+7: 0.59+/-0.04 — True
Worst p-value: 0.0023190903681731367 — False

+8: $0.57+/-0.04$ — True

Worst p-value: 0.0010904572057621894 — False

+9: $0.53+/-0.04$ — True

Worst p-value: 0.0024318453627428568 — False

KOLMOGOROV COMPLEXITY TEST

(o) Universal Maurer Test: $0.48+/-0.04$ — True

Worst p-value: 0.00044865646624060805 — False

(o) Binary Matrix Test: $0.51+/-0.04$ — True

Worst p-value: 0.018466454673416247 — True

(o) Linear Complexity Test: $0.56+/-0.04$ — True

Worst p-value: 0.011159136136235604 — True

Listing 5: Statevector

Simulation Data Summary: Used 50 files

—> Entropy Estimations:

—> H_{min} = 0.99905+/-0.00009 bits
—> Worst H_{min} = 0.9970167079085166 bits
—> H = 1.00000+/-0.00009 bits
—> Worst H = 0.9999969090958023 bits

—> Complexity Estimations:

—> Maximum Complexity = 1315633+/-7 bits
—> Worst Complexity = 1315527.0 bits
—> Deficiency Function = 8684367+/-7
—> Worst Deficiency Function = 8684473.0
—> p-value = 0.0+/-0 — False
—> Worst p-value = 0.0 — False

—> NIST Battery Test:

—> FREQUENCY IN SEQUENCES

(o) Monobit_Test: 0.48+/-0.04 — True
Worst p-value: 0.01561133146977929 — True
(o) Within a Block: 0.47+/-0.05 — True
Worst p-value: 0.012862353769499252 — True
(o) Run Test: 0.50+/-0.04 — True
Worst p-value: 0.015376389842722339 — True
(o) Longest One Block: 0.51+/-0.04 — True
Worst p-value: 0.0008036226601178258 — False

SPECTRAL TESTS

(o) Spectral Test: 0.53+/-0.04 — True
Worst p-value: 0.026030185336883407 — True

STATISTICAL MECHANICS TESTS

(o) Approximate Entropy Test: 0.48+/-0.04 — True
Worst p-value: 0.02103140111619472 — True
(o) Cusum Test: 0.51+/-0.04 — True
Worst p-value: 0.01781834836811223 — True
(o) Random Excursions Test:
-4: 0.54+/-0.04 — True
Worst p-value: 1.4905888800398476e-07 — False
-3: 0.49+/-0.04 — True
Worst p-value: 0.002908753690234438 — False
-2: 0.53+/-0.04 — True
Worst p-value: 0.010347344021839108 — True
-1: 0.40+/-0.04 — True
Worst p-value: 0.013499639308818139 — True

+1: 0.393+/-0.034 — True
 Worst p-value: 0.033111258606628224 — True
 +2: 0.44+/-0.04 — True
 Worst p-value: 0.01883298067277677 — True
 +3: 0.41+/-0.04 — True
 Worst p-value: 0.02277549927573395 — True
 +4: 0.45+/-0.04 — True
 Worst p-value: 0.005942364971878184 — False
 (o) Random Excursions Variant Test:
 -9: 0.57+/-0.04 — True
 Worst p-value: 0.0067697040510309794 — False
 -8: 0.55+/-0.04 — True
 Worst p-value: 0.0041333238046909575 — False
 -7: 0.56+/-0.04 — True
 Worst p-value: 0.0027848612507980234 — False
 -6: 0.54+/-0.04 — True
 Worst p-value: 0.0005430464605858447 — False
 -5: 0.55+/-0.04 — True
 Worst p-value: 0.0005790632172192767 — False
 -4: 0.56+/-0.04 — True
 Worst p-value: 0.03095480557144473 — True
 -3: 0.55+/-0.04 — True
 Worst p-value: 0.06335310330466161 — True
 -2: 0.55+/-0.04 — True
 Worst p-value: 0.08904032277674373 — True
 -1: 0.47+/-0.04 — True
 Worst p-value: 0.029464713672882826 — True
 +1: 0.47+/-0.04 — True
 Worst p-value: 0.011124547193728929 — True
 +2: 0.44+/-0.04 — True
 Worst p-value: 0.006957208875786424 — False
 +3: 0.43+/-0.04 — True
 Worst p-value: 0.005169553367516743 — False
 +4: 0.44+/-0.05 — True
 Worst p-value: 0.002290239963384394 — False
 +5: 0.44+/-0.04 — True
 Worst p-value: 0.0006880412349851304 — False
 +6: 0.47+/-0.04 — True
 Worst p-value: 0.00035292404011109073 — False
 +7: 0.48+/-0.04 — True
 Worst p-value: 0.0013942543271505095 — False

+8: $0.52+/-0.04$ — True

Worst p-value: 0.00578180190814794 — False

+9: $0.56+/-0.04$ — True

Worst p-value: 0.0026636815394173426 — False

KOLMOGOROV COMPLEXITY TEST

(o) Universal Maurer Test: $0.48+/-0.04$ — True

Worst p-value: 0.014998186412165337 — True

(o) Binary Matrix Test: $0.45+/-0.04$ — True

Worst p-value: 0.021819388126449048 — True

(o) Linear Complexity Test: $0.42+/-0.04$ — True

Worst p-value: 0.03601233308141816 — True

Listing 6: BCrypt

BCryptGenRandom Data Summary: Used 50 files

—> Entropy Estimations:

—> H_{min} = 0.99876+/-0.00013 bits
 —> Worst H_{min} = 0.9959344483936639 bits
 —> H = 1.00000+/-0.00013 bits
 —> Worst H = 0.9999942554086683 bits

—> Complexity Estimations:

—> Maximum Complexity = 1315629+/-8 bits
 —> Worst Complexity = 1315499.0 bits
 —> Deficiency Function = 8684371+/-8
 —> Worst Deficiency Function = 8684501.0
 —> p-value = 0.0+/-0 — False
 —> Worst p-value = 0.0 — False

—> NIST Battery Test:

—> FREQUENCY IN SEQUENCES

(o) Monobit_Test: 0.45+/-0.04 — True
 Worst p-value: 0.047608016203377394 — True
 (o) Within a Block: 0.42+/-0.04 — True
 Worst p-value: 0.02482018475286391 — True
 (o) Run Test: 0.54+/-0.04 — True
 Worst p-value: 0.0318801403786231 — True
 (o) Longest One Block: 0.48+/-0.04 — True
 Worst p-value: 0.03565813213542178 — True

SPECTRAL TESTS

(o) Spectral Test: 0.49+/-0.04 — True
 Worst p-value: 0.01860036154796288 — True

STATISTICAL MECHANICS TESTS

(o) Approximate Entropy Test: 0.54+/-0.04 — True
 Worst p-value: 0.00753682701210032 — False
 (o) Cusum Test: 0.48+/-0.04 — True
 Worst p-value: 0.038363194153233776 — True
 (o) Random Excursions Test:
 -4: 0.48+/-0.04 — True
 Worst p-value: 0.02245741327459976 — True
 -3: 0.43+/-0.04 — True
 Worst p-value: 0.0031087144006727223 — False
 -2: 0.50+/-0.04 — True
 Worst p-value: 0.021871549553646077 — True
 -1: 0.49+/-0.04 — True
 Worst p-value: 0.01715735698944311 — True

+1: 0.49+/-0.04 — True
Worst p-value: 0.002204895668234438 — False

+2: 0.48+/-0.04 — True
Worst p-value: 0.004089029871584249 — False

+3: 0.45+/-0.04 — True
Worst p-value: 0.0026128246944936116 — False

+4: 0.50+/-0.04 — True
Worst p-value: 0.009560390803484259 — False

(o) Random Excursions Variant Test:

-9: 0.51+/-0.04 — True
Worst p-value: 0.006354165852817353 — False

-8: 0.47+/-0.04 — True
Worst p-value: 0.0038714952577722584 — False

-7: 0.46+/-0.04 — True
Worst p-value: 0.002910550633538461 — False

-6: 0.46+/-0.04 — True
Worst p-value: 0.002589017492617387 — False

-5: 0.47+/-0.04 — True
Worst p-value: 0.0031964089750470374 — False

-4: 0.51+/-0.04 — True
Worst p-value: 0.0037473906518201306 — False

-3: 0.48+/-0.04 — True
Worst p-value: 0.010696984005328848 — True

-2: 0.41+/-0.04 — True
Worst p-value: 0.005180343707620587 — False

-1: 0.41+/-0.04 — True
Worst p-value: 0.002390866415162272 — False

+1: 0.48+/-0.04 — True
Worst p-value: 0.022619786042556576 — True

+2: 0.50+/-0.04 — True
Worst p-value: 0.03290021000045662 — True

+3: 0.53+/-0.04 — True
Worst p-value: 0.015595332881048803 — True

+4: 0.49+/-0.04 — True
Worst p-value: 0.006622711701174252 — False

+5: 0.49+/-0.04 — True
Worst p-value: 0.007019984521690876 — False

+6: 0.51+/-0.04 — True
Worst p-value: 0.0030757558415633267 — False

+7: 0.52+/-0.04 — True
Worst p-value: 0.0030706768033579445 — False

+8: $0.51+/-0.04$ — True

Worst p-value: 0.0034430908721744713 — False

+9: $0.50+/-0.04$ — True

Worst p-value: 0.0027593280006589804 — False

KOLMOGOROV COMPLEXITY TEST

(o) Universal Maurer Test: $0.48+/-0.04$ — True

Worst p-value: 0.006390577520465176 — False

(o) Binary Matrix Test: $0.54+/-0.04$ — True

Worst p-value: 0.005175619560382332 — False

(o) Linear Complexity Test: $0.45+/-0.04$ — True

Worst p-value: 0.019118589297361554 — True

Listing 7: Segundo Orden para IBM

SECOND ORDER TESTS RESULTS (Based on KS distance and Shen's proposal)

Files from directories:

RNG: Outputs/IBM

Mixed: Mix_Sequences/Mixed_IBM

epsilon = 0.01 $\rightarrow$ K = 1.9549752159266272

- (o) Monobit Test: 1.0 — True
- (o) Frequency Within a Block Test: 0.98 — True
- (o) Run Test: 1.0 — True
- (o) Longest One Block Test: 0.30000000000000004 — True
- (o) Spectral Test: 1.0 — True
- (o) Approximate Entropy Test: 0.98 — True
- (o) Cusum Test: 1.0 — True
- (o) Maurer Test: 0.14 — True
- (o) Binary Matrix Test: 0.09999999999999998 — True
- (o) Linear Complexity Test: 0.16000000000000003 — True
- (o) Random Excursions Test:
 - −4: 0.16 — True
 - −3: 0.2 — True
 - −2: 0.24 — True
 - −1: 0.1 — True
 - +1: 0.28 — True
 - +2: 0.08000000000000002 — True
 - +3: 0.07999999999999996 — True
 - +4: 0.14 — True
- (o) Random Excursions Variant Test:
 - −9: 0.33999999999999997 — True
 - −8: 0.33999999999999997 — True
 - −7: 0.31999999999999995 — True
 - −6: 0.21999999999999997 — True
 - −5: 0.21999999999999997 — True
 - −4: 0.24 — True
 - −3: 0.19999999999999996 — True
 - −2: 0.21999999999999997 — True
 - −1: 0.14 — True
 - +1: 0.16000000000000003 — True
 - +2: 0.18 — True
 - +3: 0.29999999999999993 — True
 - +4: 0.18000000000000002 — True
 - +5: 0.19999999999999996 — True
 - +6: 0.26 — True

+7: 0.39999999999999997 — True
+8: 0.39999999999999997 — True

Listing 8: Segundo Orden para PCG64

SECOND ORDER TESTS RESULTS (Based on KS distance and Shen's proposal)

Files from directories:

RNG: Outputs/Simulation

Mixed: Mix_Sequences/Mixed_Sim

epsilon = 0.01 $\rightarrow$ K = 1.9549752159266272

- (o) Monobit Test: 0.16 — True
- (o) Frequency Within a Block Test: 0.13999999999999996 — True
- (o) Run Test: 0.14 — True
- (o) Longest One Block Test: 0.16000000000000003 — True
- (o) Spectral Test: 0.16000000000000003 — True
- (o) Approximate Entropy Test: 0.14 — True
- (o) Cusum Test: 0.1 — True
- (o) Maurer Test: 0.18 — True
- (o) Binary Matrix Test: 0.26000000000000006 — True
- (o) Linear Complexity Test: 0.28 — True
- (o) Random Excursions Test:
 - −4: 0.18000000000000005 — True
 - −3: 0.15999999999999998 — True
 - −2: 0.19999999999999996 — True
 - −1: 0.17999999999999994 — True
 - +1: 0.15999999999999992 — True
 - +2: 0.1 — True
 - +3: 0.16000000000000003 — True
 - +4: 0.19999999999999996 — True
- (o) Random Excursions Variant Test:
 - −9: 0.18 — True
 - −8: 0.2 — True
 - −7: 0.16000000000000003 — True
 - −6: 0.13999999999999999 — True
 - −5: 0.16000000000000003 — True
 - −4: 0.16000000000000003 — True
 - −3: 0.17999999999999994 — True
 - −2: 0.24000000000000005 — True
 - −1: 0.14000000000000007 — True
 - +1: 0.12 — True
 - +2: 0.15999999999999992 — True
 - +3: 0.32 — True
 - +4: 0.26 — True

+5: 0.160000000000000003 — True
+6: 0.14 — True
+7: 0.160000000000000003 — True
+8: 0.14 — True

Referencias

- [1] Akihiro Yamaguchi y Asaki Saito. «Second-level randomness test based on the Kolmogorov-Smirnov test». En: *JSIAM Letters* 14 (21 de oct. de 2022), págs. 73-76. DOI: [10.48550/arXiv.2110.08023](https://doi.org/10.48550/arXiv.2110.08023).
- [2] Alexander Shen. «Randomness Tests: Theory and Practice». En: *Fields of Logic and Computation III* 12180 (23 de mayo de 2020). DOI: https://doi-org.bua.idm.oclc.org/10.1007/978-3-030-48006-6_18.
- [3] Amazon Braket. *Amazon Braket Quotas*. Amazon Braket Developer Guide. URL: <https://docs.aws.amazon.com/braket/latest/developerguide/braket-quotas.html> (visitado 01-05-2026).
- [4] Amazon Braket. *Analog Hamiltonian Simulation*. Amazon Braket Developer Guide. URL: <https://docs.aws.amazon.com/braket/latest/developerguide/braket-analog-hamiltonian-simulation.html> (visitado 01-05-2026).
- [5] Amazon Braket. *Características de los dispositivos de Amazon Braket*. Amazon Braket. URL: <https://us-west-2.console.aws.amazon.com/braket/home?region=us-west-2#/devices> (visitado 01-05-2026).
- [6] Andrew L. Rukhin. «Statistical Testing of Randomness: New and Old Procedures». En: *Randomness through Computation: Some Answers, More Questions*. Gaithersburg, MD: World Scientific, págs. 33-51. ISBN: 978-981-4462-63-1.
- [7] AQT. *AQT: IBEX Q1*. URL: <https://www.aqt.eu/products/ibex-q1/> (visitado 12-05-2026).
- [8] AWS. *Amazon Braket*. URL: <https://aws.amazon.com/es/braket/>.
- [9] April Bacon. «Randomness and Cryptography». En: *Courant Newsletter* 14.2 (2019).
- [10] Stephen M. Barnett. *Quantum Information*. 1era edición. Oxford Master Series in Atomic, Optical and Laser Physics. Oxford, Reino Unido: Oxford University Press, 2009. ISBN: 978-0-19-852763-3.
- [11] Barron A. y T. Cover. «Minimum Complexity Density Estimation». En: *IEEE Transactions on Information Theory* 37.4 (jul. de 1991), págs. 1034-1054. URL: <https://isl.stanford.edu/~cover/papers/transIT/1034barr.pdf>.
- [12] Lawrence E. Bassham et al. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. 2010. URL: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=906762.
- [13] Bernardo Cascales et al. *Análisis Funcional*. Primera Edición. Textos Universitarios. Matemáticas 1. Murcia, España: Real Sociedad Matemática Española, 2013. ISBN: 978-84-940688-1-2.
- [14] Laurent Bienvenu et al. «Algorithmic tests and randomness with respect to a class of measures». En: *Proceedings of the Steklov Institute of Mathematics* 274 (16 de oct. de 2011), págs. 34-89. DOI: [10.1134/S0081543811060058](https://doi.org/10.1134/S0081543811060058).

- [15] Joseph Bowles, Marco Túlo Quintino y Nicolas Brunner. «Certifying the Dimension of Classical and Quantum Systems in a Prepare-and-Measure Scenario with Independent Devices». En: *Physical Review Letters* (2014).
- [16] Brian C.Hall. *Quantum Theory for Mathematicians*. Vol. 267. Graduate Texts in Mathematics 136. University of Notre Dame, IN, USA: Springer, 2013. ISBN: 978-1-4614-7116-5. DOI: [10.1007/978-1-4614-7116-5](https://doi.org/10.1007/978-1-4614-7116-5).
- [17] Christian Bustelo y Elisabeth Ortega. *CTS-2023-0055-TestQRNG*. HPCNow!, 15 de sep. de 2023. URL: https://www.cesga.es/wp-content/uploads/2023/10/informe_HPCNow.pdf.
- [18] Cameron Foreman, Richie Yeung y Florian J. Curchod. «Statistical Testing of Random Number Generators and Their Improvement Using Randomness Extraction». En: *Entropy* 26.12 (2024). DOI: <https://doi.org/10.3390/e26121053>.
- [19] Jian Cao et al. «Min-Entropy Estimation for Continuous-Variable Quantum Random Number Generators via Deep Neural Networks». En: *Advanced Quantum Technologies* 8.11 (nov. de 2025). DOI: <https://doi.org/10.1002/qute.202500072>.
- [20] David D.Awschalom et al. «Challenges and Opportunities for Quantum Information Hardware». En: *Science* 390.6777 (), págs. 1004-1010. DOI: [10.1126/science.adz8659](https://doi.org/10.1126/science.adz8659).
- [21] Bruno De Finetti. «Foresight: Its Logical Laws, Its Subjective Sources». En: *Annales de l'Institut Henri Poincaré* 7 (1937). URL: https://www.stat.unm.edu/~ronald/courses/Int_Bayes/definetti_exchangeability.pdf (visitado 22-02-2026).
- [22] Sean Devine. *Algorithmic Information Theory: Review For Physicists and Natural Scientists*. Victoria University of Wellington, New Zealand: IOP, 30 de mayo de 2014. ISBN: 978-0-7503-2638-4. URL: <https://cdn.jsdelivr.net/gh/it-ebooks-0/it-ebooks-2018-11to12/Algorithmic%20Information%20Theory%20-%20Review%20For%20Physicists%20And%20Natural%20Scientists.pdf>.
- [23] Rod Downey y Denis R. Hirschfeldt. «Algorithmic Randomness». En: *Proceedings of ACM Conference*. ACM Conference. ACM, New York, USA, 20 de mayo de 2019. URL: <https://www.math.uchicago.edu/~drh/Papers/Papers/algrand.pdf>.
- [24] Elvis Han Cui, Yihao Li y Zhuang Liu. *The Kolmogorov-Smirnov Statistic Revisited*. Universidad de California, Los Ángeles, 27 de feb. de 2025. DOI: <https://doi.org/10.48550/arXiv.2503.11673>.
- [25] Fatih Sulak et al. «On the independence of statistical randomness tests included in the NIST test suite». En: *Turkish Journal of Electrical Engineering and Computer Sciences* 25.5 (1 de ene. de 2017). DOI: [10.3906/elk-1605-212](https://doi.org/10.3906/elk-1605-212).
- [26] Cristiana Filippis y Mingione Giuseppe. «The sharp growth rate in nonuniformly elliptic Schauder theory». En: *Duke Math Journal* 174.9 (15 de jun. de 2025). DOI: [10.1215/00127094-2024-0075](https://doi.org/10.1215/00127094-2024-0075).
- [27] G. J. Chaitin. *Algorithmic Information Theory*. tercera edición. Yorktown Heights, Estados Unidos, 23 de nov. de 2009. ISBN: 978-0-511-60885-8. DOI: <https://doi.org/10.1017/CBO9780511608858>.

- [28] George C. Canavos. *Probabilidad Y Estadística: Aplicaciones y Métodos*. Trad. por Edmundo Gerardo Urbina Medal. primera edición. Florida, Estados Unidos: McGraw-Hill, 1998. 303-353. ISBN: 968-451-856-0.
- [29] George Marsaglia. *Random Numbers CDROM including the Diehard battery of tests of randomness*. Informe técnico. Universidad de Florida, 1995. URL: <https://ani.stat.fsu.edu/diehard/>.
- [30] Guillem Guigó i Corominas. *Applications of a Quantum Random Number Generator to simulations in condensed matter physics*. Barcelona, España, oct. de 2018.
- [31] Yassine Hamoudi. «Quantum Algorithms for the Monte Carlo Method». Tesis doctoral en Informática. Instituto de Investigación en Informática Fundamental: Universidad de Paris, jul. de 2021.
- [32] Hiroshi Haramoto y Makoto Matsumoto. *Again, random numbers fall mainly in the planes: xorshift128+ generators*. Hiroshima, Japón, 2019. DOI: <https://doi.org/10.48550/arXiv.1908.10020>.
- [33] P. Hellekalek. «Good Random Number Generators are (not so) Easy to Find». En: *Mathematic and Computers in Simulation* 46 (1998), págs. 485-505. DOI: [https://doi.org/10.1016/S0378-4754\(98\)00078-0](https://doi.org/10.1016/S0378-4754(98)00078-0).
- [34] Hiroshi Miura. *PyPPMd Python Module*. 2021. URL: <https://pyppmd.readthedocs.io/en/latest/index.html> (visitado 25-04-2026).
- [35] IBM. *IBM Quantum Platform*. URL: <https://quantum.cloud.ibm.com/>.
- [36] J.J. Sakurai y Jim Napolitano. *Modern Quantum Mechanics*. Segunda Edición. Cambridge, UK: Cambridge University Press. ISBN: 978-1-108-42241-3. DOI: [10.1017/9781108422413](https://doi.org/10.1017/9781108422413).
- [37] F. James. «A review of pseudorandom number generators». En: *Computer Physics Communications* 60.3 (oct. de 1990), págs. 329-344. DOI: [https://doi.org/10.1016/0010-4655\(90\)90032-V](https://doi.org/10.1016/0010-4655(90)90032-V).
- [38] Jan Balewski et al. «Engineering Quantum States with Neutral Atoms». En: *IEEE International Conference on Quantum Computing and Engineering* (2024). DOI: <https://doi.org/10.1109/QCE60285.2024.00144>.
- [39] Jeff Thompson. *DiehardCDROM*. Github. 2019. URL: <https://github.com/jeffThompson/DiehardCDROM> (visitado 24-03-2026).
- [40] Jonathan Wurtz et al. *Aquila Whitepaper*. QuEra Computing Inc., 16 de jun. de 2023. URL: <https://www.quera.com/aquila>.
- [41] Mehran Kardar. *Statistical Physics of Particles*. 1era edición. New York: Cambridge University Press, 2007. ISBN: 978-0-521-87342-0.
- [42] Kanish Karera, Areej Khan y Humanyou Tariq. «Construction Of a Quantum Random Number Generator». En: *4th International Conference on Innovations in Computer Science*. 2024. DOI: [10.1109/ICONICS64289.2024.10824418](https://doi.org/10.1109/ICONICS64289.2024.10824418).
- [43] Steven Kho Ang. *Randomness_testsuite*. Randomness_testsuite. URL: https://github.com/stevenang/randomness_testsuite (visitado 25-02-2026).

- [44] Donald Ervin Knuth. *The Art of Computer Programming*. séptima. Vol. 2. 4 vols. Addison-Wesley, 1998. ISBN: 13 978-0-201-89684-8.
- [45] Kohtaro Tadaki. *A Statistical Mechanical Interpretation of Algorithmic Information Theory*. Vol. 36. SpringerBriefs in Mathematical Physics 1. Chubu University, Japan: Springer Singapore, 2019. ISBN: 978-981-15-0738-0. DOI: <https://doi.org/10.1007/978-981-15-0739-7>.
- [46] Christian Kollmitzer et al. *Quantum Random Number Generation: Theory and Practice*. Switzerland: Springer, 2020. ISBN: 978-3-319-72596-3.
- [47] A. N. Kolmogorov. *Foundations of the Theory of Probability*. 2da edición. New York: Chelsea Publishing company. ISBN: 56-11512.
- [48] Pierre L'Ecuyer. *Testing Random Number Generators*. Montréal, Canada, 1992. DOI: [10.1145/167293.167354](https://doi.org/10.1145/167293.167354). (Visitado 13-02-2026).
- [49] Karel Lang. «On Measuring Randomness». Master's Thesis in computer science. Turku, Finlandia: Abo Akademi University. URL: https://www.doria.fi/bitstream/handle/10024/181285/lang_karel.pdf?sequence=2 (visitado 21-02-2026).
- [50] Y. Lin y F. Wang. «Linear Relaxation in Large Two-Dimensional Ising Models». En: *Physical Review* 93 (2016). DOI: [10.1103/PhysRevE.93.022113](https://doi.org/10.1103/PhysRevE.93.022113).
- [51] Tommaso Lunghi et al. «Self Testing Quantum Random Number Generator». En: *Physical Review Letters* 114 (2015).
- [52] *lzip 1.2.0*. pypi.org. 25 de nov. de 2022. URL: <https://pypi.org/project/lzip/> (visitado 25-04-2026).
- [53] Vaisakh Mannalatha, Sandeep Mishra y Anirban Pathak. «A comprehensive review of quantum random number generators: concepts, classification and the origin of randomness». En: *Quantum Inf Process* 22.439 (2023). DOI: <https://doi.org/10.1007/s11128-023-04175-y>. URL: <https://doi.org/10.1007/s11128-023-04175-y>.
- [54] George Marsaglia. «Random Numbers Fall Mainly in the Planes». En: *Proceedings of the National Academy of Sciences of the United States of America* 61 (1968), págs. 25-28.
- [55] Matthew Robinson. *Symmetry and The Standard Model: Mathematics and Particle Physics*. Springer, 2011. ISBN: 978-1-4419-8266-7. DOI: [10.1007/978-1-4419-8267-4](https://doi.org/10.1007/978-1-4419-8267-4).
- [56] Michael A. Nielsen e Isaac L. Chuang. *Quantum Computation and Quantum Information*. 6.ª ed. United Kingdom: Cambridge University Press, 2016. ISBN: 978-1-107-00217-3.
- [57] Satyadev Nandakumar. *Martin-Löf Randomness*. Kanpur, India, 1 de mar. de 2023. URL: <https://www.cse.iitk.ac.in/users/satyadev/sp24/mlrandom.pdf> (visitado 21-02-2026).
- [58] Nicu Neculache, Vlad-Andrei Petcu y Emil Simion. *A remark on the NIST 800-22 Binary Matrix Rank Test*. 2022. URL: https://www.researchgate.net/publication/357935696_A_remark_on_the_NIST_800-22_Binary_Matrix_Rank_Test.

- [59] Ning Ma y Heng Li. «Understanding and Estimating the Execution Time of Quantum Programs». En: (15 de nov. de 2025). DOI: <https://doi.org/10.48550/arXiv.2411.15631>.
- [60] Notezik. *Esfera de Bloch*. 2025.
- [61] NumPy. *Numpy.Random.BitGenerator.integers*. Numpy API Reference. URL: <https://numpy.org/doc/stable/reference/random/generated/numpy.random.Generator.integers.html> (visitado 28-04-2026).
- [62] M.E O'Neill. *PCG, A Family of Better Random Number Generators*. PCG, A Better Random Number Generator. 2026. URL: <https://www.pcg-random.org/index.html> (visitado 09-01-2026).
- [63] *os_Miscellaneous operating system interfaces*. Python 3.14.4 documentation. URL: <https://docs.python.org/3/library/os.html#os.urandom> (visitado 28-04-2026).
- [64] François Panneton y Pierre L'Ecuyer. *On the Xorshift Random Number Generators*. 23 de nov. de 2004. URL: <https://www.iro.umontreal.ca/~lecuyer/myftp/papers/xorshift.pdf>.
- [65] Pawel Lorek et al. «On testing pseudorandom generators via statistical tests based on the arcsine law». En: *Journal of Computational and Applied Mathematics* 380 (2020). DOI: [10.1016/j.cam.2020.112968](https://doi.org/10.1016/j.cam.2020.112968).
- [66] Peter D. Grünwald y Paul M.B. Vitányi. «Algorithmic Information Theory». En: *Philosophy of Information* 8 (2008).
- [67] Python Software Foundation. *gzip-Support for gzip files*. Docs.Python. 2001. URL: <https://docs.python.org/3/library/gzip.html> (visitado 25-04-2026).
- [68] Qiskit Development Team. *Qiskit AER*. Qiskit. Sep. de 2025. URL: <https://github.com/Qiskit/qiskit-aer> (visitado 18-05-2026).
- [69] *Random Generator*. Numpy API Reference. URL: <https://numpy.org/doc/stable/reference/random/generator.html>.
- [70] Reif, F. *Fundamentals of Statistical and Thermal Physics*. 1.^a ed. Waveland Press Inc., 2009. ISBN: 1-57766-612-7.
- [71] Richard Feynman, Robert Leighton y Matthew Sands. *Quantum Mechanics*. New Millennium Edition. Vol. 3. The Feynman Lectures on Physics. Caltech University, California, USA: Hachette Book Group, 2010. ISBN: 978-0-465-02417-9.
- [72] Rigetti. *Cepheids-1-108Q Quantum Processor*. Rigetti Systems. URL: <https://qcs.rigetti.com/qpus> (visitado 16-04-2026).
- [73] Robert G. Brown. *Dieharder test suite*. Robert G. Brown's General Tools Page. URL: <https://webhome.phy.duke.edu/~rgb/General/dieharder.php> (visitado 26-03-2026).
- [74] Scott Sheffield. «What is a random surface?» En: *Proceedings of the ICM contribution for 2022* (4 de mar. de 2022). DOI: <https://doi.org/10.48550/arXiv.2203.02470>.
- [75] Sebastiano Matarazzo. «From Classical to Quantum: Statistical Testing of Randomness». Master's Thesis in Cybersecurity. Padova, Italia: Universidad de Padova, 2025. URL: https://thesis.unipd.it/retrieve/48cb9112-498b-4284-acc2-6041e410c58c/Matarazzo_Sebastiano_Master_Thesis.pdf.

- [76] A. Shen. «Around Kolmogorov Complexity: Basic Notions and Results». En: *Measures of Complexity* (4 de sep. de 2015), págs. 75-115. DOI: https://doi.org/10.1007/978-3-319-21852-6_7.
- [77] Alexander Shen. *Making Randomness Tests More Robust*. Montpellier, Francia: ESCAPE-Systèmes Complexes, Automates et Pavages, 2018. URL: <https://hal.science/hal-01707610> (visitado 15-03-2026).
- [78] Jaideep Singh et al. *A Compact Quantum Random Number Generator Based on Balanced Detection of Shot Noise*. Abu Dhabi, United Arab Emirates, 30 de sep. de 2024.
- [79] Juan Soto. *Statistical Testing of Random Number Generators*. Gaithersburg, MD, 1999.
- [80] SpinQ Technology Inc. *Ultimate Guide to Coherence Time: Everything You Need To Know*. 16 de mayo de 2025. URL: <https://www.spinquanta.com/news-detail/ultimate-guide-to-coherence-time> (visitado 16-04-2026).
- [81] Sushil Jajodia, Pierangela Samarati y Moti Yung. *Linear Feedback Shift Register*. En: *Encyclopedia of Cryptography, Security and Privacy*. Springer, Cham, 10 de mayo de 2025, págs. 1428-1431. ISBN: 978-3-030-71520-5. DOI: https://doi.org/10.1007/978-3-030-71522-9_357.
- [82] *The Basic Logic of Quantum Mechanics*. Col. de Leonard Susskind. Stanford University, California, USA., 2012. URL: <https://theoreticalminimum.com/courses/quantum-mechanics/2012/winter/lecture-2>.
- [83] Tom Lancaster y Stephen J. Blundell. *Quantum Field Theory for the Gifted Amateur*. Primera Edición. Oxford, Reino Unido: Oxford University Press, 2014. ISBN: 978-0-19-969932-2.
- [84] Matthew Treinish. *qiskit 2.3.0 documentation*. Python Package Index Blog. 2026. URL: <https://pypi.org/project/qiskit/> (visitado 10-02-2026).
- [85] Amos Tversky y Daniel Kahneman. «Judgment under Uncertainty: Heuristics and Biases». En: *Nature Science* 185.4157 (27 de sep. de 1974), págs. 1124-1131. DOI: [DOI:10.1126/science.185.4157.1124](https://doi.org/10.1126/science.185.4157.1124).
- [86] Valshnavi Kumar. «Experimenting with Quantum True Random Number Generators on NISQ Computers Using High-level Quantum Programming». En: *International Journal of Theoretical Physics* 64.238 (3 de sep. de 2025). DOI: <https://doi-org.bua.idm.oclc.org/10.1007/s10773-025-06104-4>.
- [87] *Welcome to the docs for pyQuil*. pyQuil. 2021. URL: <https://pyquil-docs.rigetti.com/en/stable/>.